

Senior Engineer's Complete Compendium

Three domains. Every concept. Explained from first principles — with Mermaid diagrams you can render anywhere.

Paste any diagram block into mermaid.live for an interactive render. Mermaid is also natively supported in GitHub markdown, GitLab, Notion, Confluence, and VS Code (Mermaid Preview extension).

Table of Contents

Part 1 — Java and JVM Internals Java Memory Model · Happens-Before · Volatile · Instruction Reordering · Escape Analysis · False Sharing · CAS · ABA Problem · AQS · ForkJoinPool · Virtual Threads · Semaphores · GC Roots · G1 GC · Object Header Layout · Type Erasure · Bridge Methods · Double-Checked Locking · readResolve · serialVersionUID · StampedLock · NIO Selector · DirectByteBuffer · ClassLoader · JIT Inlining · Deoptimisation · @Transactional Proxy · CGLIB · MethodHandles · VarHandle · TLAB · Sealed Classes · Lock Inflation · Write Barriers · DispatcherServlet · Stream Short-Circuiting · Spliterator · Try-With-Resources

Part 2 — Databases and Distributed Systems Indexing · Clustering · Normalisation · Read Replicas · Replication Modes · Quorum · Consensus · CAP Theorem · BASE vs ACID · MVCC · Snapshot Isolation · 2PC and 3PC · WAL · Checkpointing · Compaction · Bloom Filter · LSM Tree vs B-Tree · Query Planner · Deadlock · Lock Types · Isolation Anomalies · Backpressure · Circuit Breaker · Rate Limiting · CDC · Consistent Hashing · Partitioning · Idempotency · Exactly-Once Semantics

Part 3 — Frontend Engineering Hydration · Islands Architecture · Streaming SSR · Concurrent Rendering · Fiber Architecture · Reconciliation · Virtual DOM · Structural Sharing · Memoization · Stale Closures · Event Loop · Layout Thrashing · Critical Rendering Path · Tree Shaking · Code Splitting · Web Workers vs Service Workers · SharedArrayBuffer · Browser Compositing · GPU Acceleration · Service Worker Lifecycle · Cache Strategies · CORS · CSP · Suspense · Selective Hydration · Server Components · Web Vitals · CRDTs · AbortController · Memory Leaks · Accessibility Tree · ARIA Live Regions

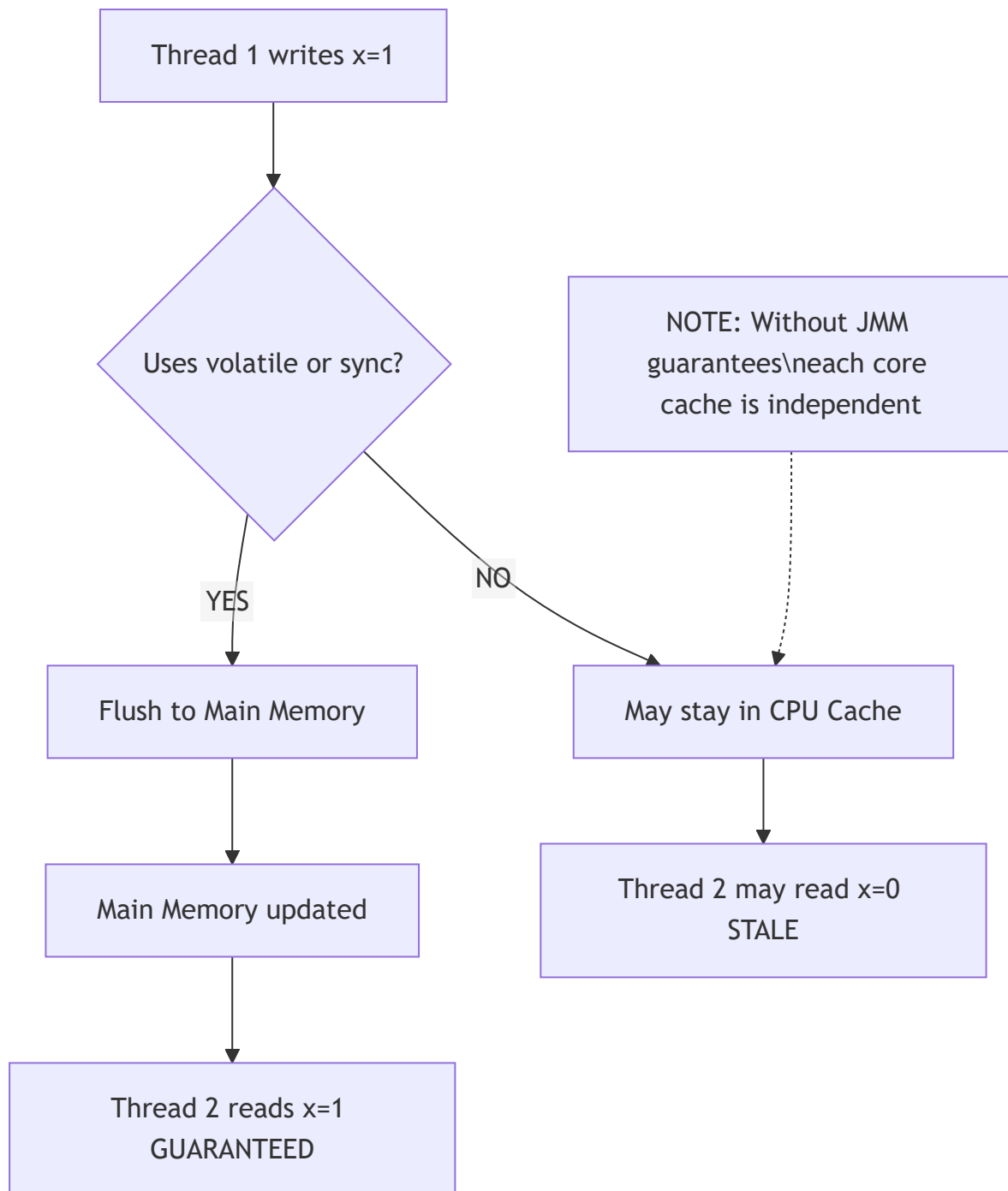


Part 1 — Java and JVM Internals

Java Memory Model (JMM)

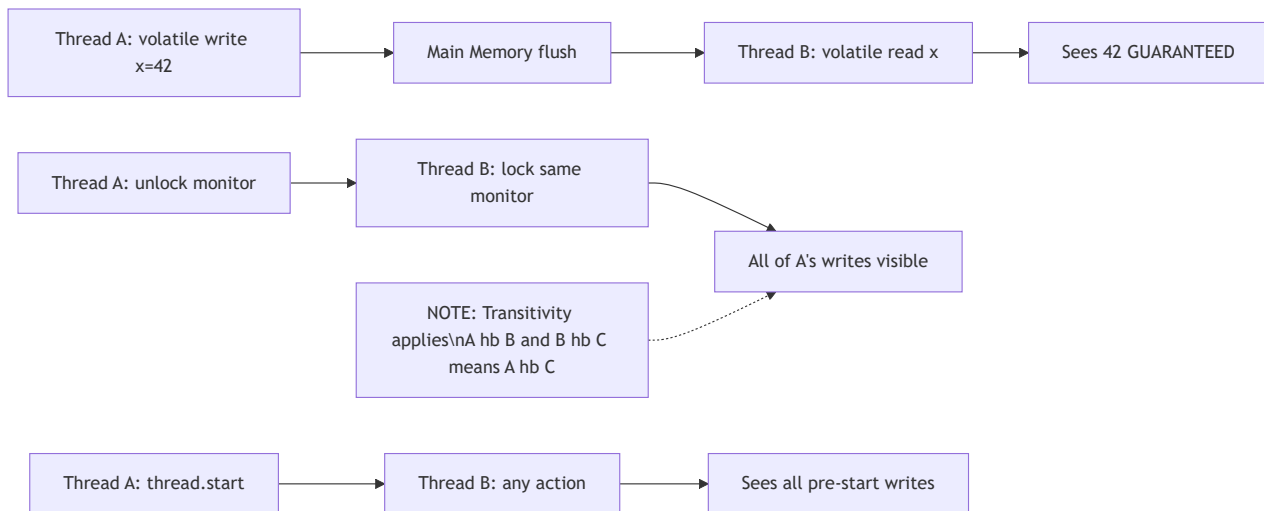
The JMM is the contract between your Java program and the hardware beneath it. Modern CPUs keep per-core caches, so two threads can simultaneously hold different values for the same variable — and both are technically "correct" from their cache's perspective. The JMM doesn't describe hardware behaviour; it defines what the language **guarantees** when you use synchronisation primitives correctly. Without it, the runtime is free to let a thread read from its stale cache forever. Think of it as the rules of a postal

system: threads are cities, and the JMM defines when a letter (a write) is guaranteed to arrive at another city (another thread).



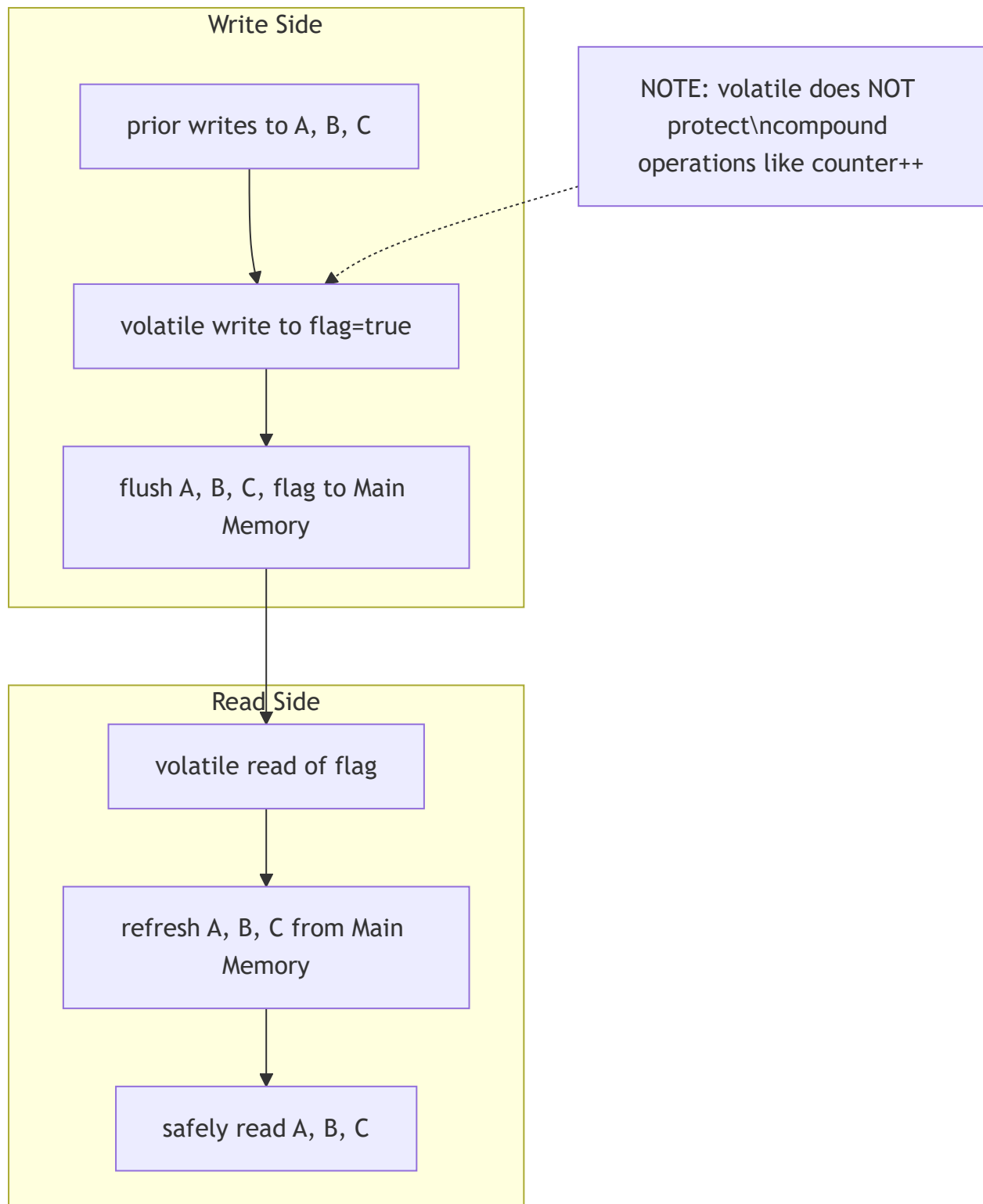
Happens-Before Relationship

Happens-before is the formal visibility guarantee mechanism inside the JMM. If action A happens-before action B, everything A wrote is **guaranteed visible** to B. This is not about physical time — it is about visibility promises. Key rules: an unlock of a monitor happens-before every subsequent lock of that same monitor; a `volatile` write happens-before every subsequent read of the same field; `thread.start()` happens-before any action in the started thread. The relationship is transitive: if A hb B and B hb C, then A hb C. If no happens-before edge exists between your write and someone else's read, the JVM makes no visibility promise at all.



Volatile Visibility Guarantees

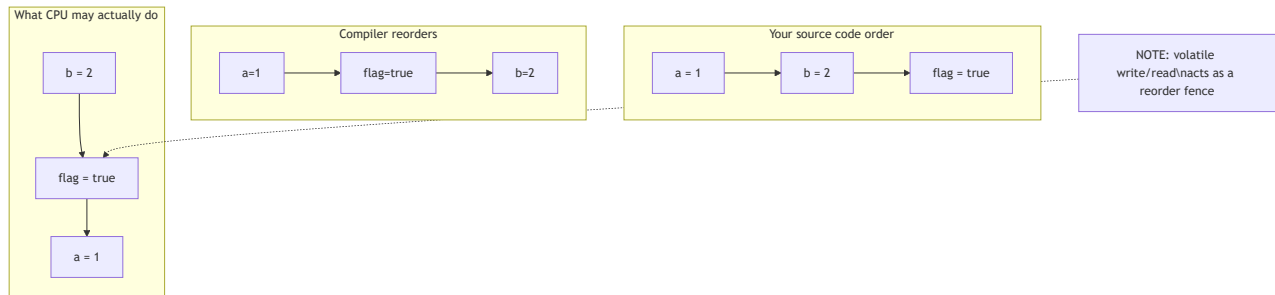
Declaring a field `volatile` gives two guarantees: **visibility** and **ordering**, but not compound atomicity. On write, the value is immediately flushed to main memory. On read, the value is always fetched from main memory — never from a thread-local cache. This eliminates stale reads. The ordering guarantee acts as a memory fence: all writes before the volatile write are visible to any thread that subsequently reads the volatile field. What volatile *cannot* do is protect compound operations. A `volatile counter++` is still three steps — read, increment, write — and two threads can interleave them, losing an update. For that you need `AtomicInteger` or synchronisation.



Instruction Reordering

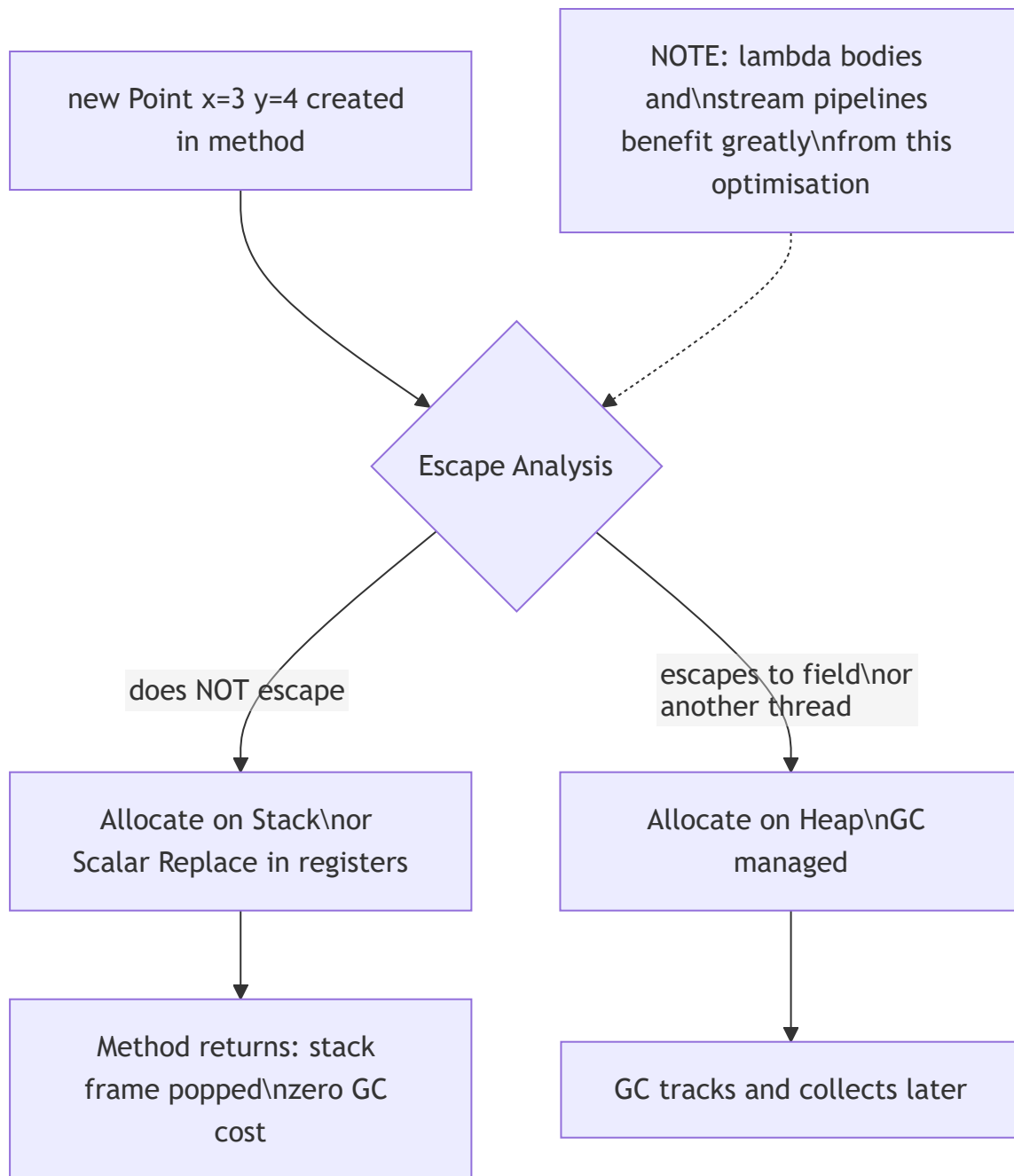
Both the compiler and the CPU are free to reorder instructions as long as the result appears correct from a single-threaded perspective. For a single thread this is safe; in a multi-threaded world it is catastrophic. Three sources exist: compiler reordering (the JIT moves instructions for better register use), CPU out-of-order execution (the chip executes instructions in whatever sequence maximises throughput), and memory system reordering (store buffers make writes visible to other cores out of order). The JMM defines exactly

which reorderings are permitted and which are forbidden across synchronisation boundaries, giving you predictable behaviour when you use the correct primitives.



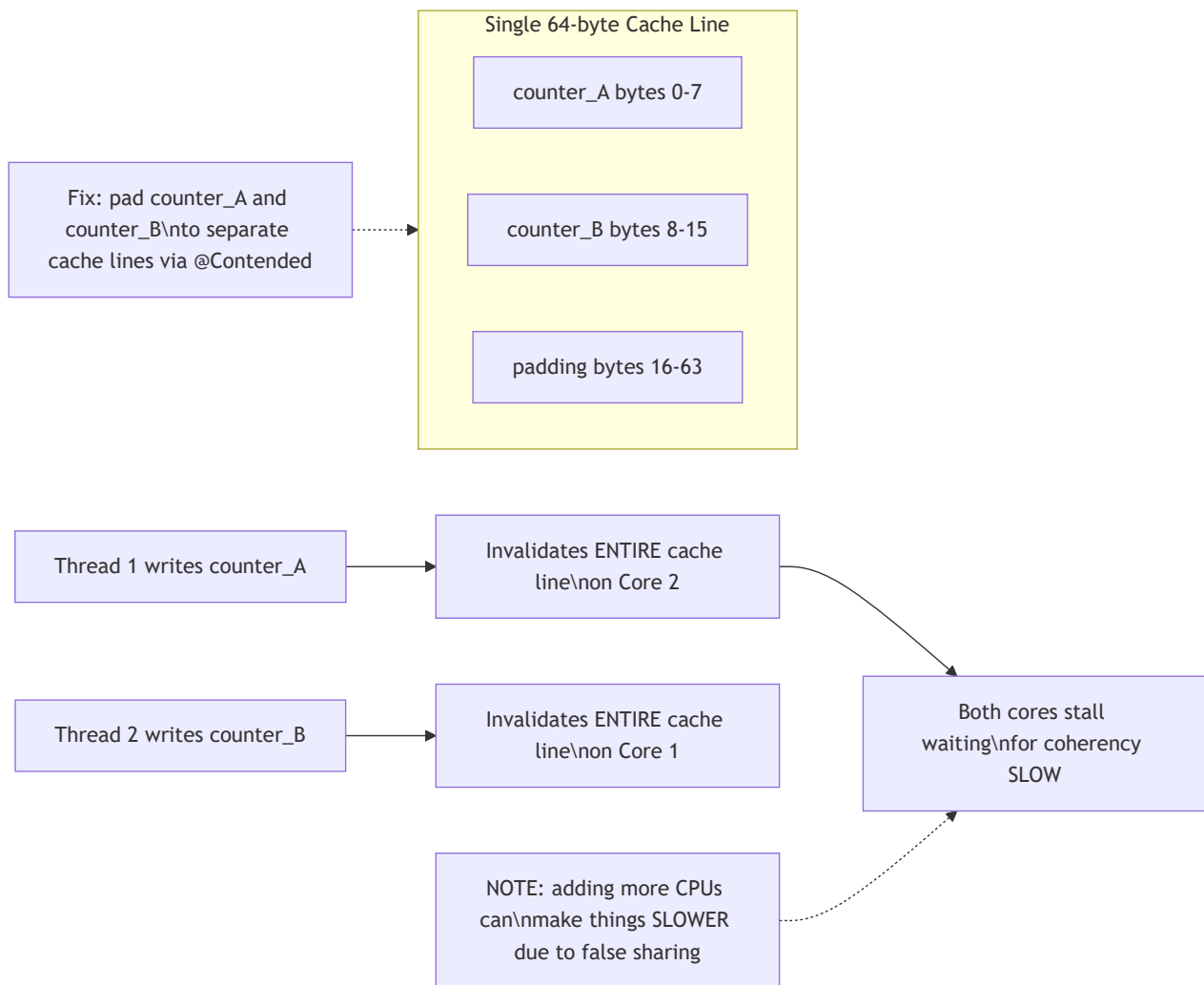
Escape Analysis

Escape analysis is a JIT optimisation that asks: does this object's reference ever leave the current method or thread? If it does not escape, the JVM can allocate it on the thread's stack instead of the heap — stack allocation is essentially free and requires no garbage collection. Even better, the JIT can *scalar-replace* the object: decompose it into its individual fields, stored in CPU registers, and never create an object at all. This is why creating small, short-lived objects in Java is often cheaper than developers expect. The practical lesson is to let the JIT do its job rather than pre-optimising by avoiding object creation.



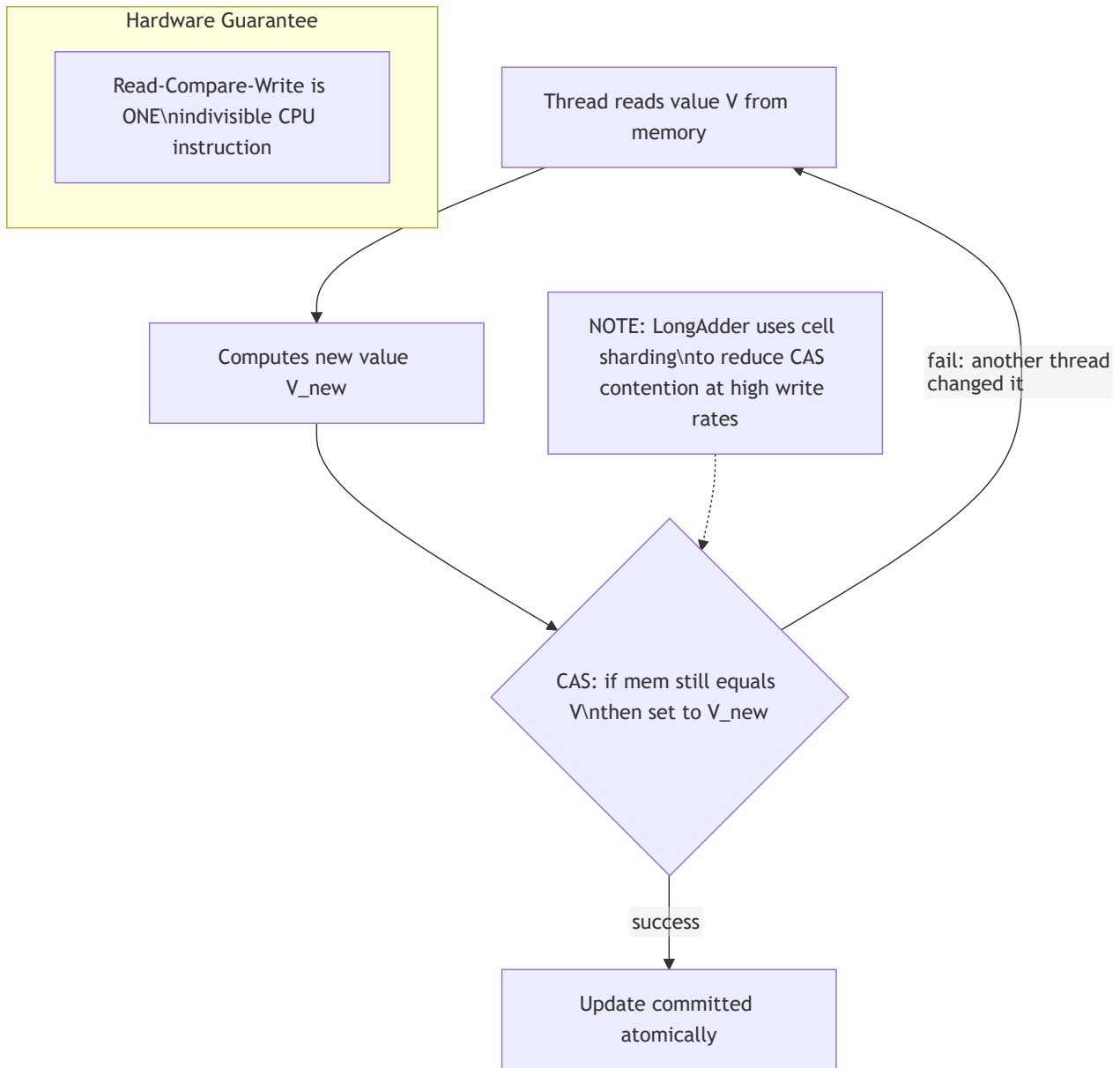
False Sharing

CPUs load memory in cache lines, typically **64 bytes** at a time. False sharing occurs when two threads write to logically independent variables that happen to reside in the same 64-byte cache line. Even though the variables are unrelated, the CPU cache coherency protocol treats them as a single shared unit. Every write by one core invalidates the cache line on all other cores, forcing a re-fetch. The result is that two supposedly independent counters can run slower than a single contended counter. Padding — placing enough dummy bytes between hot fields — fixes this. Java provides the `@Contended` annotation inside `sun.misc` to automate the padding.



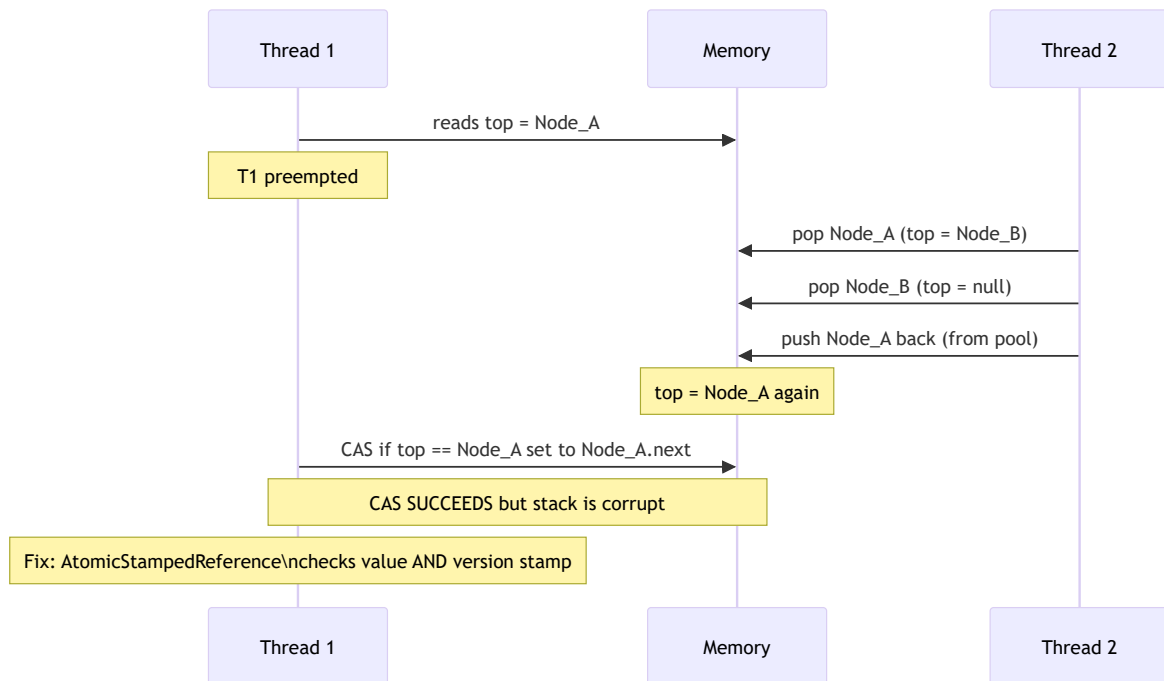
CAS — Compare-And-Swap

CAS is a hardware atomic instruction: if the value at this memory address equals the expected value, replace it with the new value — atomically. It returns whether the swap succeeded. This single instruction is the foundation of all lock-free programming in Java. `AtomicInteger.incrementAndGet()` works as a CAS loop: read current value, compute new value, attempt CAS. If another thread changed the value in between, CAS fails and the loop retries. No locks, no context switches, no blocking. Under low contention this is extremely fast. Under high contention the spinning burns CPU — which is why `LongAdder` shards counters across cells and aggregates only on read.



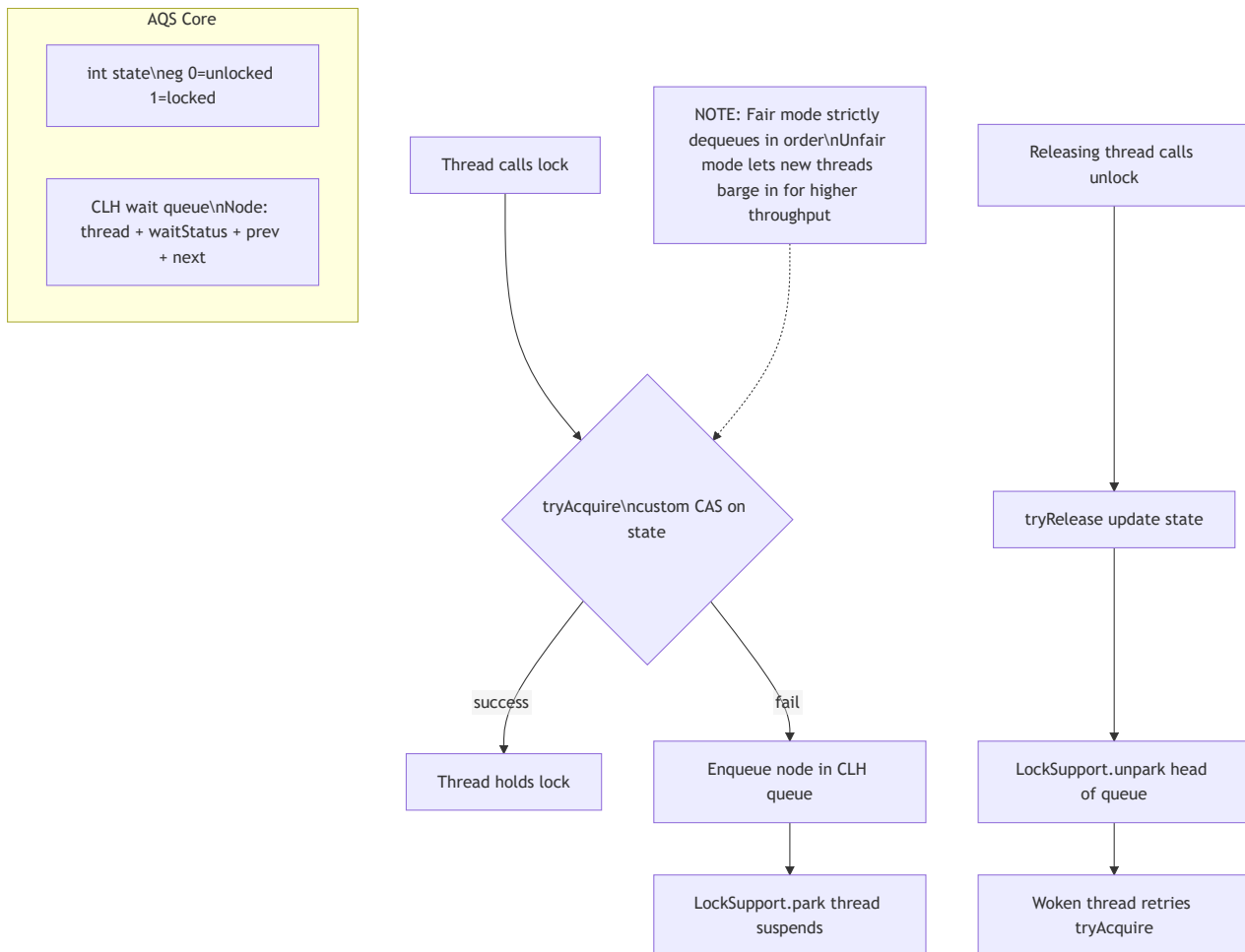
ABA Problem

The ABA problem is a subtle flaw in CAS-based algorithms. Thread 1 reads value A. It is preempted. Thread 2 changes A to B, then back to A. Thread 1 resumes and its CAS succeeds — the memory still reads A. But the object at that address may have been freed and reallocated, or the data structure may have changed, even though the pointer looks the same. The canonical example is a lock-free stack: if nodes are recycled from a pool, the top pointer can return to a previously seen address while the stack's interior is completely different. The fix is `AtomicStampedReference` — every CAS also checks a version stamp that increments on every update.



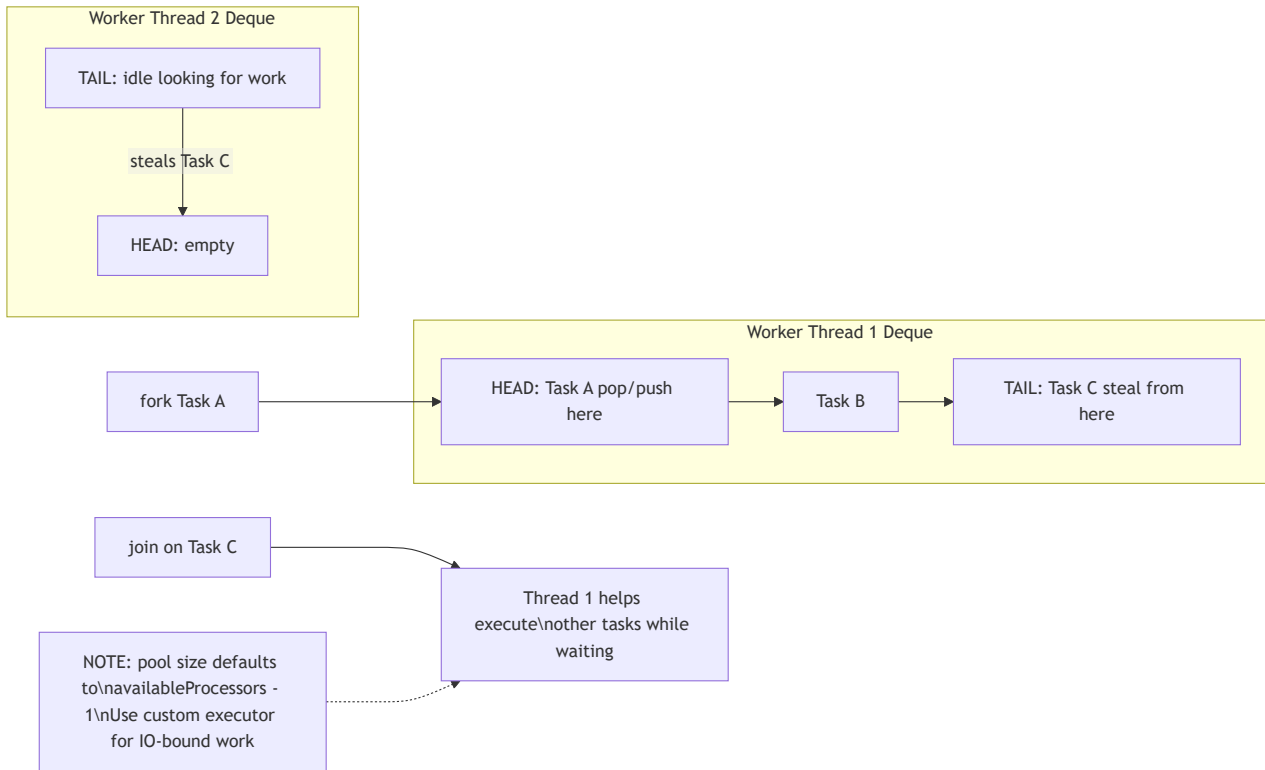
AQS — AbstractQueuedSynchronizer Internals

AQS is the backbone of Java's concurrency toolkit. `ReentrantLock`, `Semaphore`, `CountDownLatch`, and `ReadWriteLock` all build on it. It provides two things: an integer state field and a FIFO queue of waiting threads. The meaning of state is defined by the subclass — for `ReentrantLock`, 0 means unlocked and positive values count reentrancy; for `Semaphore`, state is the permit count. AQS gives you CAS on state and you override `tryAcquire` and `tryRelease`. When a thread cannot acquire, AQS parks it using `LockSupport.park()` and enqueues a node in a CLH-variant linked list. Release unparks the head of the queue. Fair vs unfair modes differ only in whether a new thread can barge in ahead of the queue.



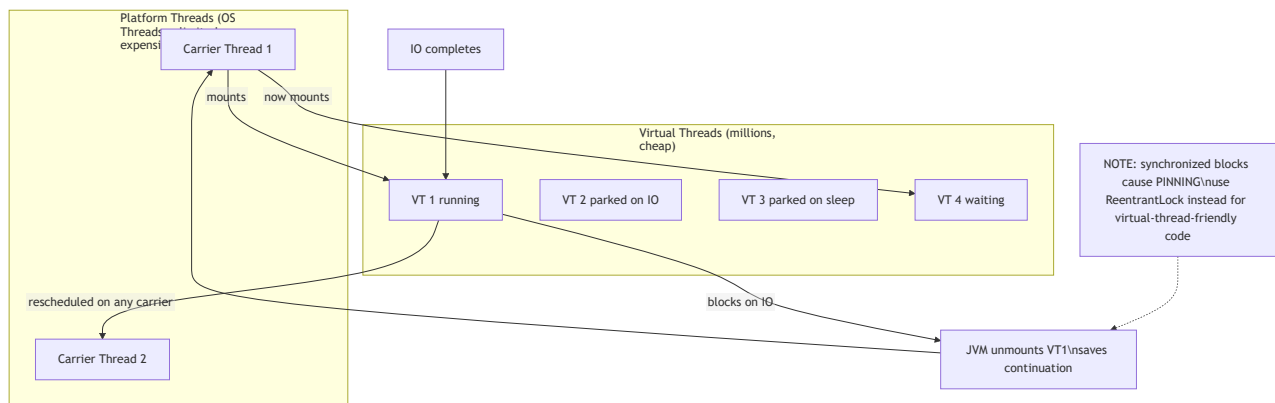
ForkJoinPool

ForkJoinPool is designed for recursive divide-and-conquer parallelism. Its core innovation is **work stealing**: each worker thread owns a double-ended deque of tasks. The owner pushes and pops from the head in LIFO order — good for cache locality. When idle, a thief steals from the tail of another thread's deque in FIFO order — good for fairness and minimising contention. `fork()` submits a subtask to the current thread's deque. `join()` waits, but rather than blocking, the waiting thread helps execute other pending tasks — this is task-joining compensation and prevents thread starvation under deep recursion. `Stream.parallel()` and `CompletableFuture.supplyAsync()` both use the common ForkJoinPool.



Virtual Threads

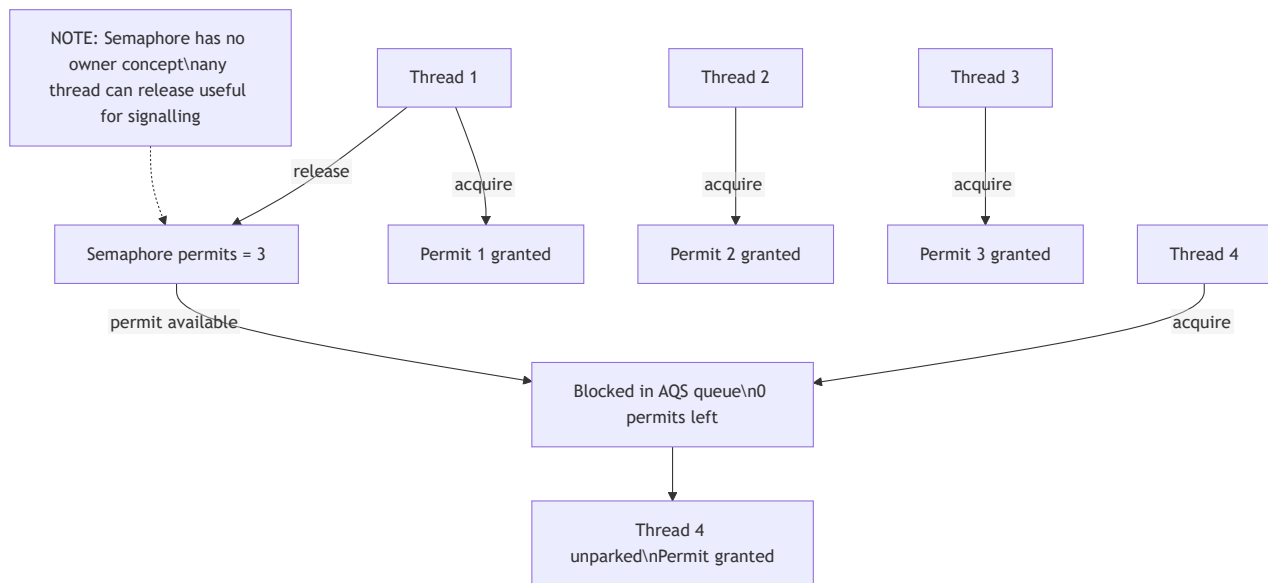
Virtual threads, introduced in Java 21 via Project Loom, are JVM-managed lightweight threads. A platform thread maps one-to-one with an OS thread and requires roughly one megabyte of stack. A virtual thread is **multiplexed**: when it blocks on I/O, sleep, or a lock, the JVM unmounts it from its carrier OS thread and parks its continuation — a snapshot of the call stack. The freed carrier thread immediately picks up another virtual thread. You can have millions of virtual threads with negligible memory overhead. Crucially, you write blocking code as normal — `Thread.sleep()`, blocking I/O — and the JVM transparently converts it to non-blocking. The main caveat is **pinning**: holding a synchronized lock while blocking pins the virtual thread to its carrier, negating the benefit.



Semaphores

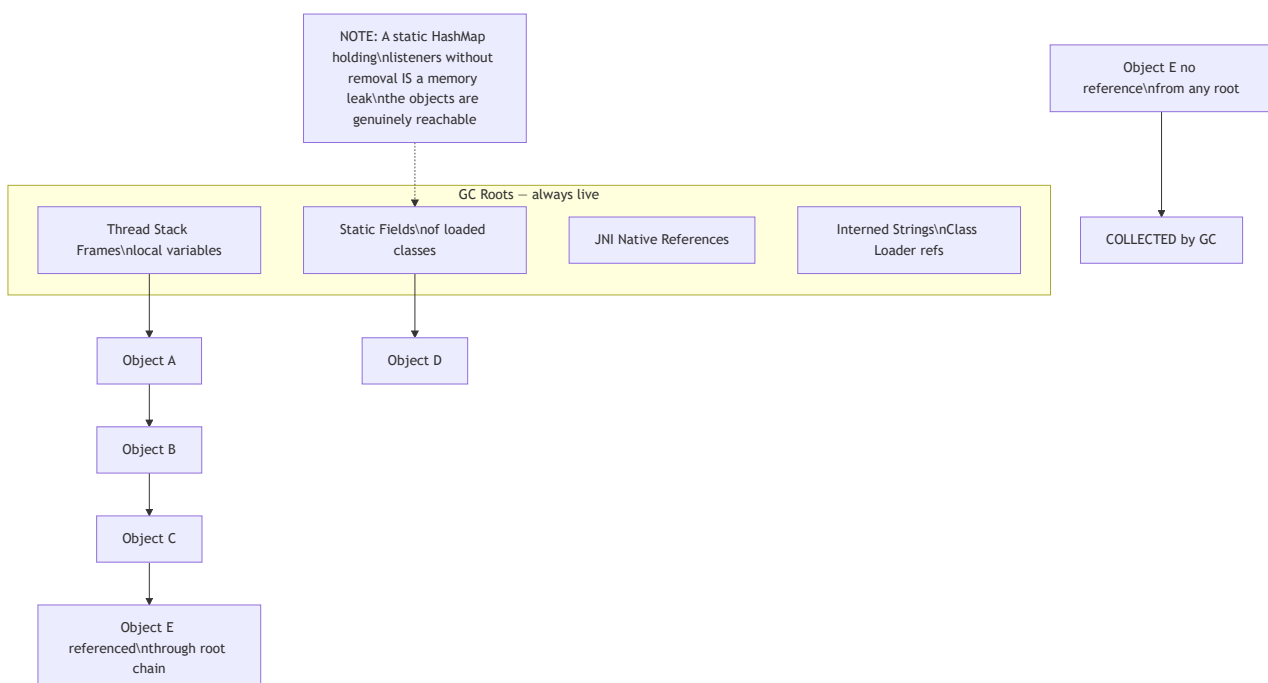
A semaphore holds a count of **permits**. A thread must acquire a permit to proceed; if none are available it blocks until one is released. Think of a parking lot with N spaces: the semaphore is the gate. This is perfect for rate-limiting access to bounded resources —

database connections, API calls, file handles. Semaphore(1) behaves like a mutex, but with a critical difference: any thread can release it, unlike an owning lock. This makes semaphores suitable for producer-consumer signalling where one thread signals another. Java's Semaphore class is built on AQS, so it inherits fair and unfair modes.



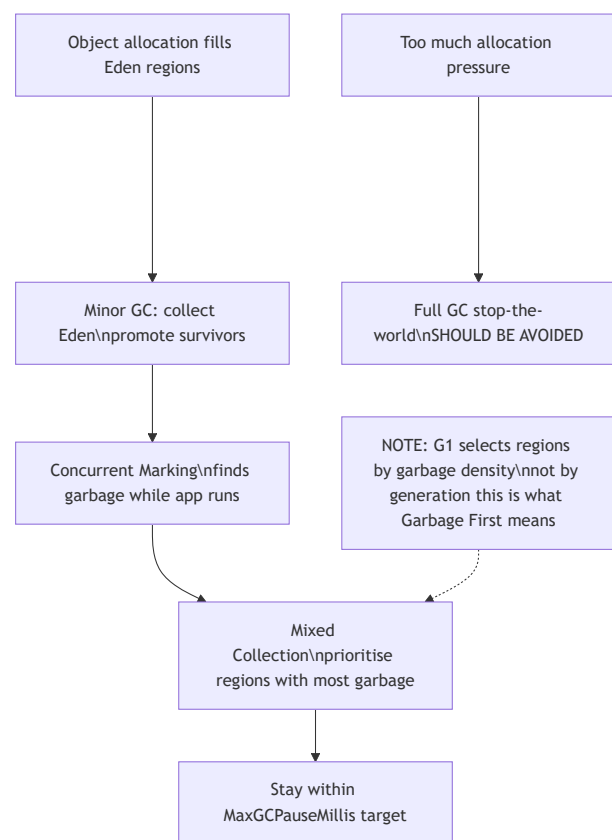
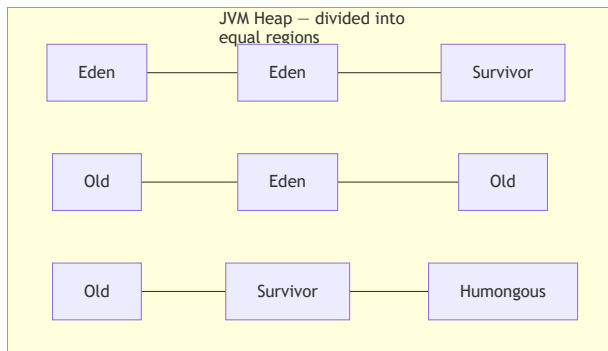
GC Roots

GC roots are the starting points for garbage collection's reachability analysis. The collector traces all object references starting from roots; any object not reachable from a root is garbage. The roots are: local variables and operand stacks in all active thread stack frames; static fields of all loaded classes; JNI native references; references held by JVM internals such as class loaders, interned strings, and synchronised monitors. Active threads themselves are roots — any object reachable from a running thread's stack survives. Memory leaks in Java usually mean you accidentally kept a reference in a long-lived structure without ever removing it. The GC correctly keeps it alive because it is reachable — the problem is your code, not the GC.



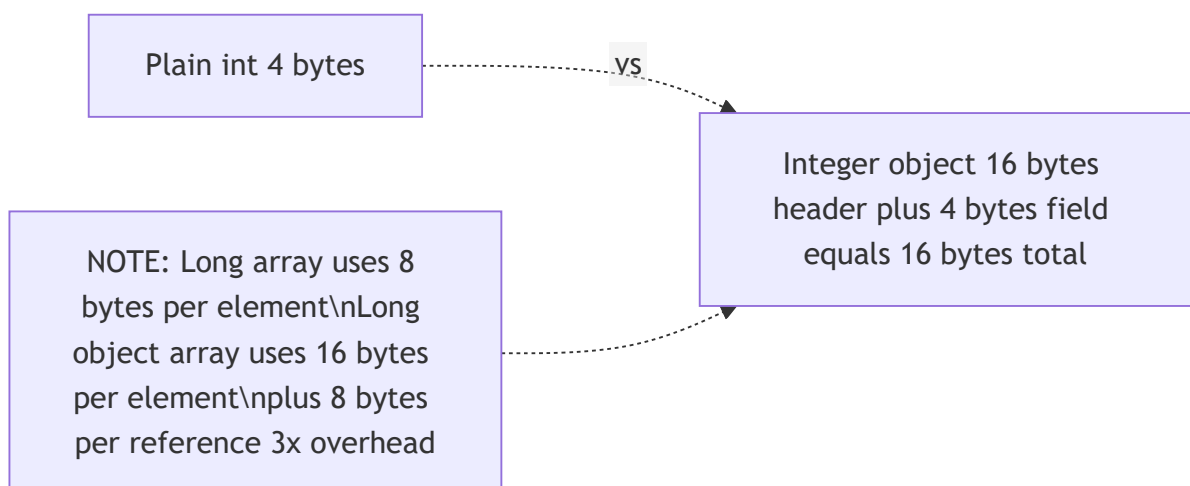
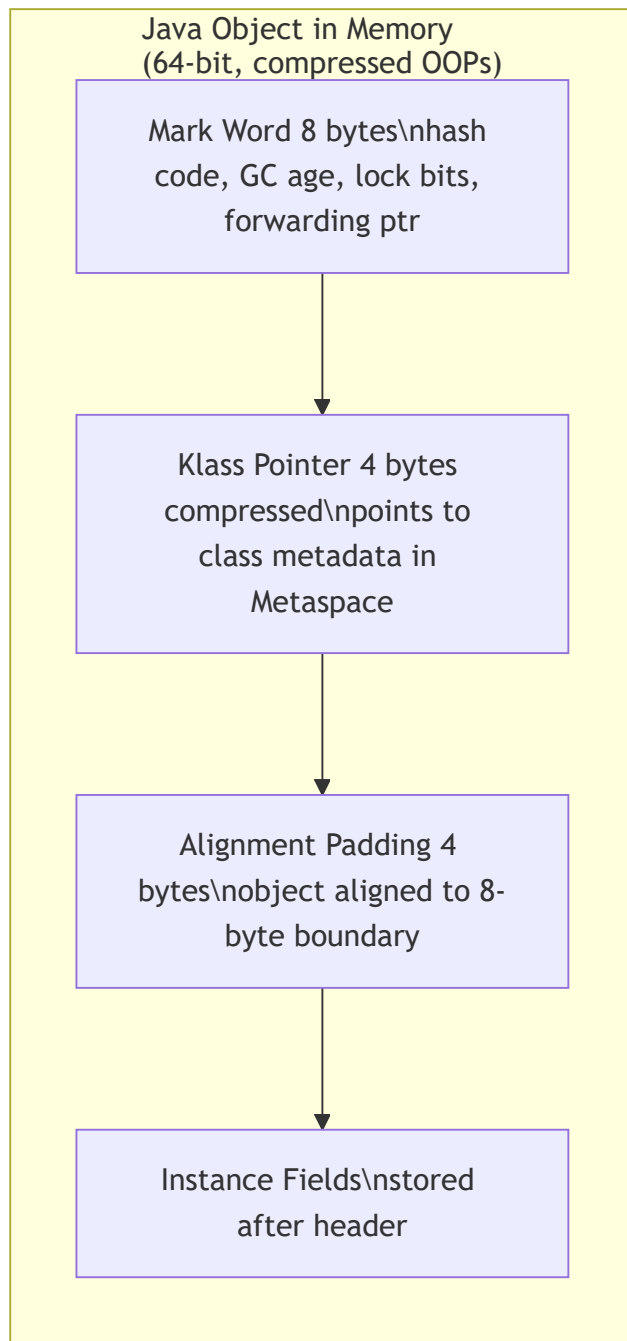
G1 GC

G1 (Garbage First), the default collector since Java 9, divides the heap into equal-sized regions of one to thirty-two megabytes each. Unlike older collectors with fixed young and old spaces, each region is labelled dynamically as Eden, Survivor, or Old. G1 tracks the density of garbage in every region. When it collects, it prioritises the regions with the most garbage — hence *Garbage First* — maximising space reclaimed per millisecond of pause. The key tuning parameter is `MaxGCPauseMillis`: G1 will select however many regions it can collect while staying within your target pause time. The danger is falling back to a stop-the-world full GC when your application generates garbage faster than G1 can collect it.



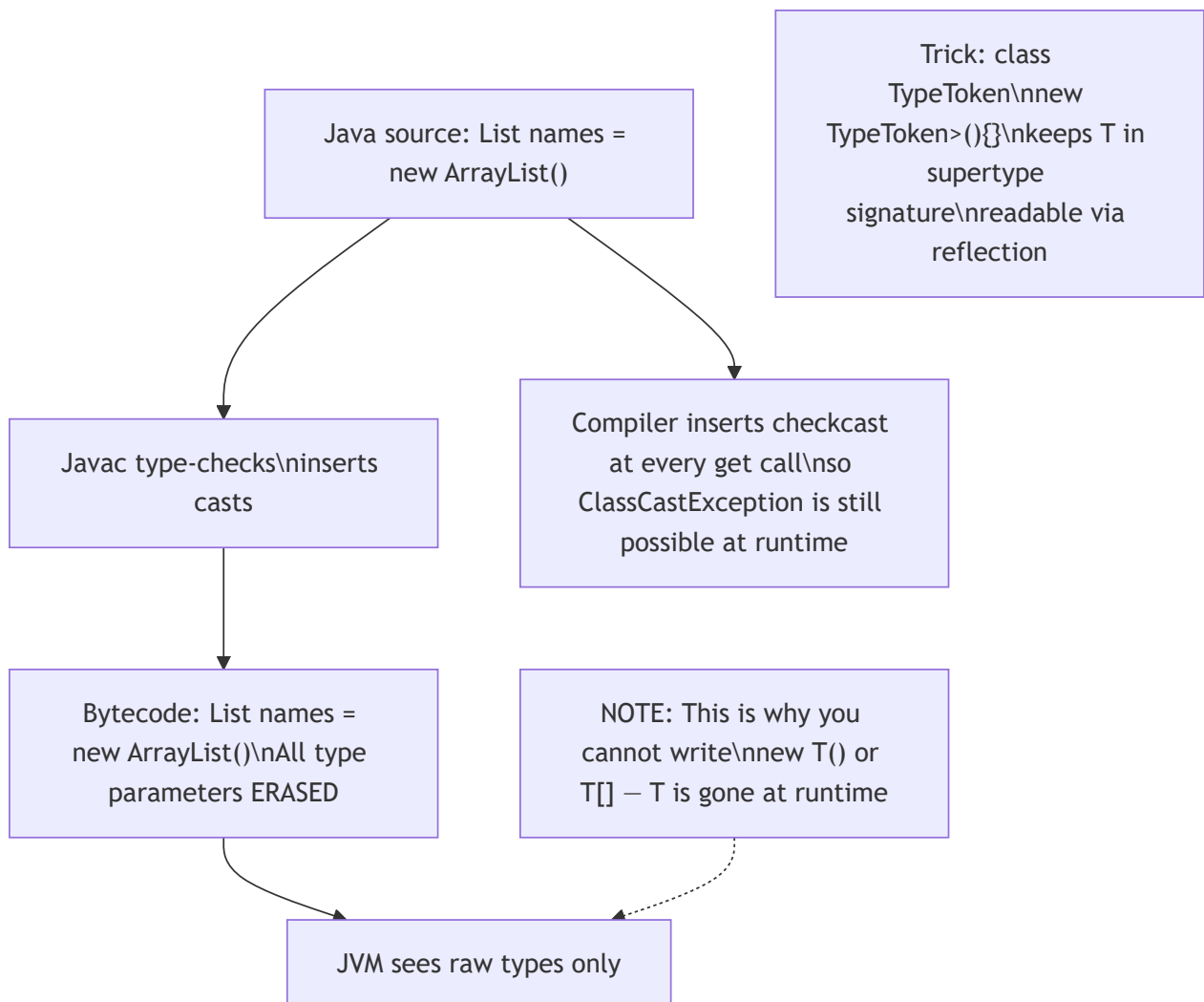
Object Header Layout

Every Java object has a header before its fields. On a 64-bit JVM the header consists of two machine words: the **mark word** and the **klass pointer**. The mark word (8 bytes) encodes the object's identity hash code, GC age (how many collections it has survived), lock state (biased, thin, or fat lock), and a GC forwarding pointer during collection. The klass pointer references the class metadata in Metaspace. With compressed OOPs enabled (the default), the klass pointer shrinks to 4 bytes, giving a 12-byte header, padded to 16 bytes for alignment. An object with no fields is still 16 bytes. An `Integer` (one int field) is also 16 bytes. This is why boxing primitives quadruples memory consumption compared to bare int arrays.



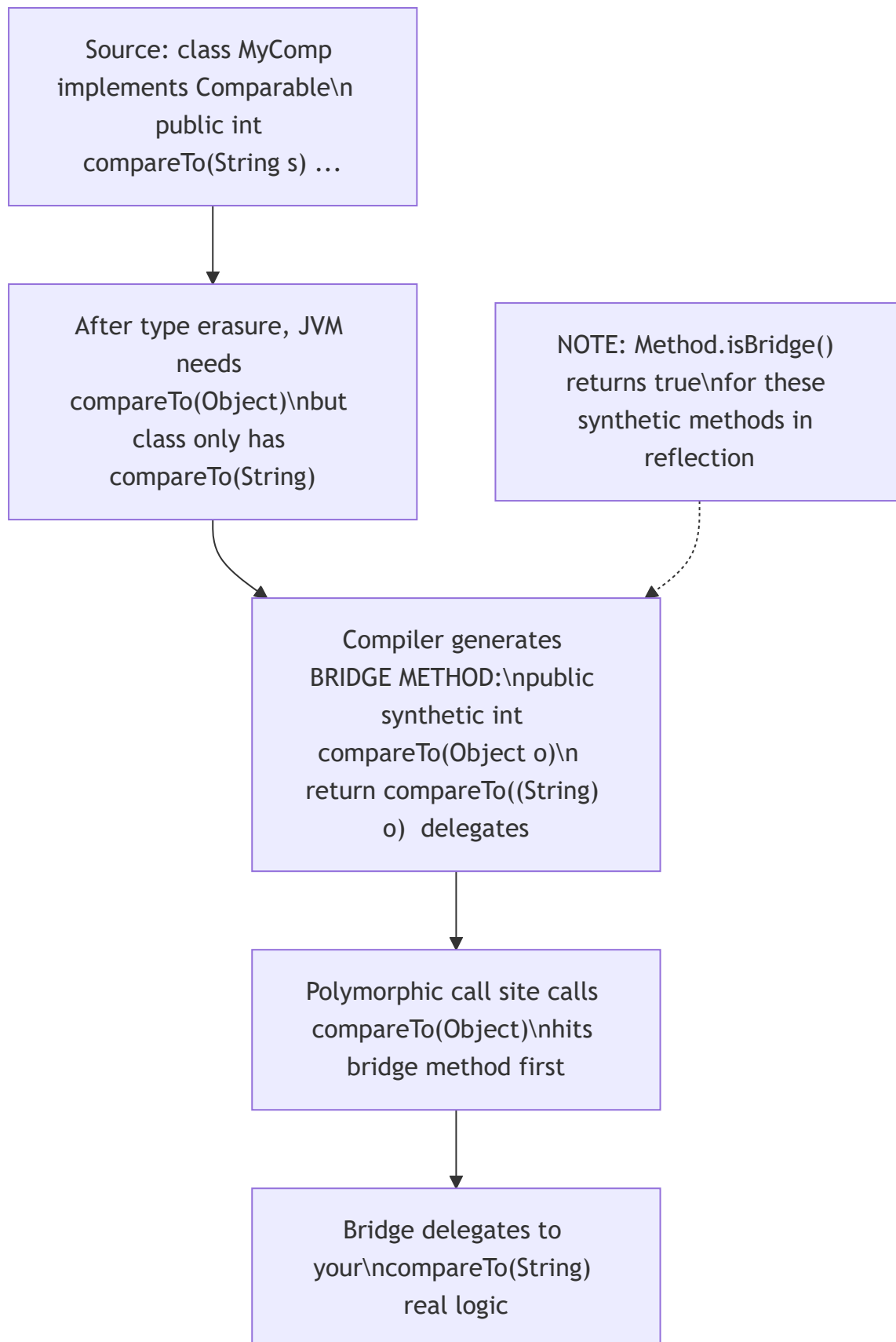
Generics Type Erasure

Java generics are a compile-time feature only. The compiler checks types, inserts casts, and then **erases** all generic type information before emitting bytecode. At runtime, `List<String>` and `List<Integer>` are both just `List`. The JVM has no awareness of what `T` was — this was a deliberate backward-compatibility decision that required zero JVM changes. The downside is that you cannot do `new T()`, `new T[]`, or `instanceof List<String>` — the information simply does not exist at runtime. You recover it with a `Class<T>` token or by using an anonymous subclass to capture the type in the supertype signature — the classic type-token pattern used by Jackson's `TypeReference`.



Bridge Methods

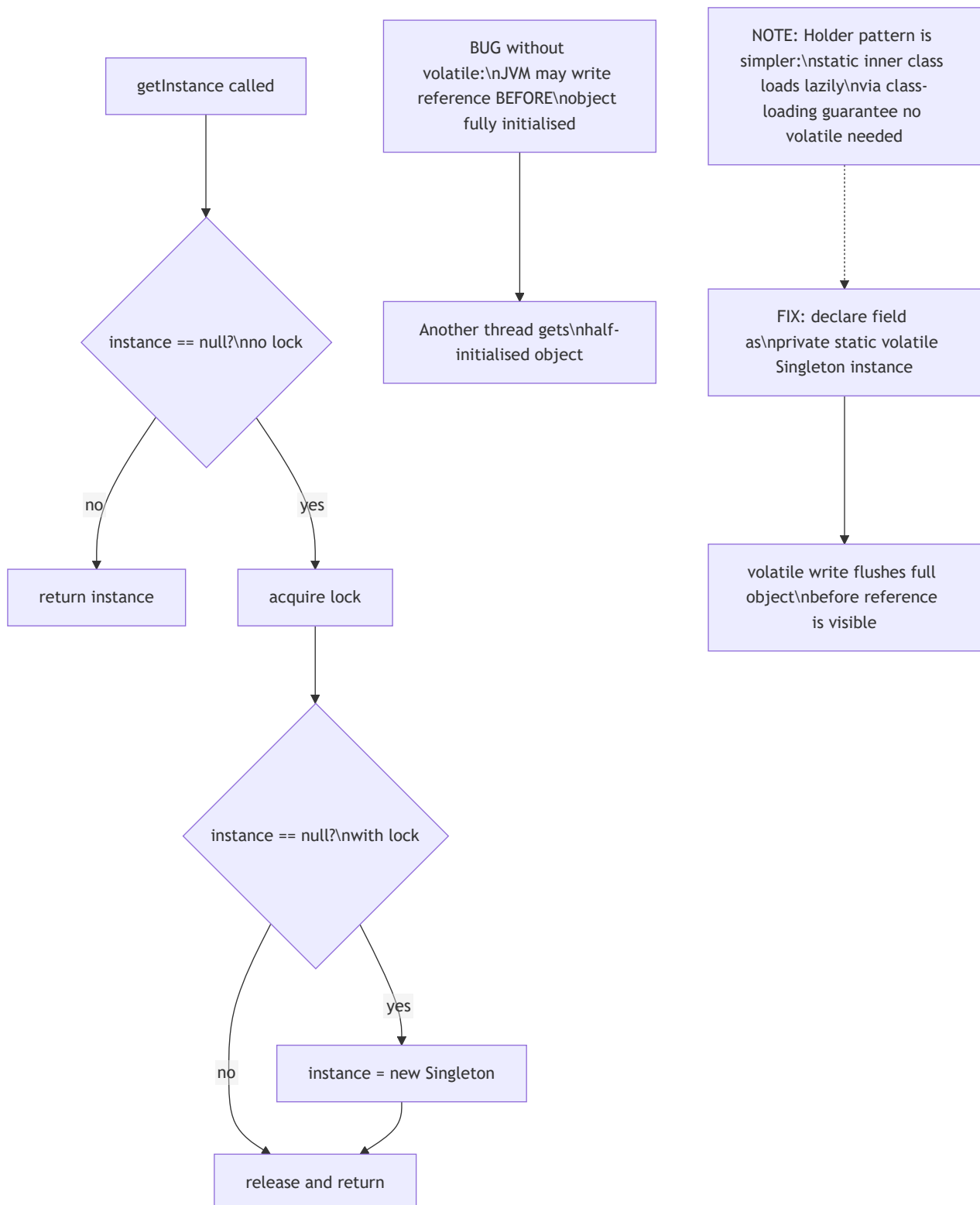
When a class overrides a generic method from a supertype, type erasure creates a signature mismatch. If you implement `Comparable<String>` with `compareTo(String s)`, the JVM's raw `Comparable` interface expects `compareTo(Object o)`. The compiler generates a **synthetic bridge method** with the erased signature that casts its argument and delegates to your real method. This bridge is what the polymorphic call site hits; your typed method handles the actual logic. Bridge methods are invisible in source code but visible in decompiled bytecode and through reflection via `Method.isBridge()`.



Double-Checked Locking

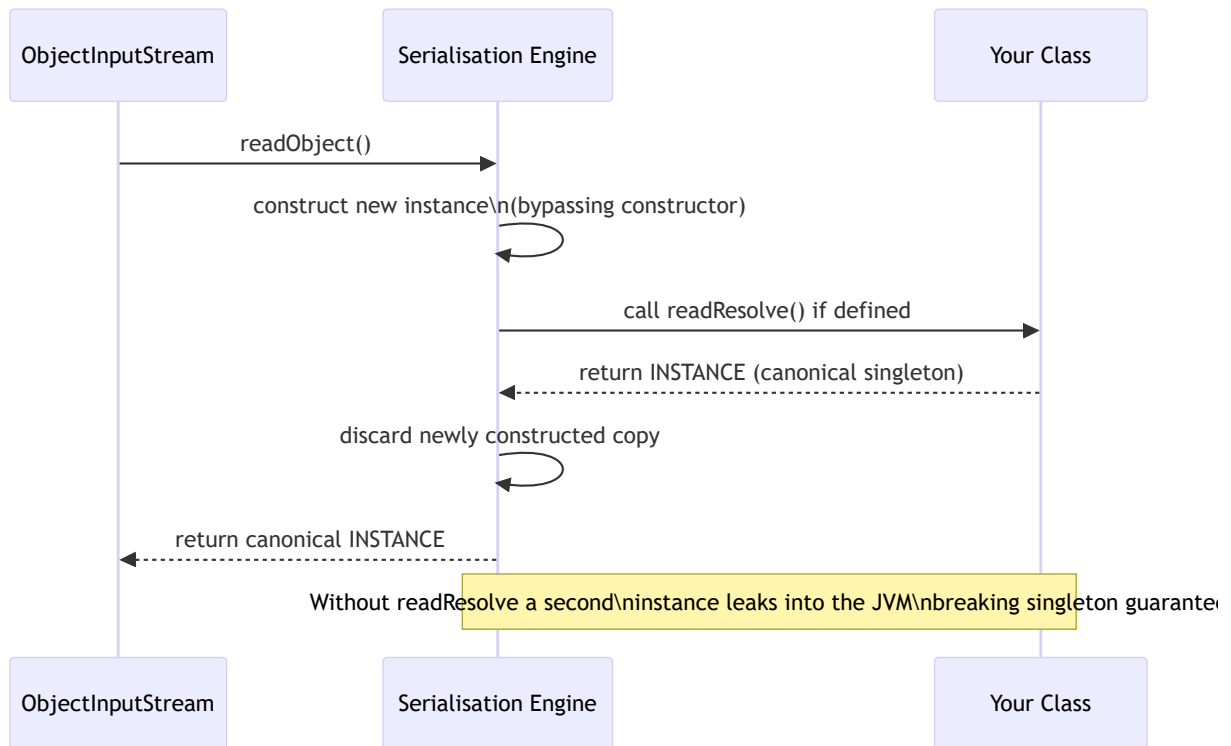
Double-checked locking is a lazy initialisation pattern that checks the instance reference before and after acquiring the lock — avoiding synchronisation overhead once initialised. The classic implementation without `volatile` was broken due to instruction reordering: the

JVM can write the reference to the field before the object is fully initialised. A second thread passes the first null check, sees a non-null reference, and returns a half-initialised object. Declaring the field `volatile` prevents the reordering. A simpler and safer alternative is the **Initialisation-On-Demand Holder** pattern, which leverages class loading guarantees — no synchronisation required at all.



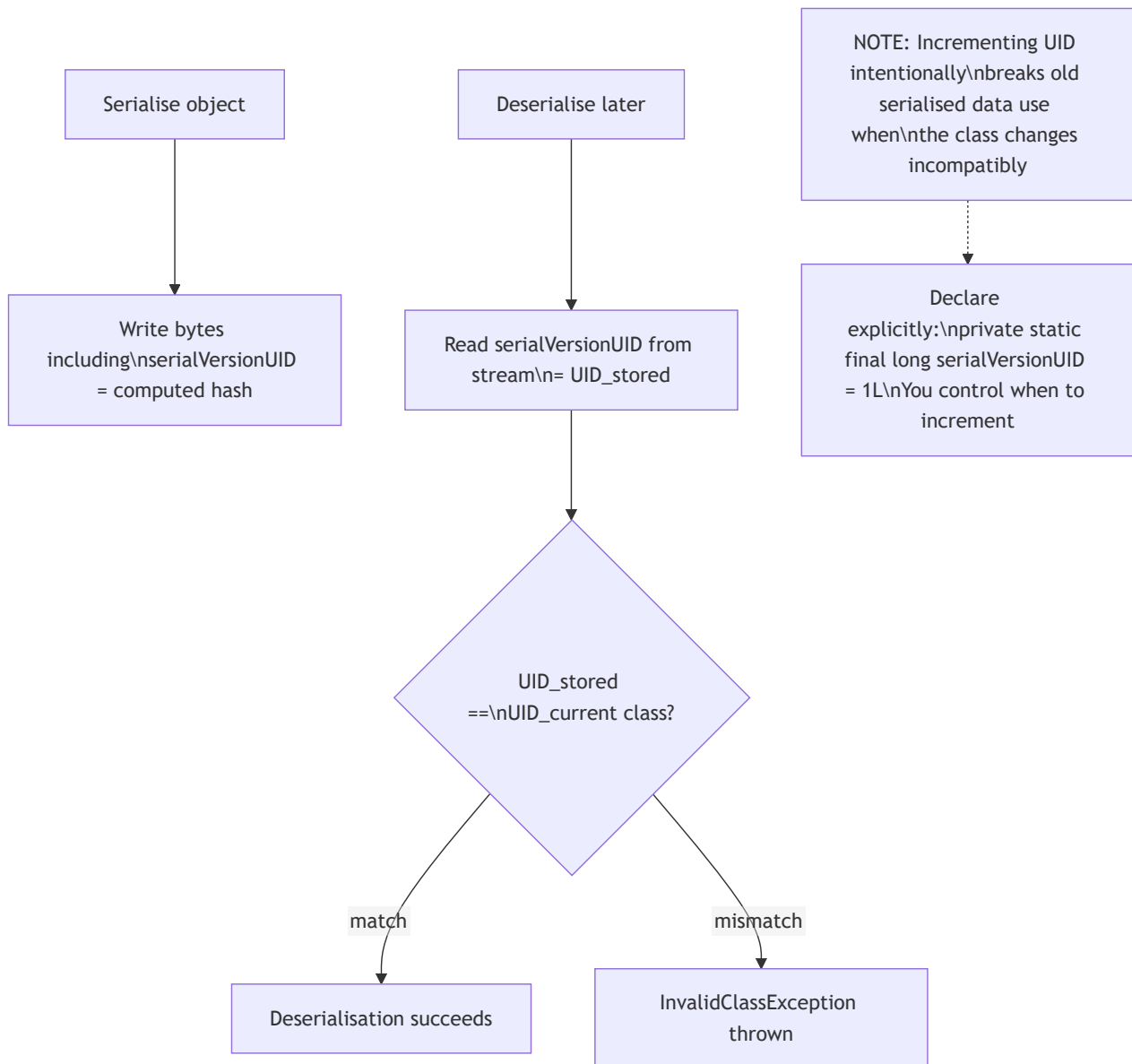
readResolve

Java serialisation normally bypasses constructors when reconstructing objects. For singletons this is catastrophic: deserialising gives you a second instance, breaking the pattern. If you define a `readResolve()` method on a `Serializable` class, the deserialisation machinery calls it after constructing the object and uses its return value as the final result — discarding the newly constructed copy. You return the canonical instance. This works for enums automatically; you cannot deserialise a duplicate enum constant because the JVM handles it at the language level.



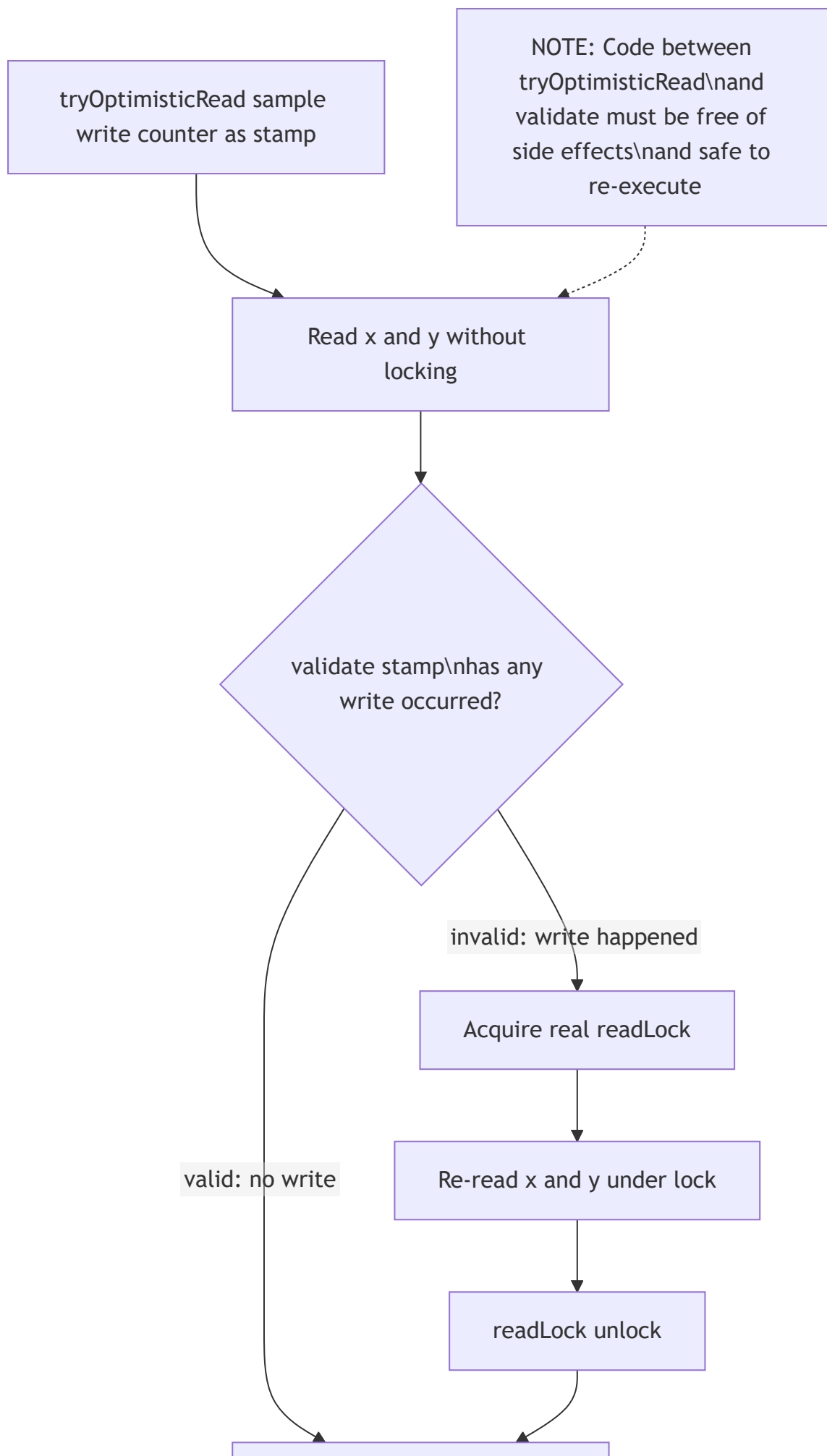
SerialVersionUID

When Java serialises an object, it writes a fingerprint of the class — the `serialVersionUID` — into the byte stream. On deserialisation it computes the current class's fingerprint and compares. A mismatch throws `InvalidClassException`. If you omit the declaration, Java auto-computes the UID from class name, interfaces, methods, and fields. This auto-computed value is sensitive to innocuous changes like adding a private helper method or recompiling with a different JDK version. Declaring it explicitly as `private static final long serialVersionUID = 1L` pins the version, giving you full control over backward compatibility. The deeper principle: serialisation makes your class definition part of your data format.



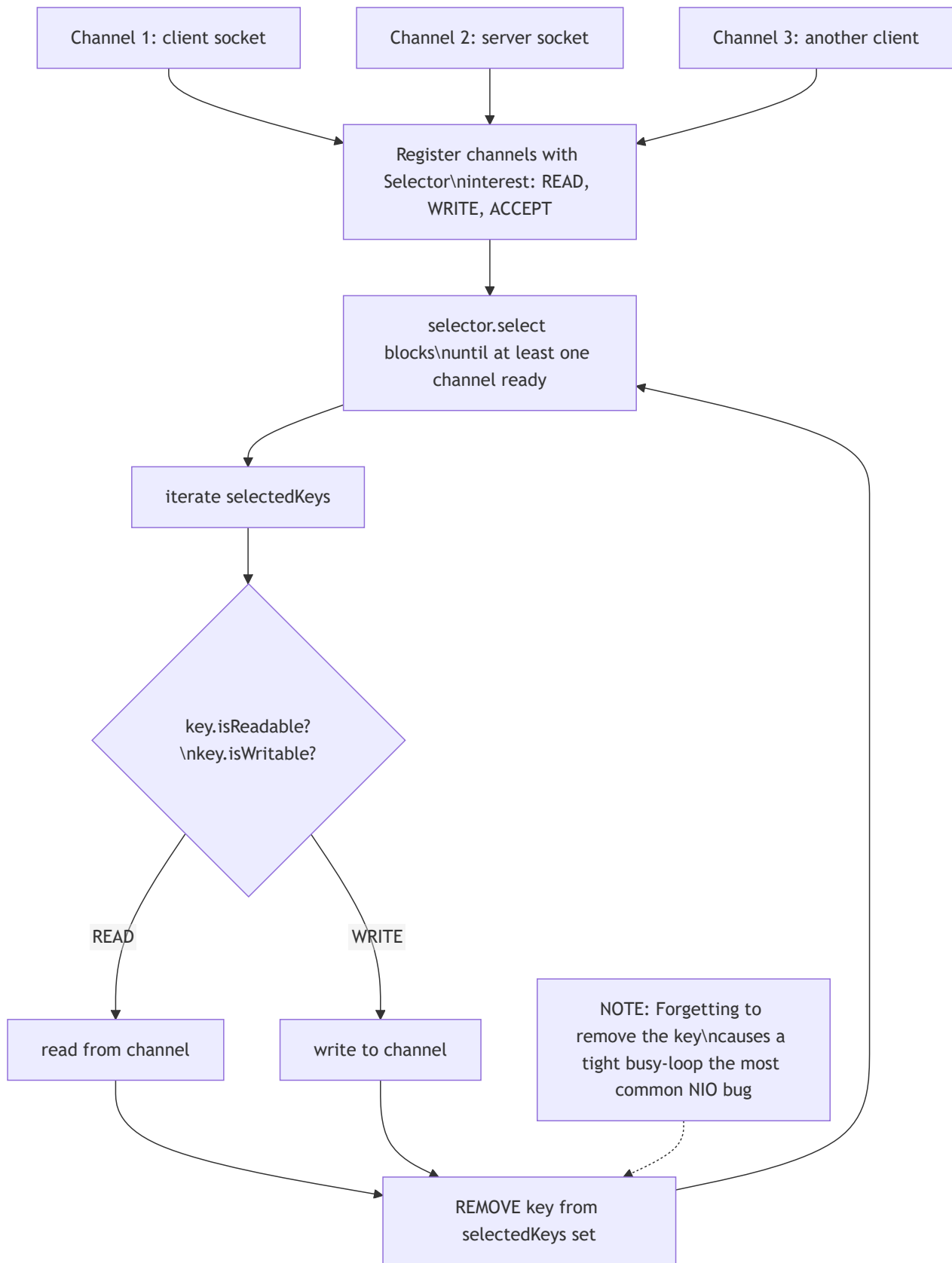
StampedLock Optimistic Reads

StampedLock adds a third locking mode beyond read and write: **optimistic read**. You sample the write counter (acquiring a stamp without actually locking), perform your read, then validate the stamp. If no write occurred in the interim, you're done — zero lock overhead. If a write did happen, you fall back to a real read lock. This pattern is extremely fast when reads vastly outnumber writes because the common path involves no synchronisation at all. The critical constraint: the code between `tryOptimisticRead()` and `validate()` must not have side effects and must tolerate reading potentially inconsistent data — the validation is what makes it safe.



NIO Selector Internals

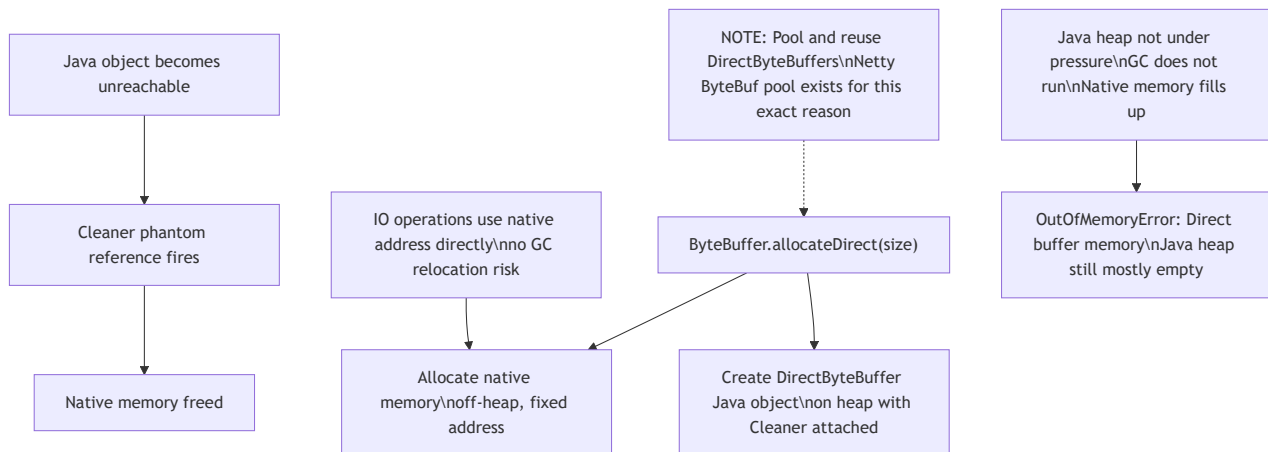
Java NIO's `Selector` enables one thread to monitor many channels simultaneously rather than dedicating a thread per connection. Internally it wraps OS-level multiplexing: `epoll` on Linux, `kqueue` on macOS. You register each channel with a set of interest operations (`OP_READ`, `OP_WRITE`, `OP_ACCEPT`, `OP_CONNECT`). Calling `select()` blocks until at least one registered channel is ready. You iterate `selectedKeys()` to find ready channels and handle them. A very common bug: if you do not remove the key from the selected set after handling it, the selector keeps returning it even when nothing new has happened, causing a busy-loop. This is the foundation of Netty, Tomcat NIO mode, and virtually every high-performance Java server.



DirectByteBuffer Lifecycle

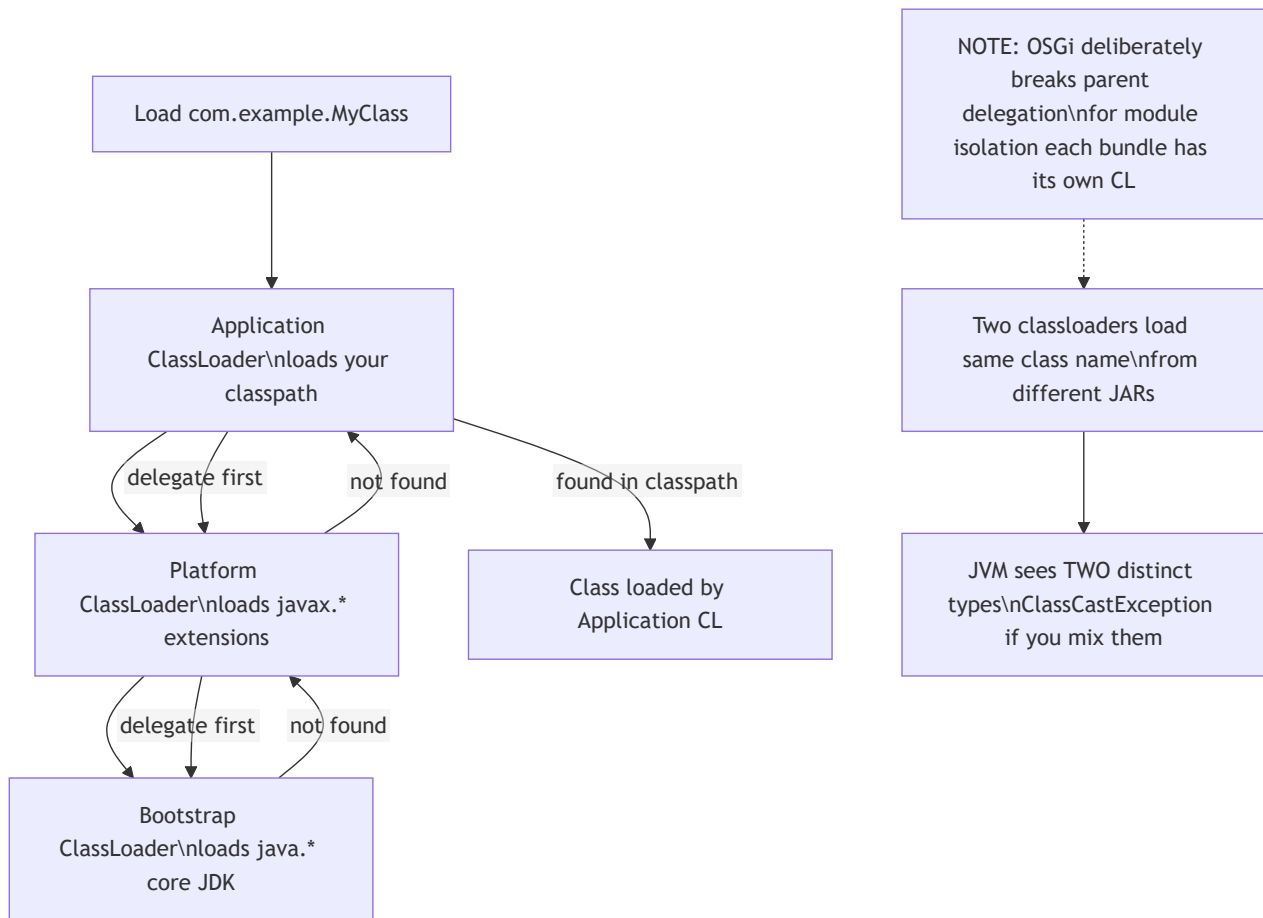
A `DirectByteBuffer` allocates memory in native (off-heap) space rather than the Java heap. This is essential for I/O operations: data being transferred to or from the OS must not move during the transfer, but the GC can relocate heap objects at any time. Direct buffers are pinned at a fixed native address. The lifecycle problem: they are managed by Java

heap objects via a `Cleaner` (a phantom reference). When the Java object is GC'd, the `Cleaner` frees the native memory. But the Java heap may not feel pressure even as native memory fills up — `OutOfMemoryError: Direct buffer memory` can occur while the Java heap is mostly empty. Control the total allocation with `-XX:MaxDirectMemorySize`.



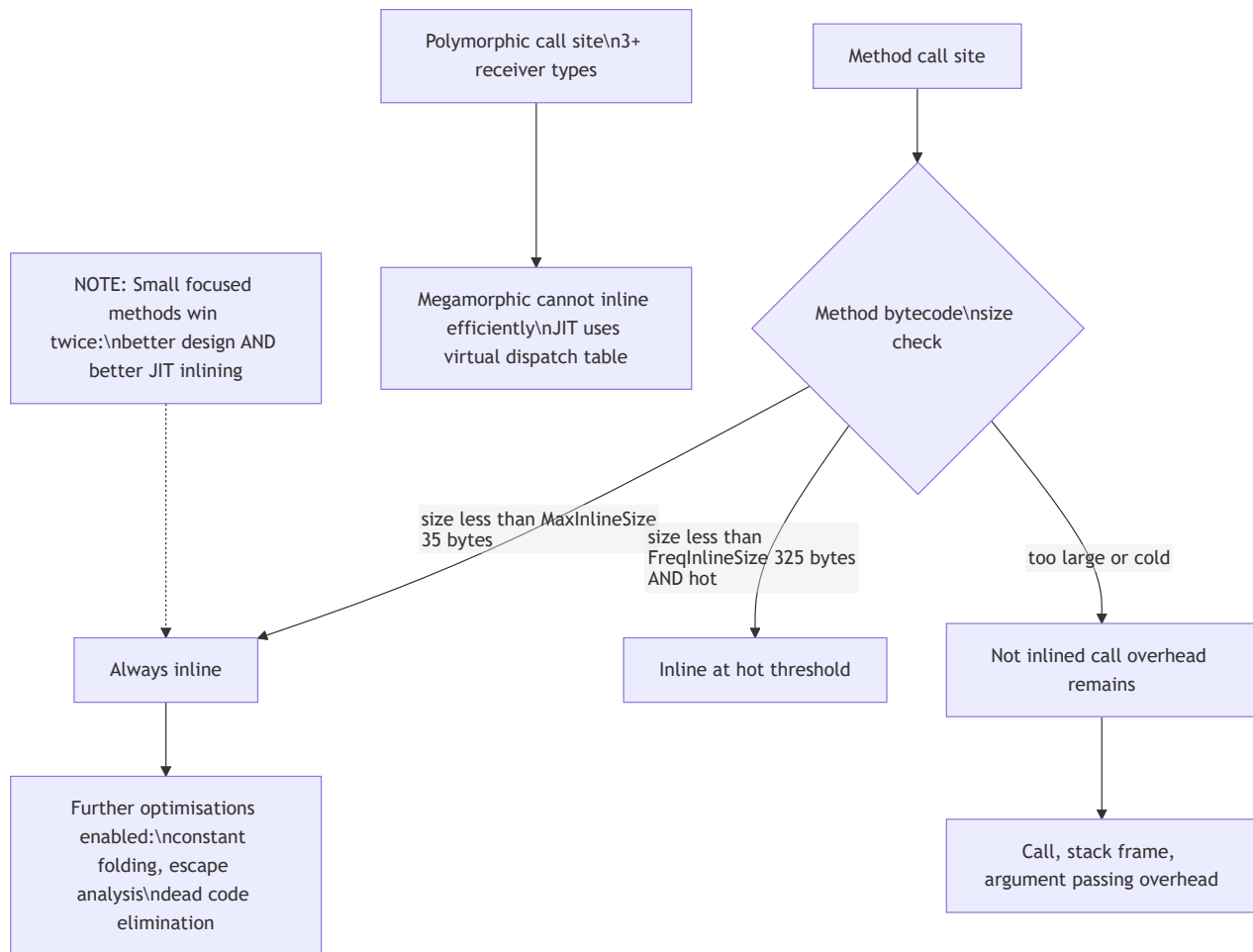
ClassLoader Delegation Model

Java's class loading uses a parent-first delegation hierarchy. When a classloader is asked to load a class it first delegates to its parent. The chain is: Bootstrap (loads `java.*` from the JDK modules) then Platform/Extension (loads `javax.*`, extensions) then Application (loads your classpath). This prevents you from accidentally overriding a JDK class — Bootstrap always wins. The critical implication for multi-classloader environments (OSGi, application servers): if two different classloaders independently load the same fully-qualified class name from different JARs, the JVM treats them as **completely different types**, causing `ClassCastException` even though the class names are identical.



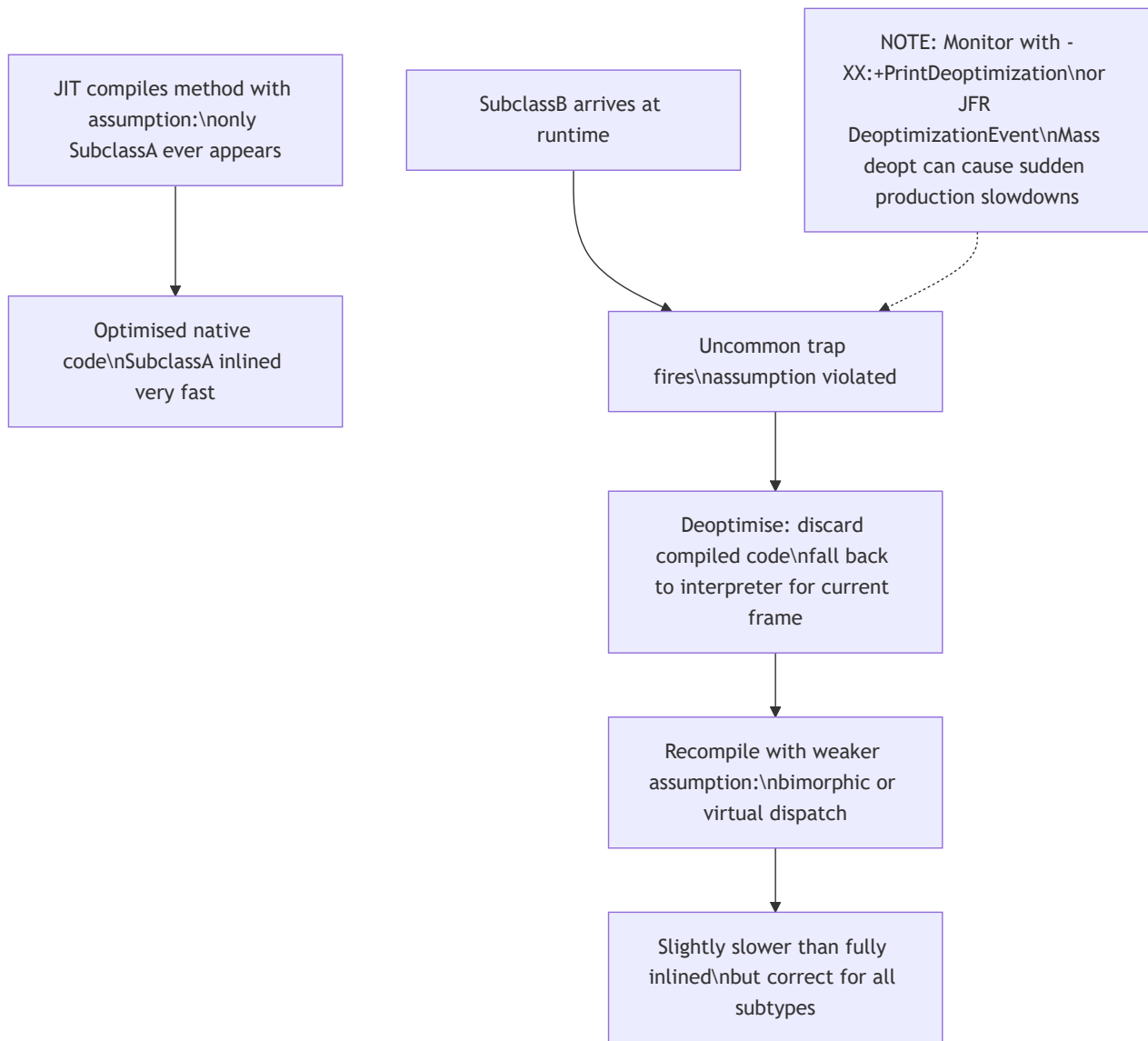
JIT Inlining Heuristics

Method inlining is the most impactful JIT optimisation: the JIT copies a callee's bytecode into the caller, eliminating call overhead and enabling further optimisations like constant propagation, escape analysis, and dead code elimination. The heuristics are size and frequency driven. A method is inlined if its bytecode is smaller than `MaxInlineSize` (default ~35 bytes). Hot methods (compiled at high tier) get a larger threshold controlled by `FreqInlineSize` (up to ~325 bytes). Methods that are too large, have too many call sites, or dispatch polymorphically to more than two types resist inlining. The lesson: small, focused methods are not just good design — they are JIT-friendly.



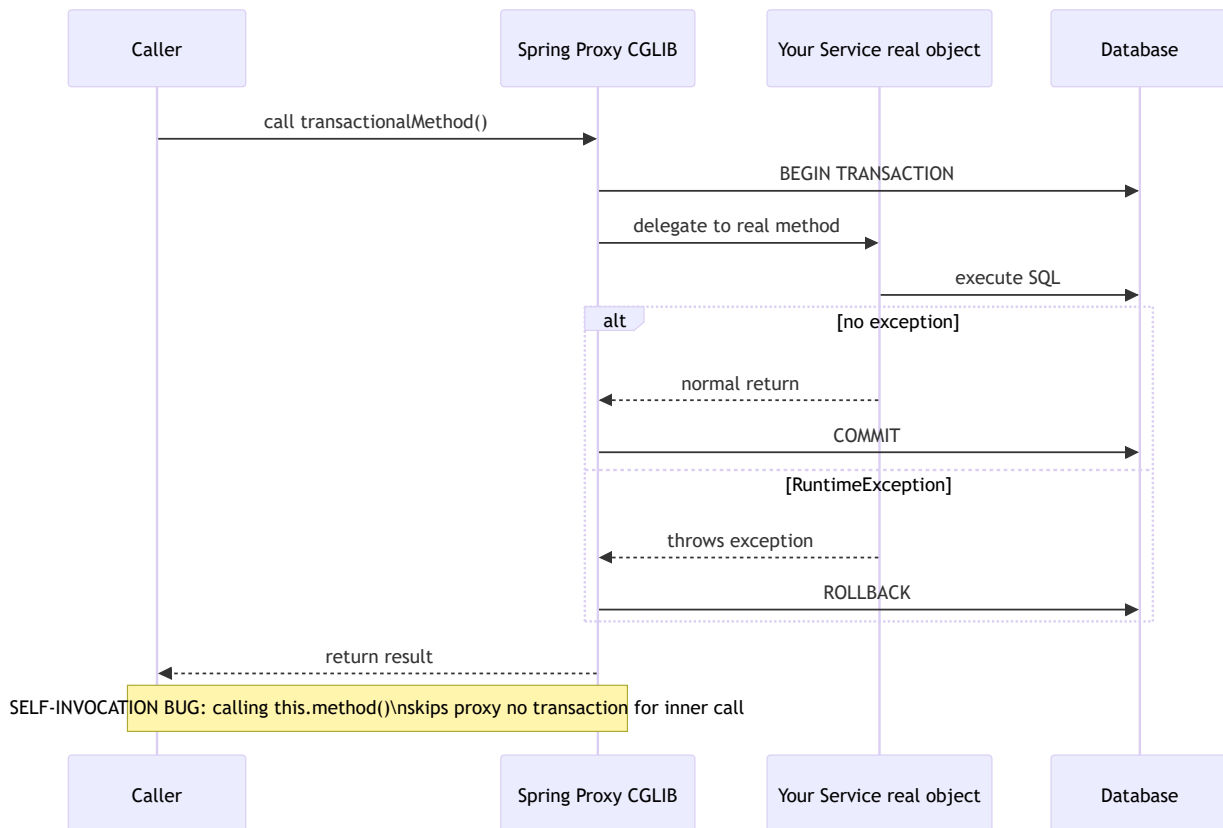
Deoptimisation

The JIT compiles code speculatively — it assumes that a virtual call always dispatches to `SubclassA` and inlines it. If `SubclassB` arrives at runtime, the assumption is violated and the JIT must **deoptimise**: discard the compiled code for that method, fall back to the interpreter for the current frame (on-stack replacement), and eventually recompile with weaker assumptions. Deoptimisation is normal and expected — it happens during class loading when new subclasses appear, when infrequent branches are suddenly taken, or when uncommon trap conditions are hit. The risk is a mass deoptimisation event under unusual input patterns that causes a sudden throughput drop in production.



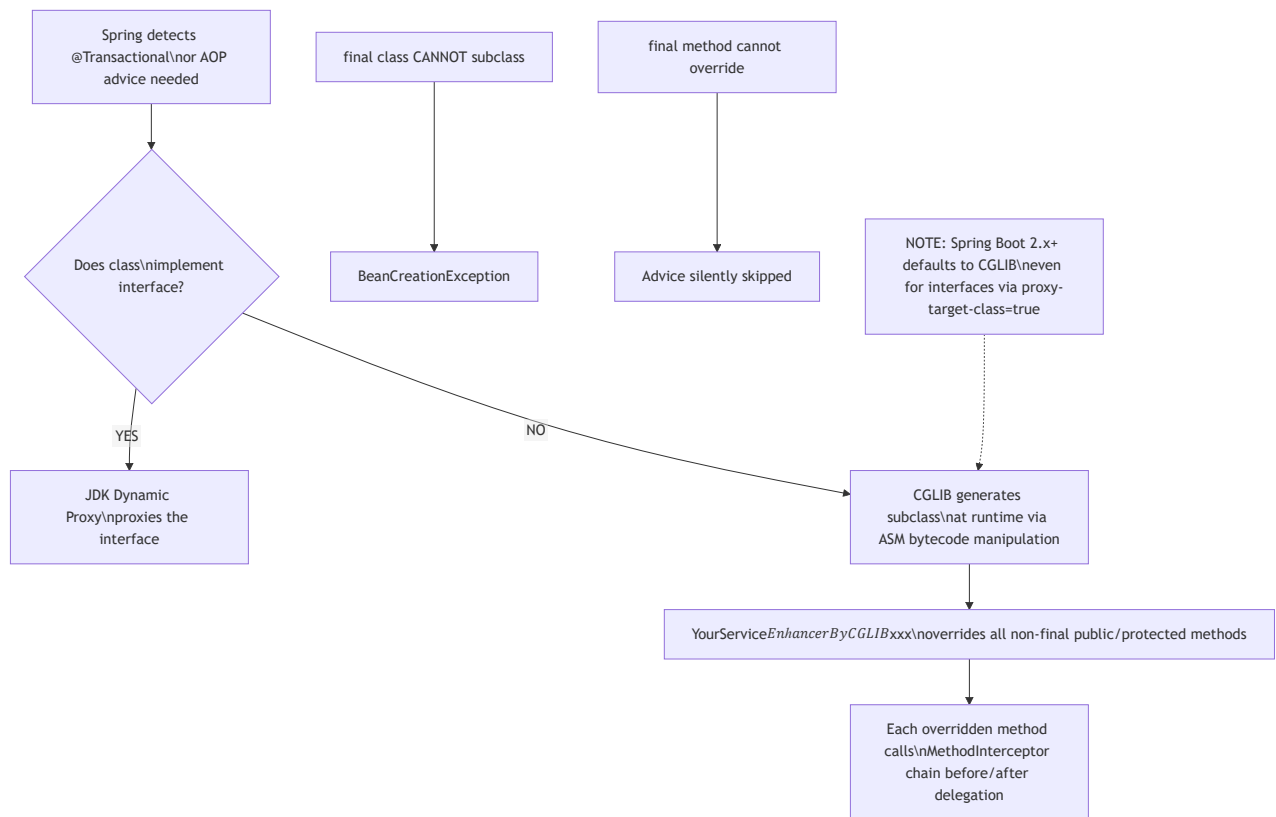
@Transactional Proxy

Spring's `@Transactional` works through AOP proxy objects. When you inject a service bean, Spring provides a proxy wrapping the real object. Calling a `@Transactional` method calls the proxy, which opens a transaction, delegates to your real method, and commits or rolls back on return. The most common gotcha is **self-invocation**: if `methodA()` calls `this.methodB()` (both annotated), it bypasses the proxy entirely — no transaction is started for `methodB`. Private methods are silently ignored — CGLIB cannot proxy them. Catching and swallowing an exception inside the method causes a commit even though you handled a failure.



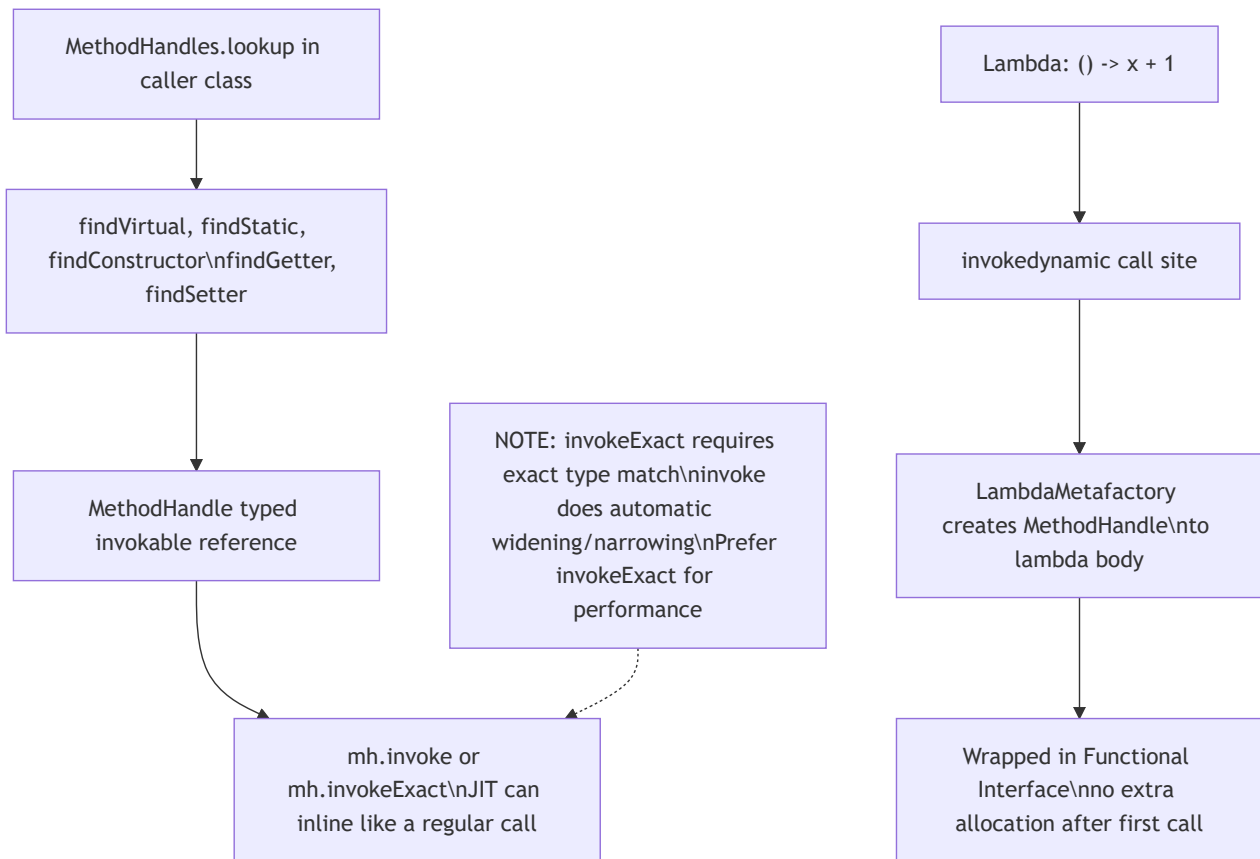
CGLIB

CGLIB (Code Generation Library) is a bytecode manipulation library Spring uses when JDK dynamic proxies are unavailable — that is, when your class does not implement an interface. CGLIB generates a subclass of your concrete class at runtime and overrides all eligible methods to insert AOP advice or transaction management. The generated class name appears in stack traces as `YourClass$$EnhancerBySpringCGLIB$$abc123`. The constraints: your class must not be `final` (CGLIB cannot subclass final classes), and target methods must be public or protected and non-final. Records and sealed classes are inherently final, which is one of the architectural pressures driving Spring toward interface-based design.



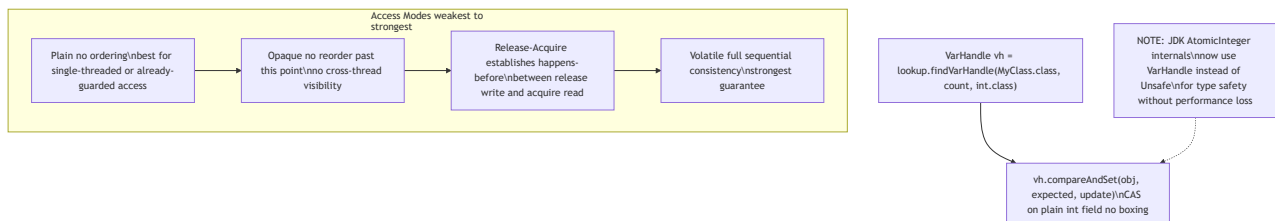
MethodHandles

`MethodHandles` provides a modern, JIT-friendly alternative to `java.lang.reflect`. A `MethodHandle` is a typed, directly invocable reference to a method, constructor, or field. Unlike reflection, `MethodHandle` invocations can be JIT-inlined, boxless, and allocation-free for primitive arguments. Lookup objects carry the caller's access rights — you can look up private methods of the enclosing class. `MethodHandles` are the implementation mechanism for `invokedynamic`, lambda expressions (`LambdaMetafactory` wraps a handle to the lambda body), and dynamic language dispatch.



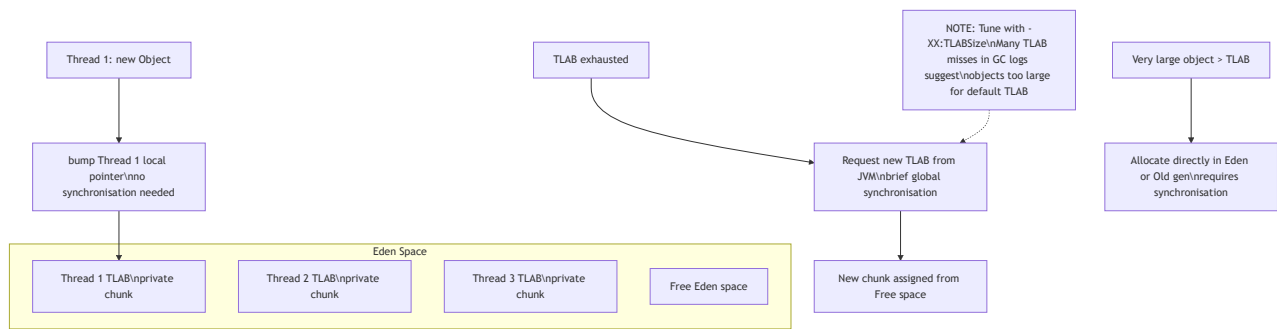
VarHandle

`VarHandle`, introduced in Java 9, generalises `MethodHandle` to variable access — instance fields, static fields, and array elements — with fine-grained memory ordering semantics. You choose from four access modes: **plain** (no ordering), **opaque** (prevents dead-code elimination but no cross-thread guarantee), **release-acquire** (establishes happens-before), and **volatile** (full volatile semantics). The big benefit: `AtomicInteger`-like CAS operations on ordinary primitive fields without wrapper object allocation. The JDK rewrote `Atomic` classes internally to use `VarHandle` for exactly this reason.



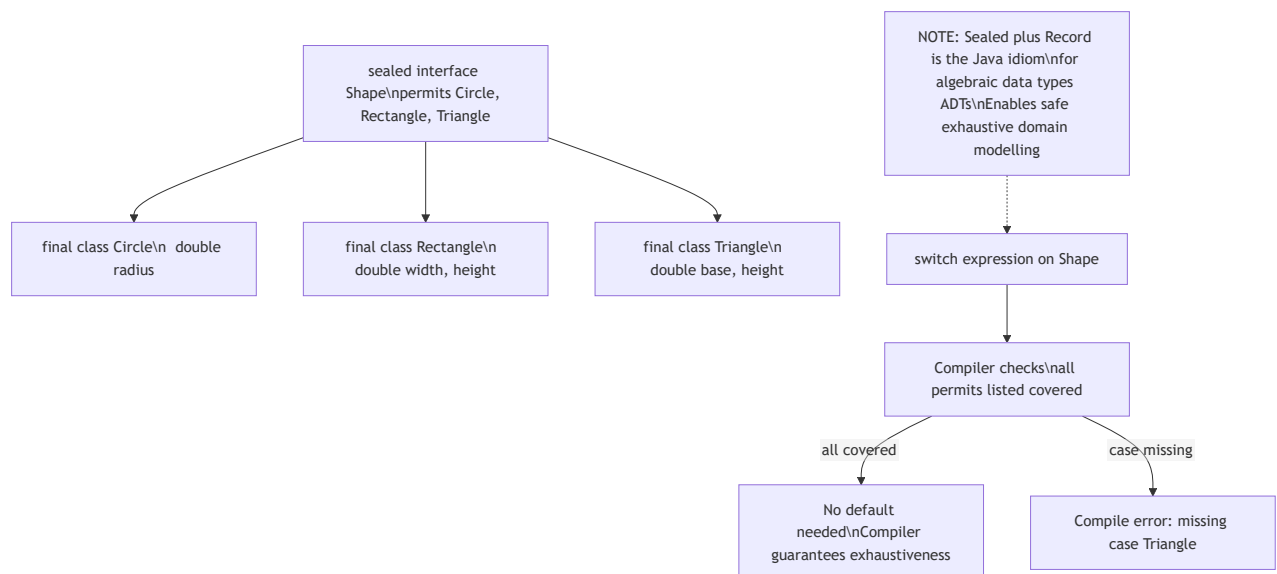
TLAB — Thread-Local Allocation Buffer

Allocating objects on the heap without TLABs would require atomic synchronisation on every allocation. TLABs solve this by giving each thread a private chunk of Eden space. Allocation within the TLAB simply bumps the thread's local pointer — no synchronisation, essentially as cheap as stack allocation. When the TLAB is exhausted, the thread requests a new one, requiring only brief synchronisation. Very large objects that exceed the TLAB size are allocated directly in Eden or Old generation with synchronisation, which is why large objects are relatively expensive to allocate frequently.



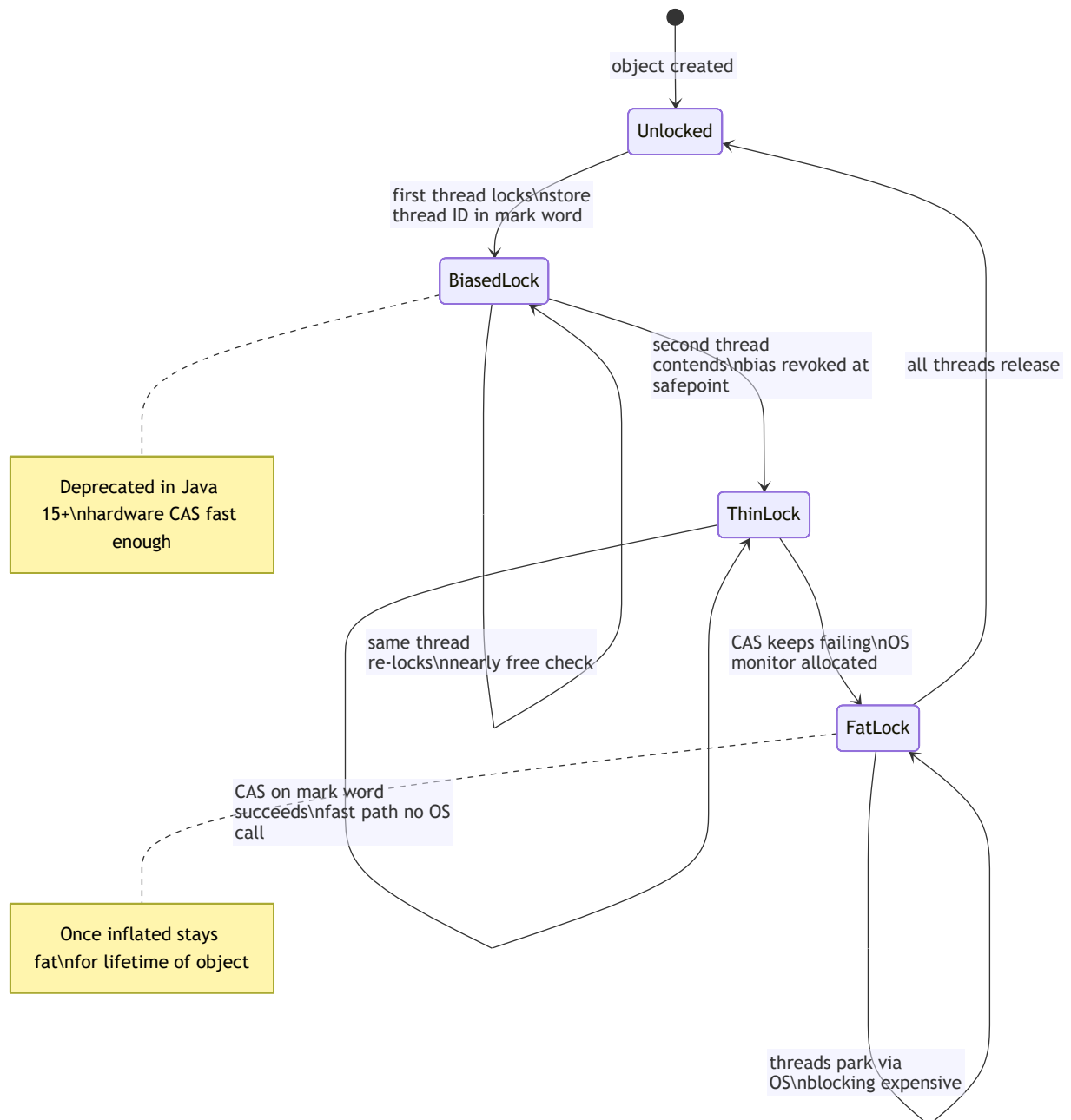
Sealed Classes

Sealed classes, introduced in Java 17, restrict which classes may extend or implement a type by declaring the permitted subclasses explicitly. This serves one primary purpose: enabling **exhaustive pattern matching**. When the compiler knows all possible subtypes, it can verify that your `switch` expression handles every case, giving you the same safety as an enum while allowing subclasses to have different fields and methods. Sealed classes pair naturally with records to create algebraic data type patterns — the Java equivalent of Scala sealed traits or Haskell sum types.



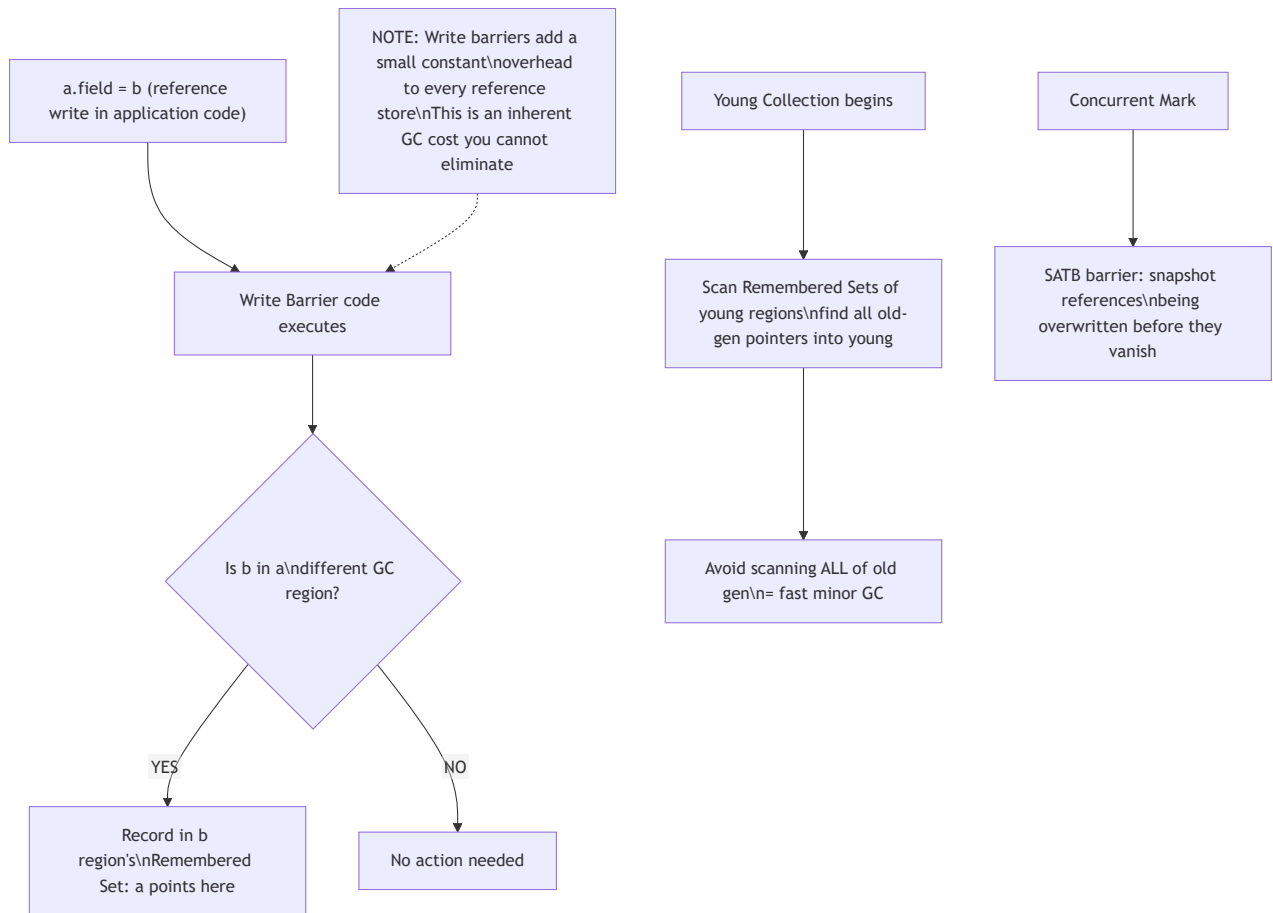
Lock Inflation

Java's `synchronized` locking passes through several states to optimise the common uncontended case. A new object starts **unlocked**. The first thread to lock it acquires a **biased lock** — just a thread ID written into the mark word. Future locks by the same thread are essentially free. When a second thread attempts to lock the biased object, the bias is revoked at a safe point, and the lock inflates to a **thin lock** — implemented with a CAS on the mark word. Under sustained contention where CAS keeps failing, the lock inflates to a **fat lock** — an OS monitor is allocated, and blocking threads are parked by the OS. Modern JVMs (Java 15+) deprecated biased locking.



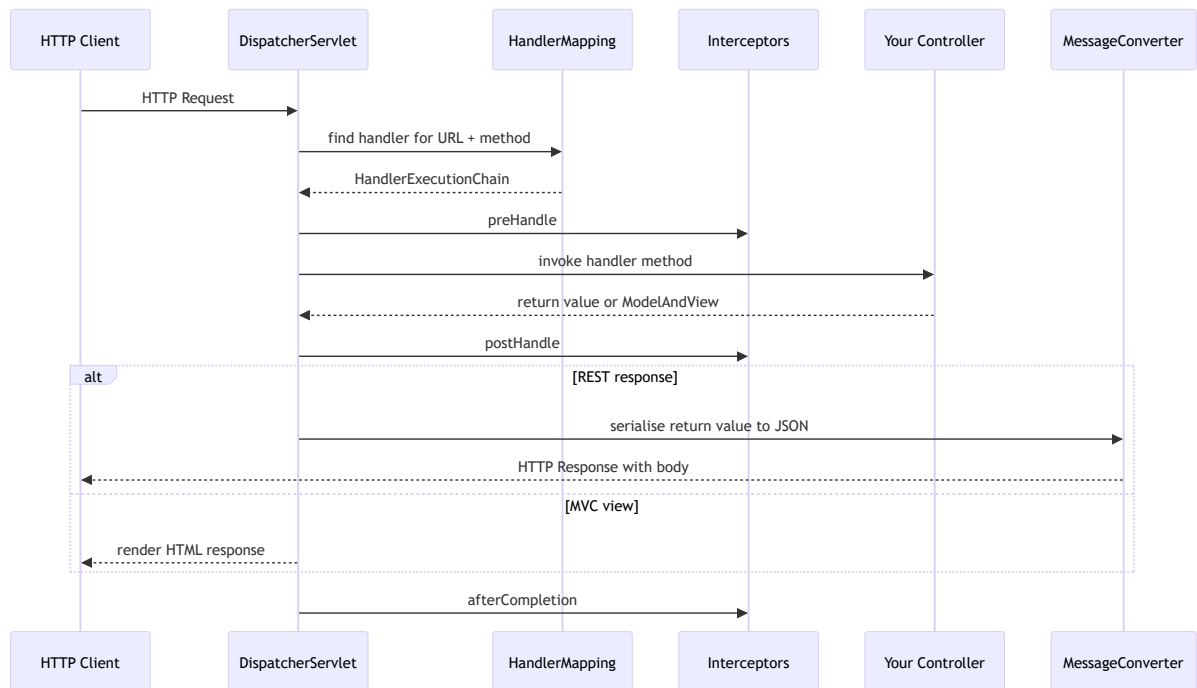
Write Barriers

A write barrier is code the GC inserts around every reference write in your application. When you write `a.field = b`, you may create a reference from an old-generation object to a young-generation object, or cross a G1 region boundary. Without tracking this, the GC would have to scan the entire old generation to find references into the young generation during a minor collection. Write barriers maintain **remembered sets** — one per region in G1 — recording cross-region references as they occur. During a young collection, the GC scans only remembered sets, not the whole heap. Write barriers also implement concurrent mark invariants.



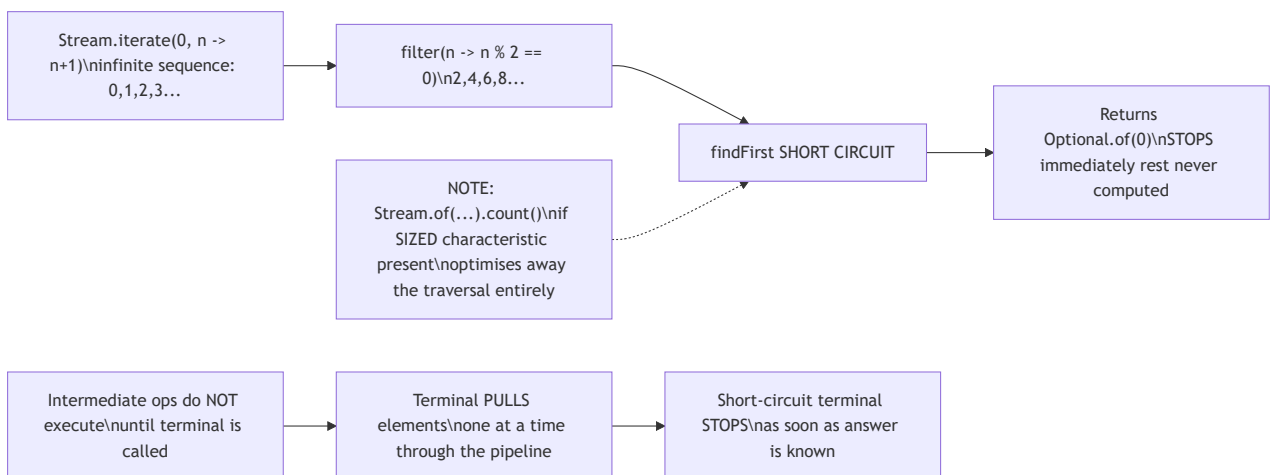
DispatcherServlet

`DispatcherServlet` is Spring MVC's front controller. Every HTTP request enters through it, and it orchestrates the entire request lifecycle. It uses `HandlerMapping` to find which controller handles the request, `HandlerAdapter` to invoke the handler, `MessageConverter` for REST response serialisation, and `ViewResolver` to resolve logical view names for server-side rendering. The pipeline runs: pre-handler interceptors, then the handler, then post-handler interceptors, then view rendering (or response writing for `@ResponseBody`), then after-completion interceptors.



Stream Short-Circuiting

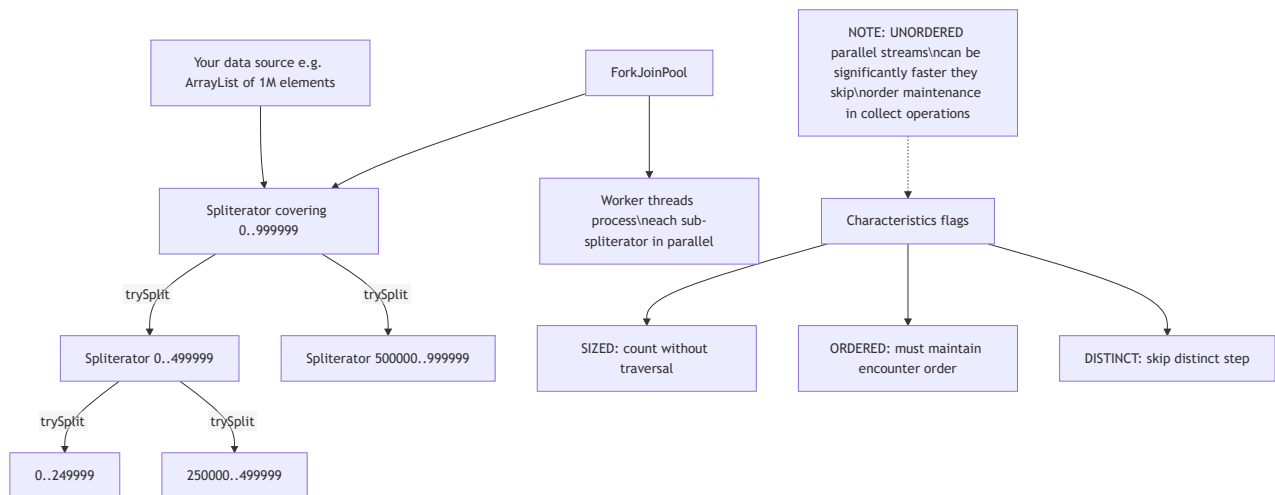
Certain stream terminal operations are **short-circuit**: they produce a result without processing the entire stream. `findFirst`, `findAny`, `anyMatch`, `allMatch`, `noneMatch`, and `limit` are all short-circuit. In a pipeline with `filter` followed by `findFirst`, elements are processed one at a time until one passes the filter — the pipeline stops immediately. This is why infinite streams (`Stream.iterate(0, n -> n+1)`) are usable with short-circuit terminals. The deeper principle is that all stream intermediate operations are lazy — they do not execute until a terminal operation is called. The terminal drives the pipeline, pulling elements through only as needed.



Splititerator

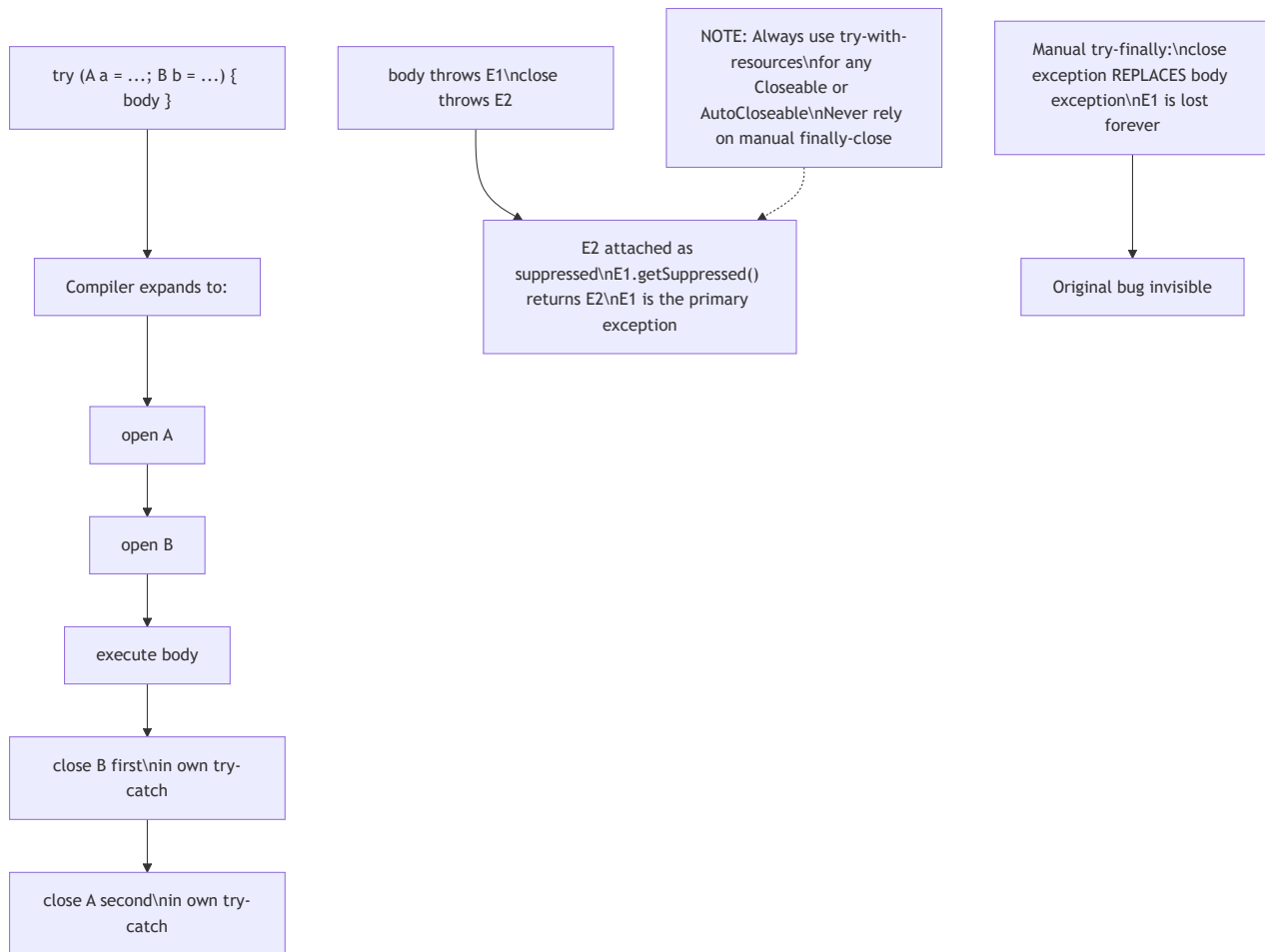
`Splititerator` (Splitting Iterator) is the data-source abstraction behind parallel streams. Like an iterator it traverses elements; unlike one it can **split itself**, yielding two `Splititerator`s each covering roughly half the source. `ForkJoinPool` uses this to recursively decompose data for parallel execution. Split iterators also advertise **characteristics**: `SIZED`, `ORDERED`,

DISTINCT, SORTED, IMMUTABLE. These characteristics enable optimisations — `SIZED` allows `count()` to short-circuit; `UNORDERED` lets certain parallel operations skip order-preserving overhead. Implementing a custom `Splitter` makes any data structure play well with `Stream.parallel()`.



Try-With-Resources

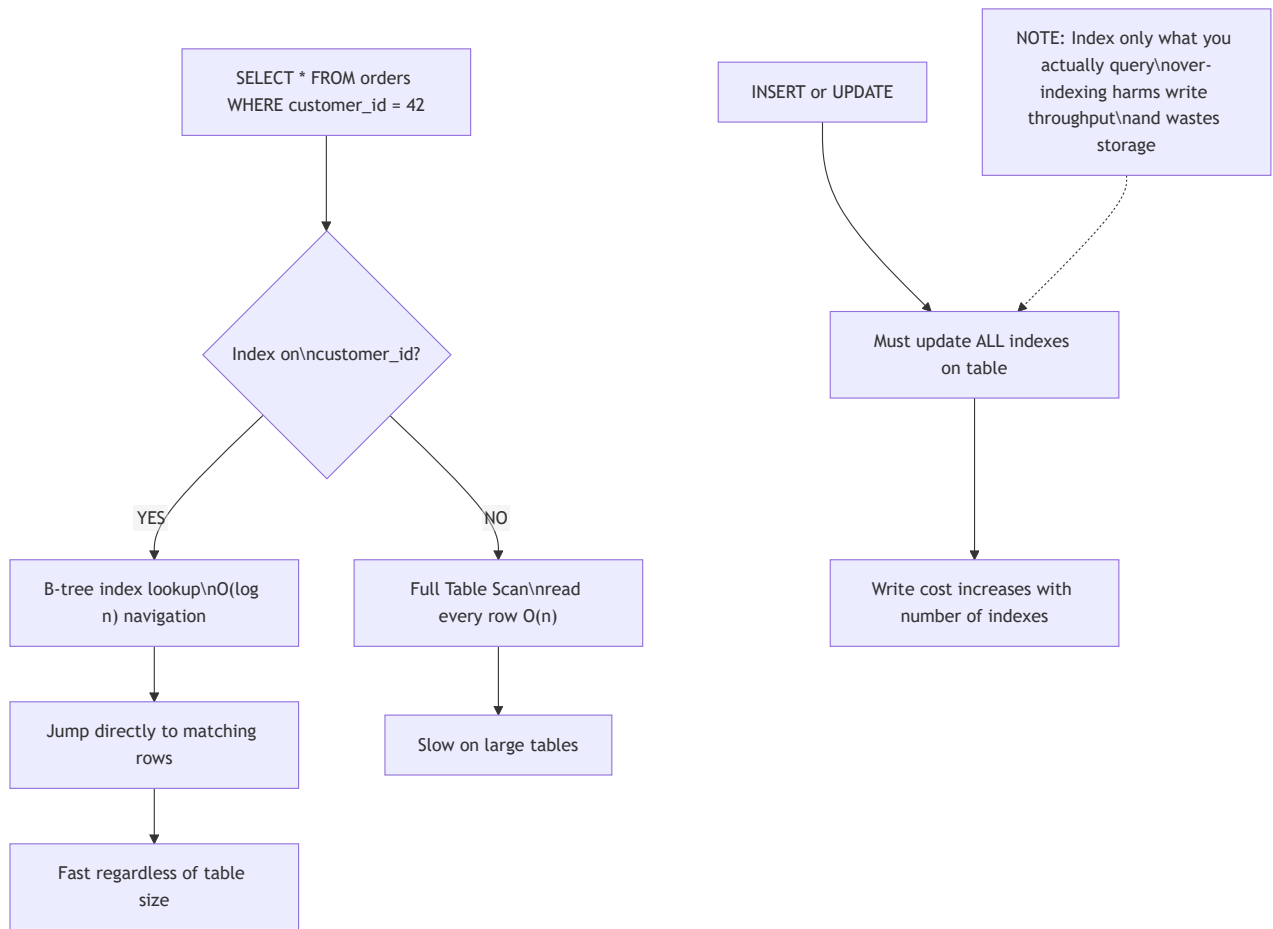
Try-with-resources ensures that `AutoCloseable` resources are closed reliably. The compiler transforms the syntax into a `try-finally` with individually guarded close calls for each resource, in reverse declaration order. The critical improvement over manual `try-finally`: if both the body and a `close()` call throw, the close exception is attached to the original exception as a **suppressed exception** via `addSuppressed()`, rather than replacing it. In manual `try-finally` the close exception would swallow the original — the bug you actually cared about would vanish.



Part 2 — Databases and Distributed Systems

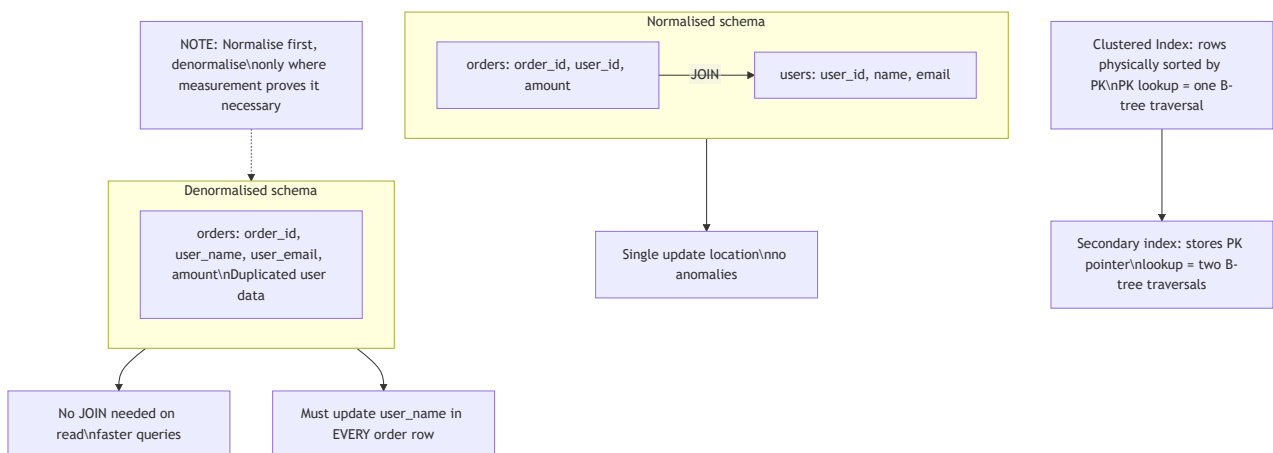
Indexing

An index is a separate data structure the database maintains alongside table data to speed up lookups. Without an index, the database must examine every row — a **full table scan** — to find matching rows. With a B-tree index, it navigates a sorted tree structure to locate matches in $O(\log n)$ operations. Indexes trade write overhead for read speed: every insert, update, or delete must also update all relevant indexes. The art of indexing is choosing which columns to index based on your actual query patterns, because over-indexing slows writes without proportionate read benefit.



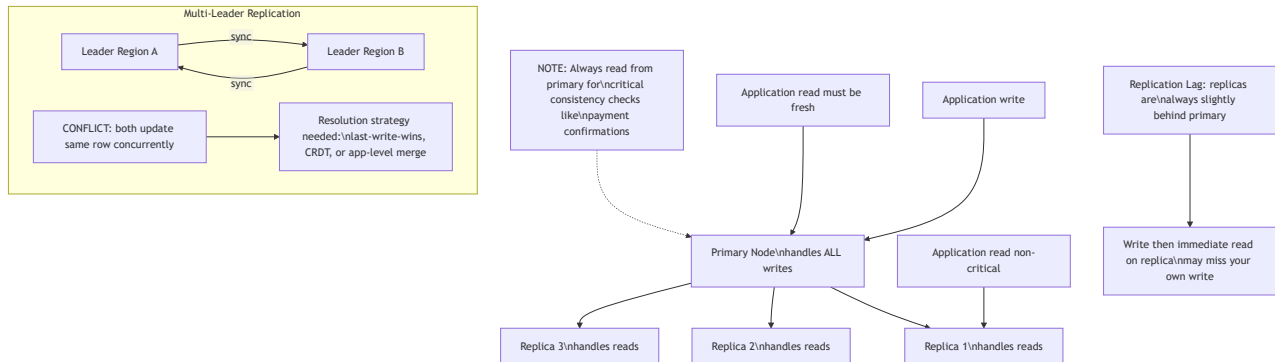
Clustering, Denormalisation, and Normalisation

Normalisation structures a relational database to eliminate data redundancy — each fact stored exactly once, updates happen in one place, no anomalies. Normal forms from 1NF through BCNF codify increasingly strict rules. Denormalisation is the deliberate reversal: you duplicate data to avoid expensive joins, accepting that updates must propagate to multiple places. A clustered index physically stores table rows in index order. In MySQL InnoDB every table has exactly one clustered index — the primary key. Leaf nodes of the B-tree are the actual rows. Secondary indexes store the primary key as a pointer, so a secondary index lookup requires two B-tree traversals.



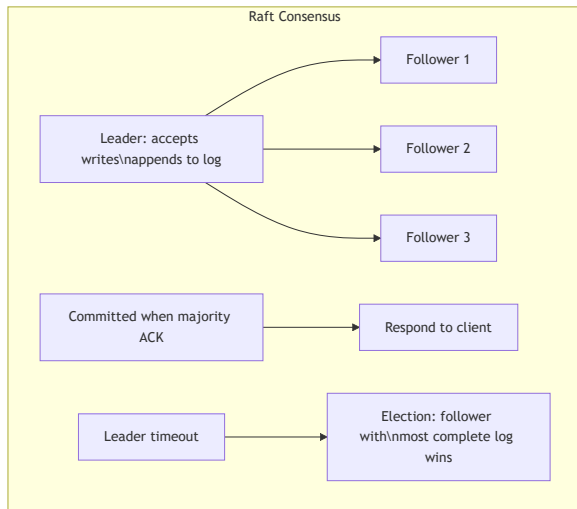
Read Replicas and Replication Modes

A read replica is a copy of the primary database that handles SELECT queries. Changes replicate from primary to replicas via WAL shipping or logical replication. The critical limitation is **replication lag**: the replica is always slightly behind the primary. Never use a replica for reads that must immediately follow a write. Leader-follower (single-leader) replication sends all writes to one leader; multi-leader allows multiple nodes to accept writes, enabling geographic distribution but requiring conflict resolution when two leaders concurrently update the same row.



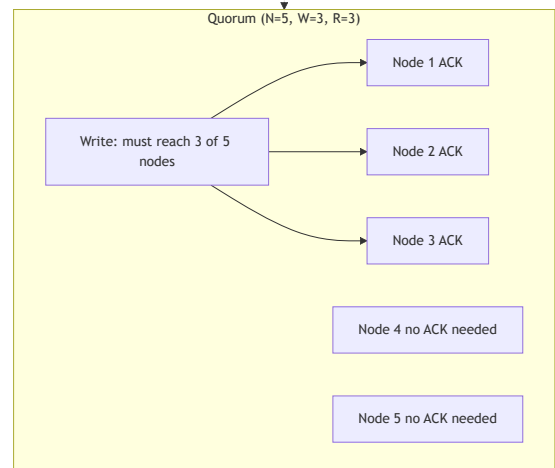
Quorum and Consensus

In a distributed system with N replicas, operations require agreement from a **quorum** — typically a majority of nodes. For writes requiring acknowledgement from W nodes and reads querying R nodes, if $W + R > N$, at least one node seen by a read must have participated in the last write, guaranteeing the latest data is always seen. Consensus algorithms like **Raft** solve the harder problem of getting distributed nodes to agree on a sequence of values even when nodes fail. Raft elects a leader by timeout, the leader appends entries to its log and replicates to followers, and commits once a majority acknowledges.



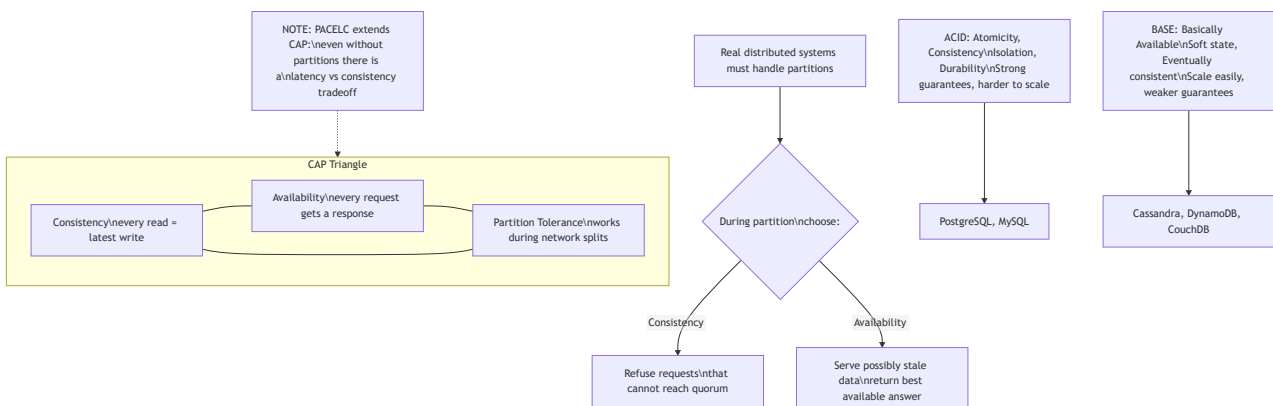
$W + R = 6$ greater than $N = 5$
At least 1 overlap guaranteed
Reads always see latest write

NOTE: Lower W means faster writes but less durability
tune based on your consistency requirements



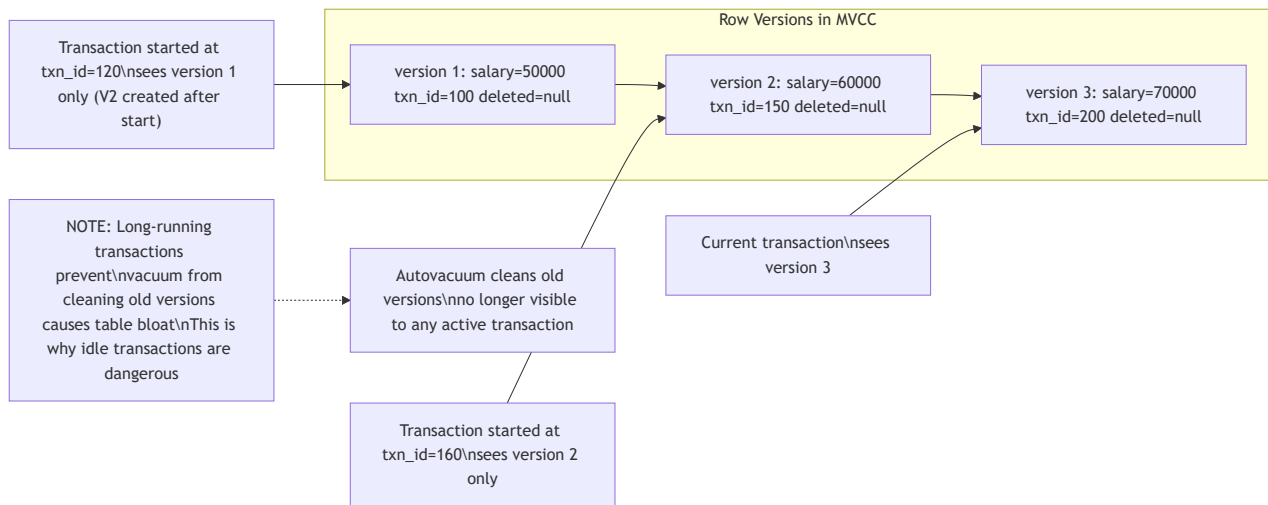
CAP Theorem and BASE vs ACID

CAP states a distributed system guarantees at most two of: **Consistency** (every read sees the most recent write), **Availability** (every request receives a response), and **Partition tolerance** (operation continues during network splits). Since network partitions will occur, you are really choosing between C and A during a partition. ACID — Atomicity, Consistency, Isolation, Durability — describes traditional database guarantees. BASE — Basically Available, Soft state, Eventually consistent — describes many distributed and NoSQL systems that trade ACID's hard guarantees for higher availability and performance.



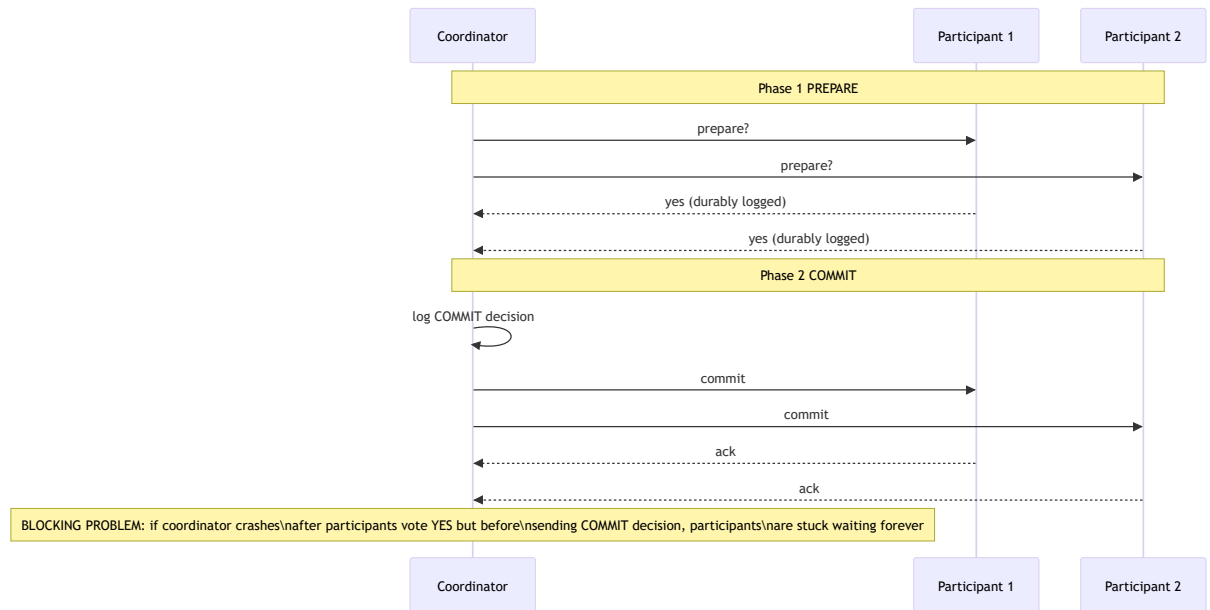
MVCC and Snapshot Isolation

MVCC (Multi-Version Concurrency Control) eliminates the reader-writer blocking problem by storing **multiple versions** of each row. Writers create new versions without overwriting old ones. Readers see the version that was current at their transaction's start time — effectively a snapshot of the database. This is how PostgreSQL achieves excellent read throughput under write load: readers never block writers and vice versa. Snapshot Isolation prevents dirty reads and non-repeatable reads but allows **write skew** — two transactions each check a condition, both find it satisfied, and together their writes violate it. Serializable Snapshot Isolation (SSI) detects write skew at commit time.



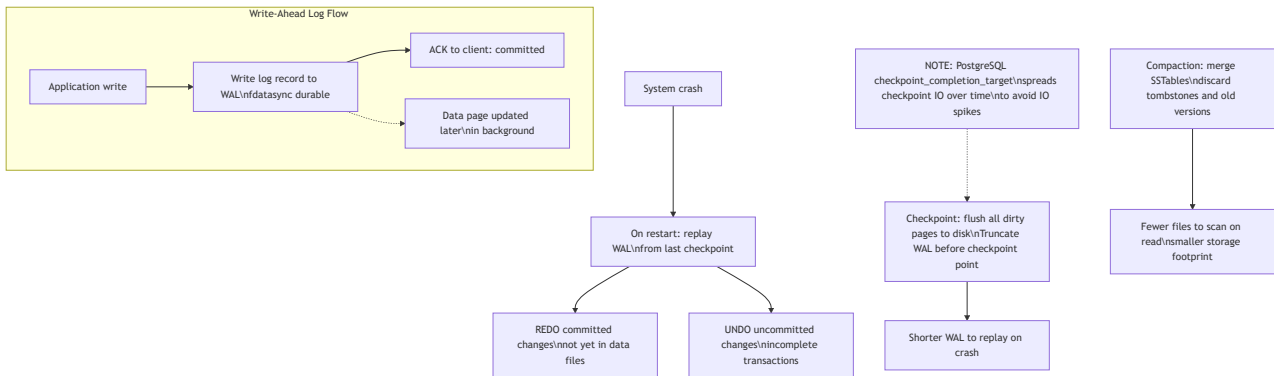
Two-Phase Commit and Three-Phase Commit

Two-Phase Commit (2PC) coordinates an atomic transaction across multiple nodes. Phase one: the coordinator asks all participants if they can commit. Each durably logs its vote. Phase two: if all voted yes, the coordinator logs a commit decision and tells everyone to commit. The critical problem: if the coordinator crashes after participants voted yes but before it sends the final decision, participants are stuck — they cannot commit or abort without the coordinator. This **blocking problem** persists until the coordinator recovers. Three-Phase Commit adds a pre-commit phase to reduce this window, but 3PC assumes synchronous networks and fails under network partitions.



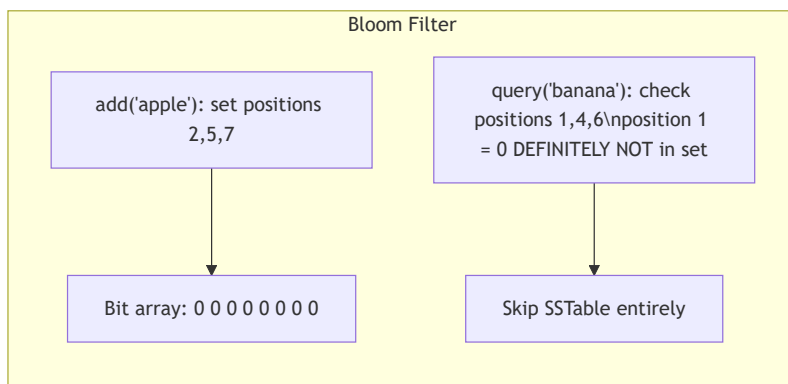
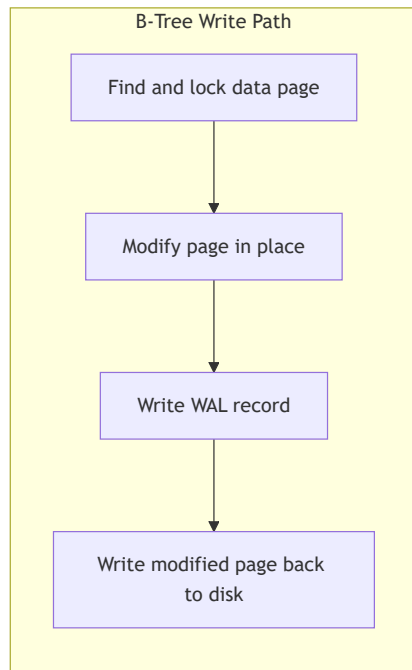
WAL, Checkpointing, and Compaction

Write-Ahead Logging ensures durability: before any data page is modified on disk, a log record describing the change is written to the WAL. Only after the log record is durably on disk is the change considered committed. On crash, the database replays the WAL from the last checkpoint to recover committed changes not yet in data files, and rolls back uncommitted ones. A checkpoint is the point in the WAL at which all modified pages before it have been flushed to disk, bounding recovery time. Compaction, relevant to LSM-tree engines, merges accumulated sorted files (SSTables), discards deleted and overwritten values, and reduces the number of files that reads must scan.

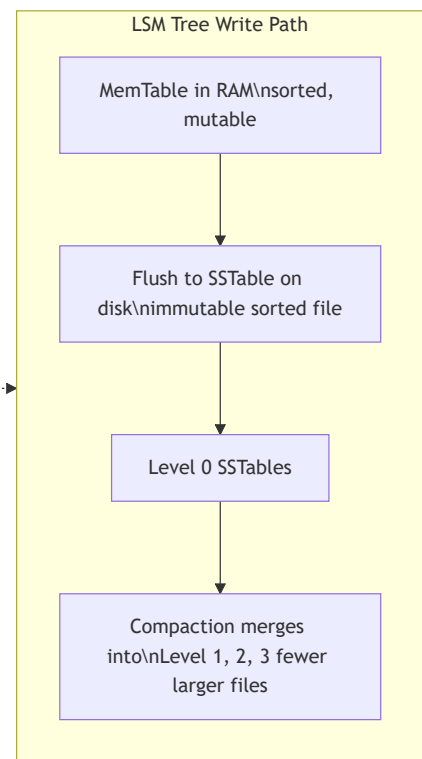


Bloom Filter and LSM Tree vs B-Tree

A Bloom filter answers set-membership queries with no false negatives but configurable false positives. Internally it is a bit array with multiple hash functions: to add, hash the element and set those bit positions; to query, check if all positions are set. If any position is zero the element is definitely absent. Cassandra uses Bloom filters to avoid checking SSTables that cannot contain a queried key. LSM trees optimise for writes: writes go to an in-memory memtable, flushed to immutable sorted SSTables on disk. Compaction merges SSTables. B-trees modify pages in place, making random reads fast but writes expensive due to scattered page updates.



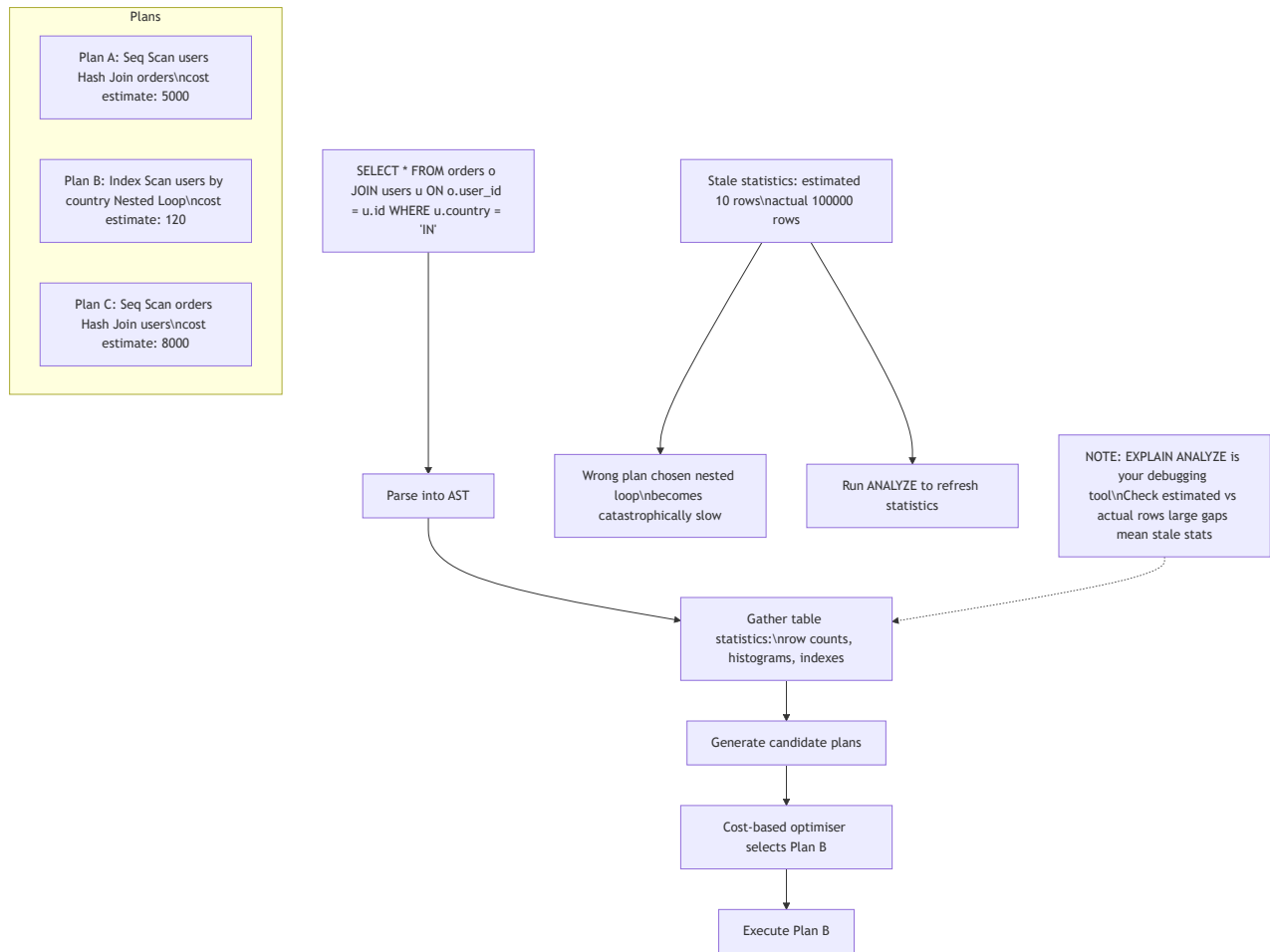
NOTE: LSM excels for write-heavy workloads
B-tree excels for read-heavy with random lookups
Choose engine based on your workload



Query Planner and Cost-Based Optimizer

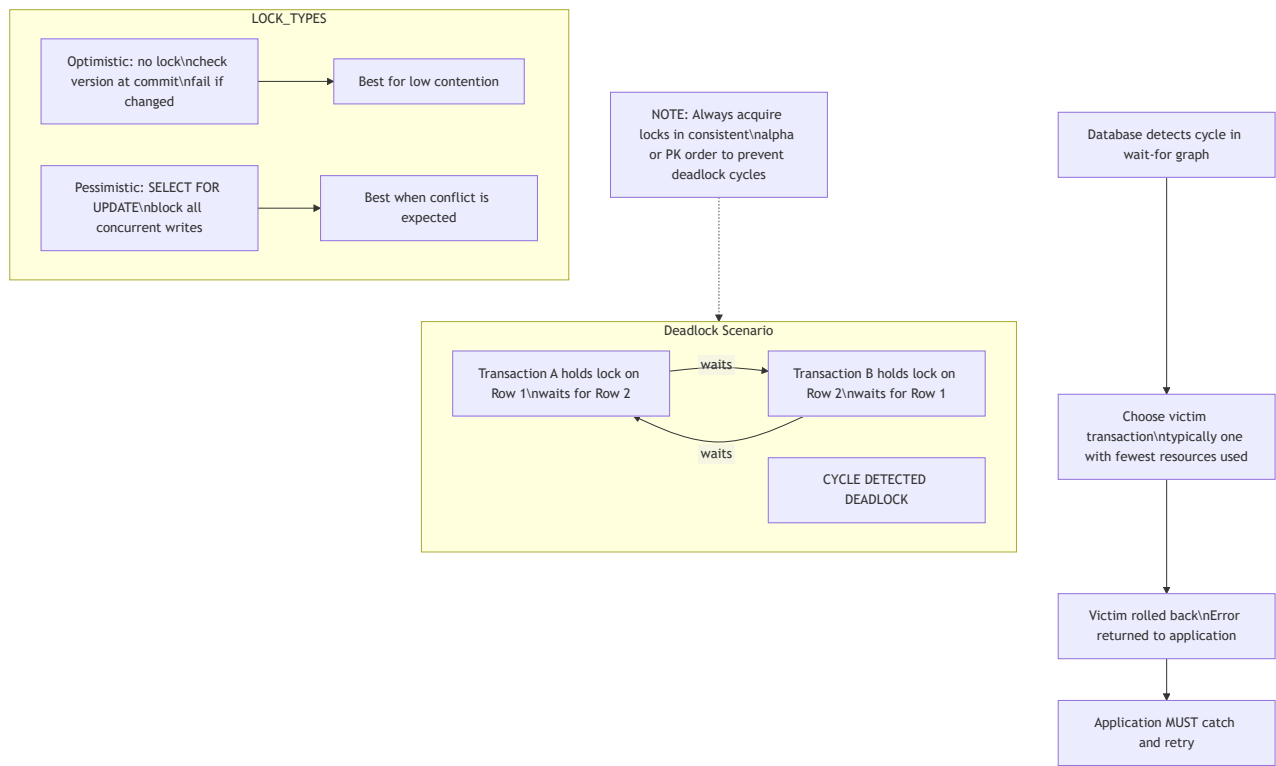
The query planner parses your SQL and generates an execution plan. The **cost-based optimiser** evaluates multiple candidate plans and selects the cheapest, estimating cost from table statistics: row counts, column cardinality, value distribution histograms, and correlation. For a join the optimiser chooses between nested-loop join (good for small tables), hash join (good for large unsorted inputs), and merge join (good for pre-sorted

inputs). With N tables there are $N!$ possible join orders — optimisers use dynamic programming or greedy heuristics to explore the search space. A large discrepancy between estimated and actual rows in `EXPLAIN ANALYZE` signals stale statistics — run `ANALYZE` to refresh.



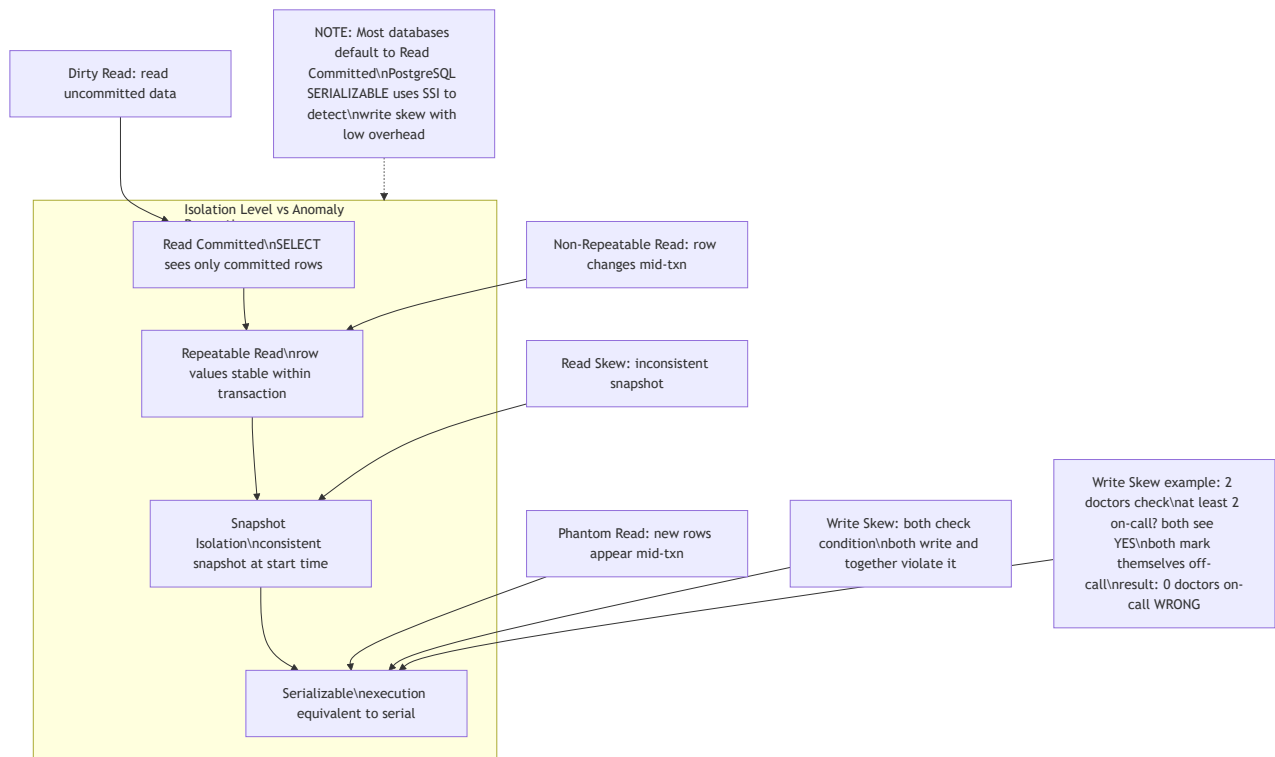
Deadlock and Lock Types

A deadlock occurs when transaction A holds a lock transaction B needs and vice versa — both wait forever. Databases detect deadlocks using a **wait-for graph**: a cycle means deadlock. One transaction is chosen as victim and killed, allowing others to proceed. Prevention: always acquire locks in a consistent order across all transactions; keep transactions short; prefer MVCC to avoid locking entirely where possible. Lock escalation replaces many fine-grained row locks with a coarser table lock when the number of locks exceeds a threshold. **Optimistic locking** (version columns) acquires no locks and validates at commit. **Pessimistic locking** (`SELECT FOR UPDATE`) holds locks during the transaction.



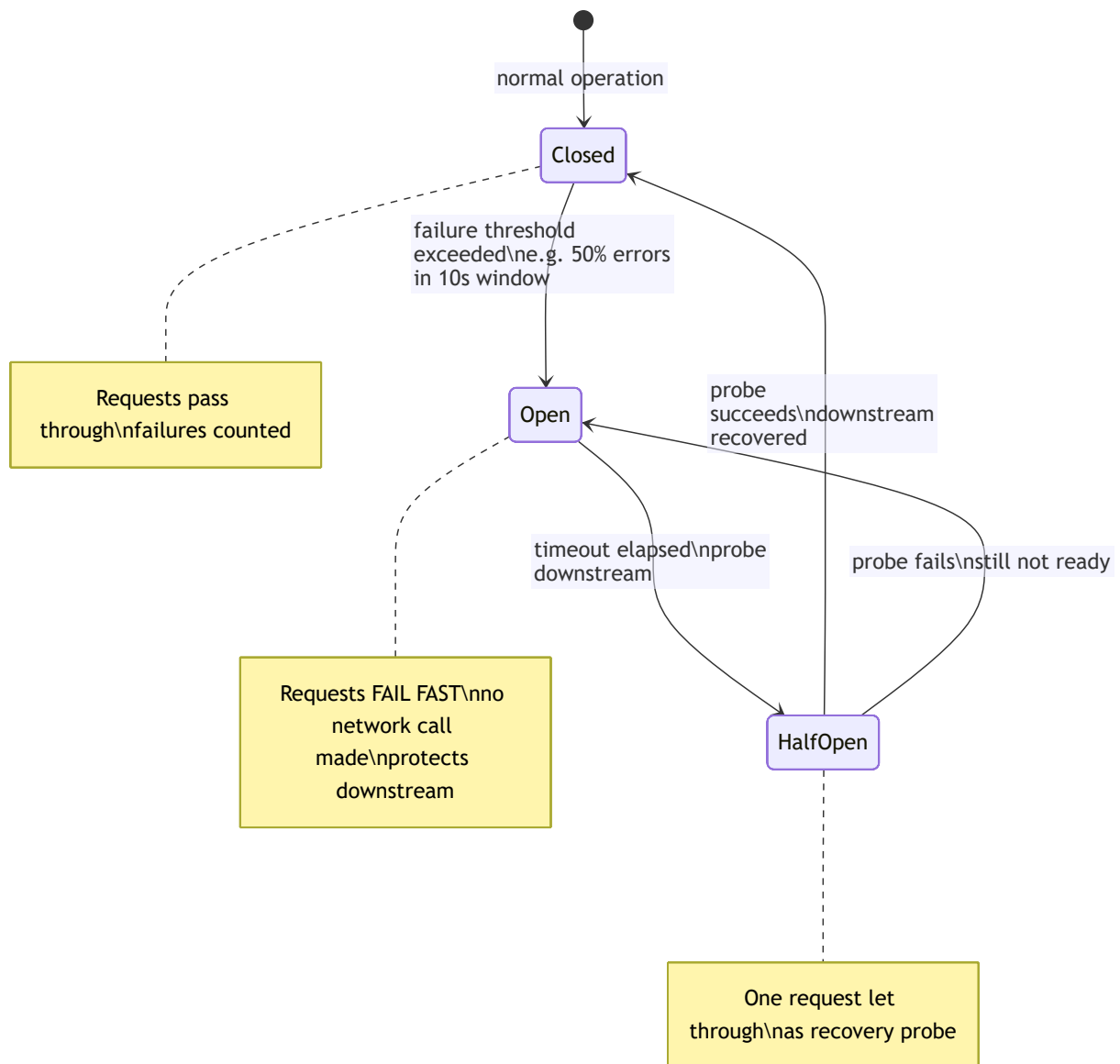
Isolation Anomalies

SQL defines four isolation levels, each preventing a subset of anomalies. **Dirty read:** reading uncommitted data that may be rolled back. **Non-repeatable read:** the same row returns different values within one transaction because another committed a change. **Phantom read:** a range query returns different rows on re-execution because another transaction inserted or deleted matching rows. **Read skew:** reading two related rows that form an inconsistent snapshot — prevented by Snapshot Isolation. **Write skew:** two transactions each read a shared condition, decide to write based on it, and together violate the invariant — prevented only by Serializable.



Backpressure, Circuit Breaker, and Rate Limiting

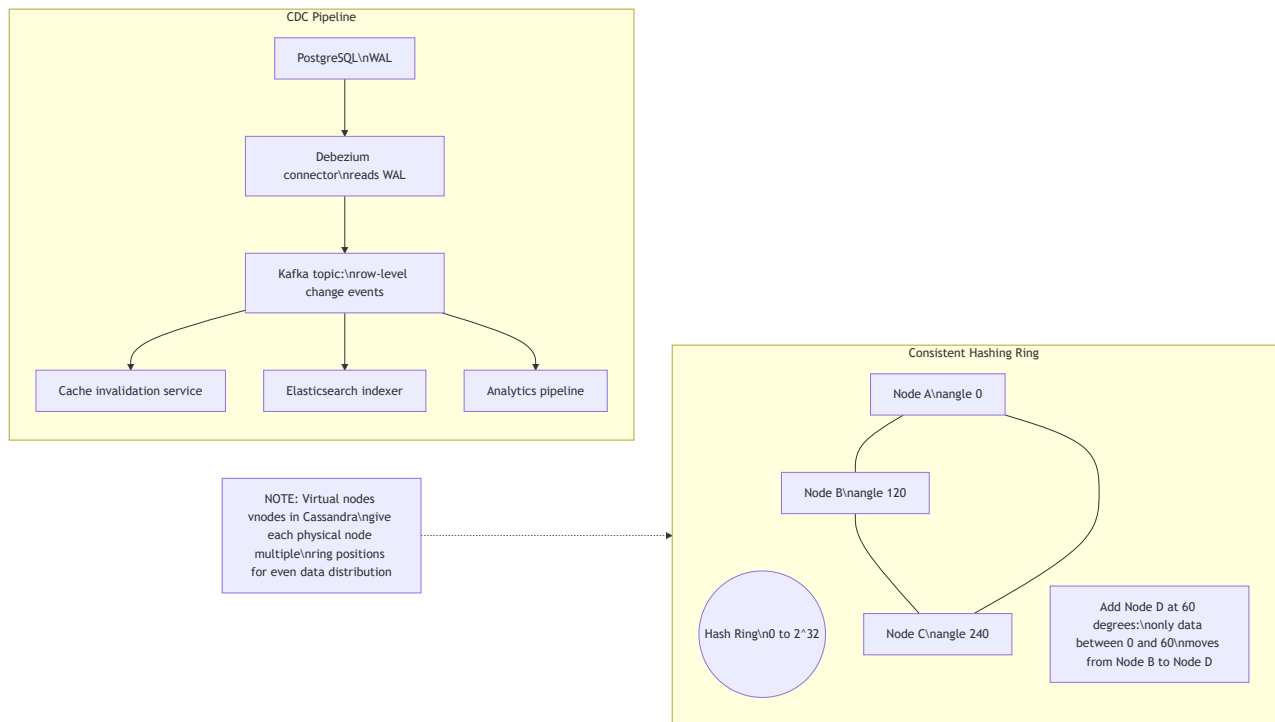
Backpressure is a flow-control signal from a downstream component telling an upstream to slow down production. Instead of dropping data or crashing, the upstream pauses until downstream catches up. Reactive Streams builds backpressure into the protocol between publisher and subscriber. A **circuit breaker** has three states: Closed (requests pass through), Open (requests fail fast after too many failures — no call attempted), and Half-Open (a probe request tests if downstream has recovered). This prevents cascading failures. Rate limiting restricts how many requests a specific client can make in a window. Throttling limits how many requests the service processes globally, protecting it from overload.



CDC, Consistent Hashing, and Partitioning

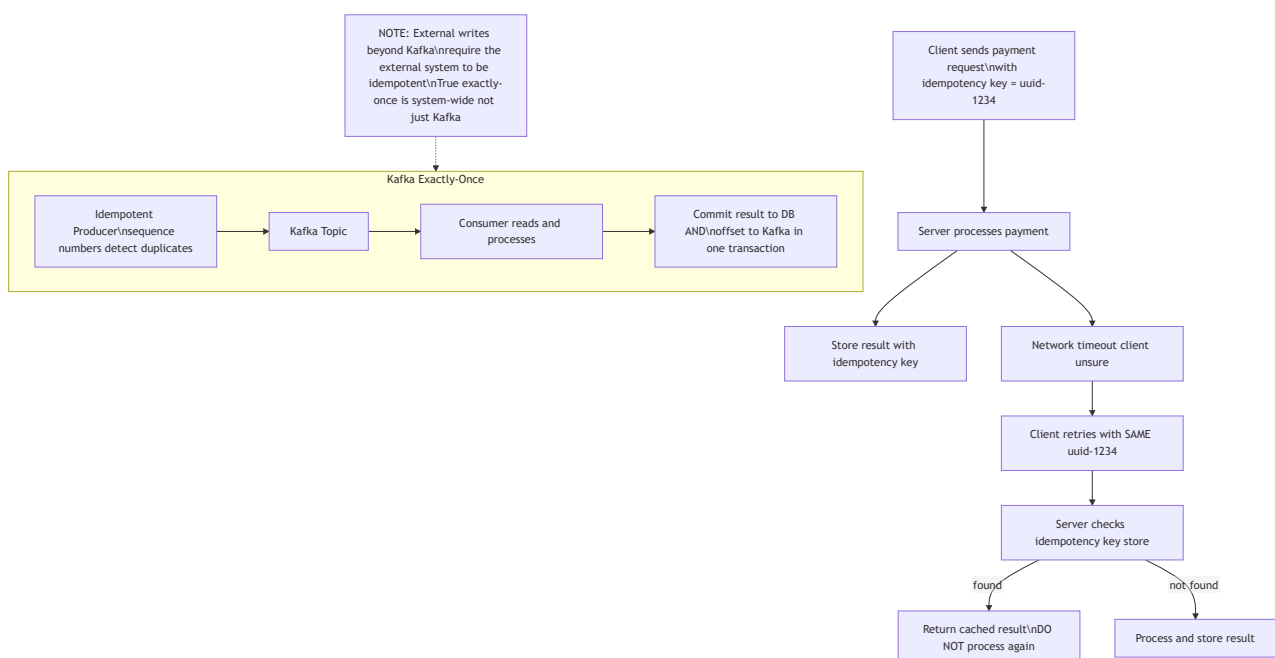
Change Data Capture (CDC) reads the database WAL to produce a real-time stream of every insert, update, and delete as events, enabling real-time integration, cache invalidation, and event sourcing. Debezium is the leading open-source CDC tool.

Consistent hashing places both data items and nodes on a hash ring; adding or removing a node only moves data from adjacent positions, minimising redistribution. This is why Cassandra and DynamoDB can add nodes without reshuffling all data. Hash partitioning distributes rows uniformly but loses range scan ability. Range partitioning enables range scans but risks hot partitions when all writes go to the current time-range shard.



Idempotency and Exactly-Once Semantics

An **idempotent** operation produces the same result whether executed once or many times. PUT with a full resource is idempotent; POST to create is not. Idempotency is essential in distributed systems where retries are necessary — a timeout does not tell you if the server received your request. With idempotent operations you can safely retry. **Exactly-once semantics** guarantees a message is processed exactly once, neither lost nor duplicated. Kafka achieves this within the Kafka ecosystem: idempotent producers use sequence numbers to detect and discard duplicates, transactional producers atomically write across partitions, and consumer offset management is atomically committed with processing results.

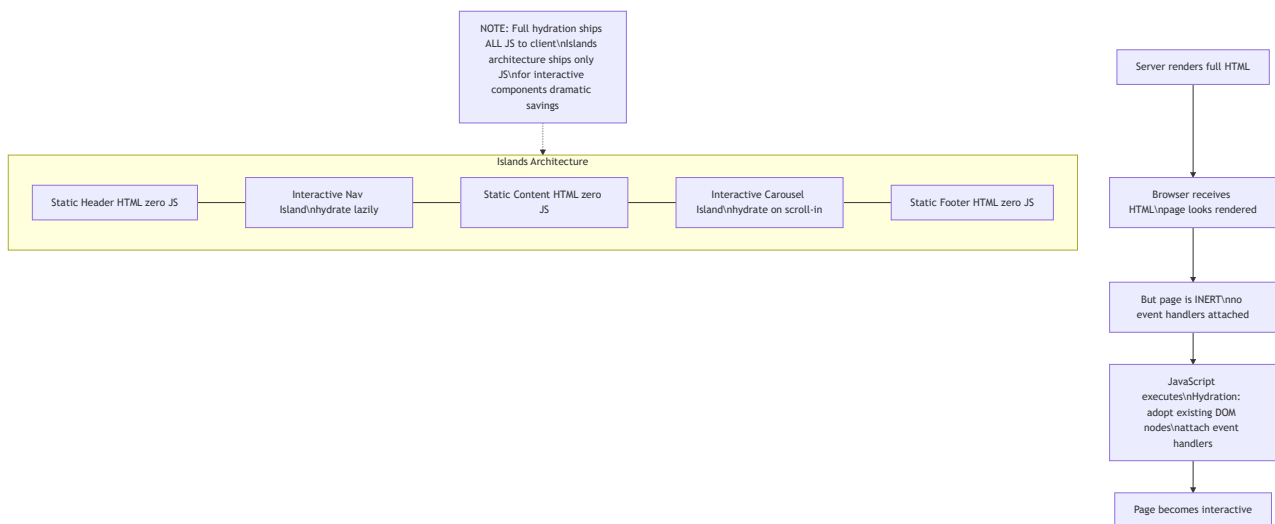




Part 3 — Frontend Engineering

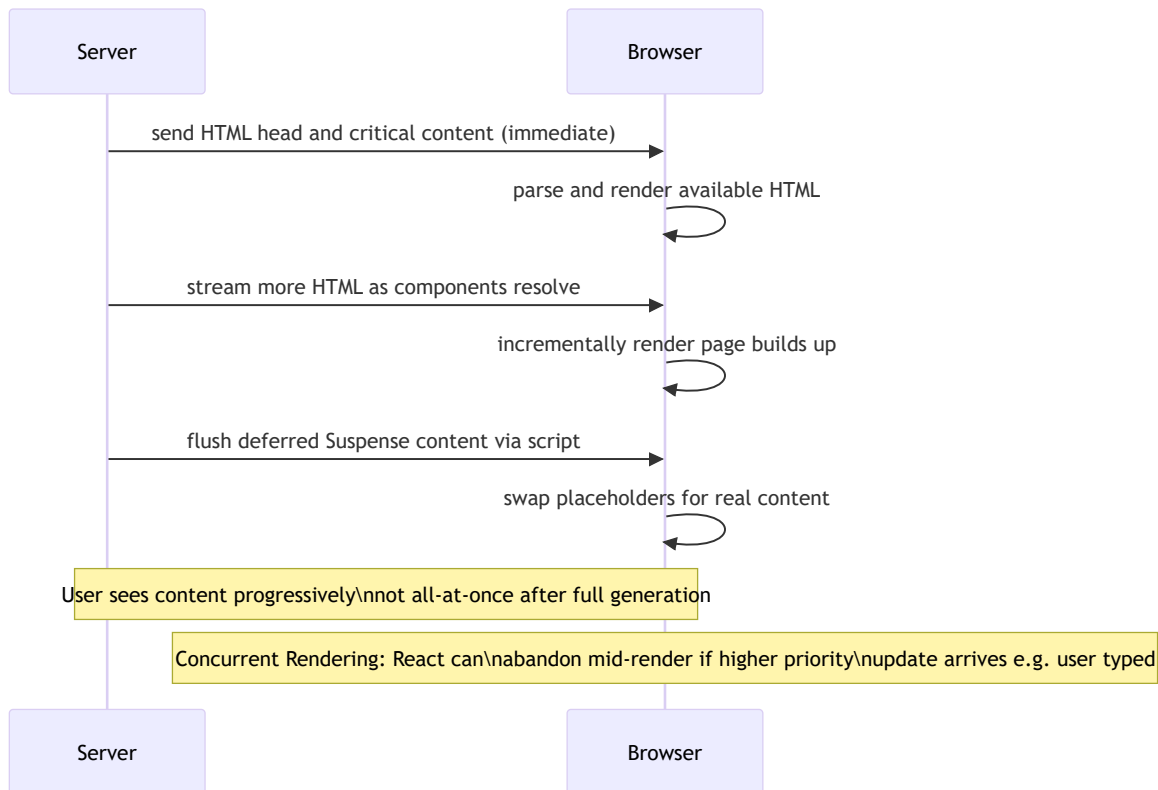
Hydration and Islands Architecture

When a server renders HTML and sends it to the browser, the page looks correct but is inert — no event handlers are attached. **Hydration** is the process of attaching JavaScript event handlers and state to the already-existing server-rendered DOM. React walks the HTML, matches it against the component tree, and adopts the existing DOM nodes rather than recreating them. Full hydration requires shipping all component code to the client regardless of interactivity. **Partial hydration** hydrates only interactive parts. **Islands architecture** formalises this: the page is mostly static HTML with isolated islands of interactivity, each hydrated independently and lazily. Astro popularised this pattern — static regions have zero JavaScript cost.



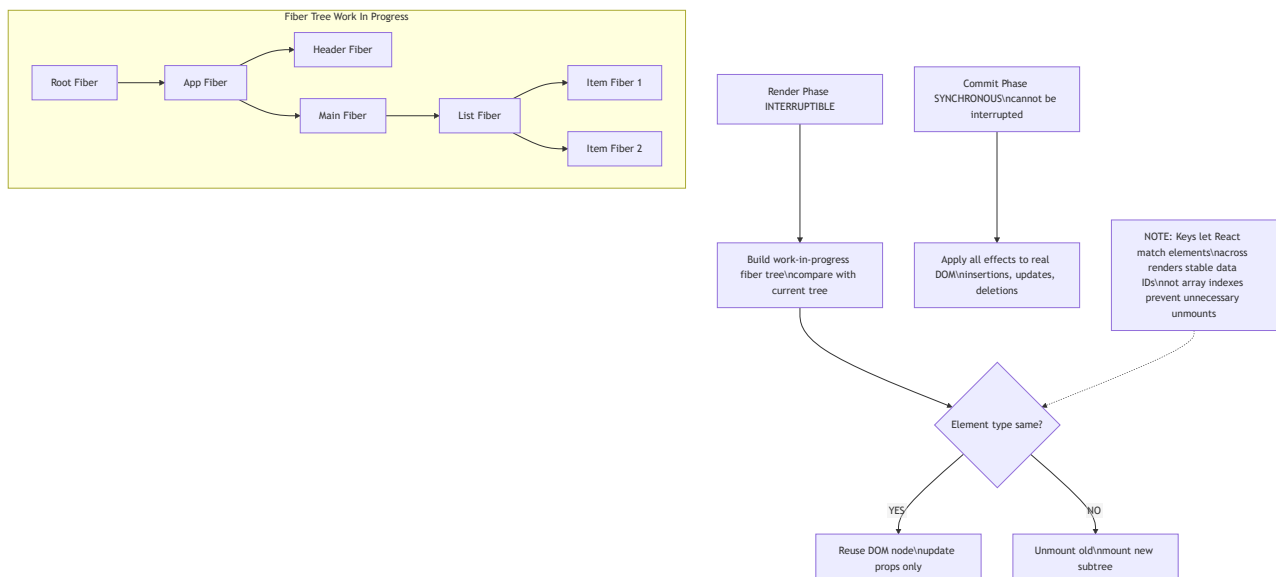
Streaming SSR and Concurrent Rendering

Traditional SSR generates the complete HTML string on the server and sends it all at once — the browser waits for the entire response before it can start rendering. **Streaming SSR** sends HTML to the browser incrementally as it is generated. React 18's `renderToPipeableStream` supports streaming with `Suspense`: the server sends a placeholder for suspended components, then flushes the actual content as it becomes available via an inline script swap. **Concurrent rendering** lets React interrupt, pause, abandon, and restart render work. When a higher-priority update arrives mid-render, React can abandon the current render, handle the urgent update first, and resume the background render — previously impossible.



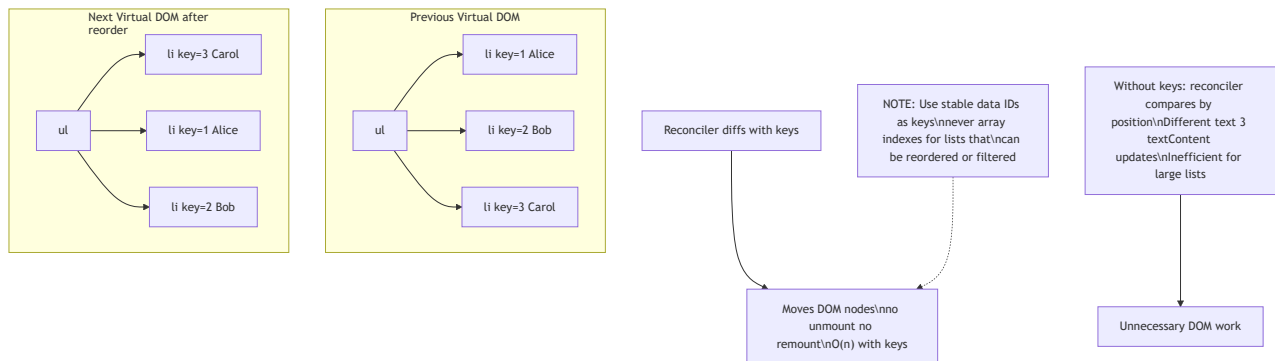
React Fiber Architecture and Reconciliation

The **Fiber** reimplementation (React 16) makes concurrent rendering possible. A fiber is a JavaScript object representing a unit of work — a node in a work-in-progress tree corresponding to a React element. Each fiber tracks component type, props, state, effects (what needs changing in the DOM), and pointers to parent, child, and sibling fibers. This explicit linked structure means React can traverse and interrupt work incrementally. The **render phase** builds a work-in-progress fiber tree (interruptible). The **commit phase** applies effects to the DOM synchronously (non-interruptible). Reconciliation diffs two fiber trees using type-based heuristics and key matching to find the minimal set of DOM changes.



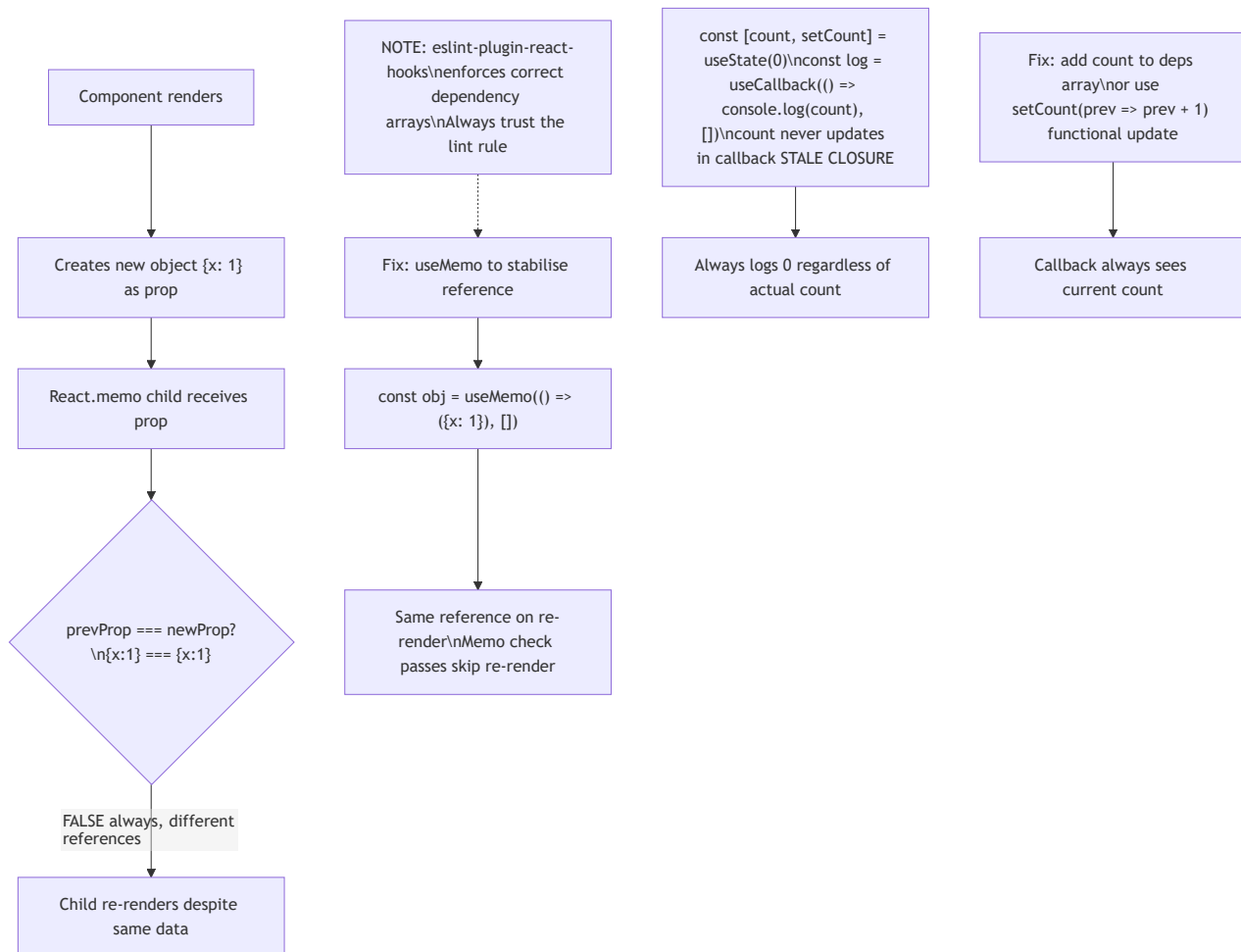
Virtual DOM, Structural Sharing, and Immutability

The virtual DOM is an in-memory tree representation. React diffs it against the previous version to find real DOM changes. Naive tree diffing is $O(n^3)$. React achieves $O(n)$ with two heuristics: elements of different types produce completely different trees, and keys identify elements across renders. **Structural sharing** makes immutability efficient: a modified data structure shares unchanged subtrees with the original rather than deep-copying. Combined with referential equality checks (`newState === oldState`), this lets `React.memo` and `PureComponent` skip re-renders cheaply when data provably has not changed.



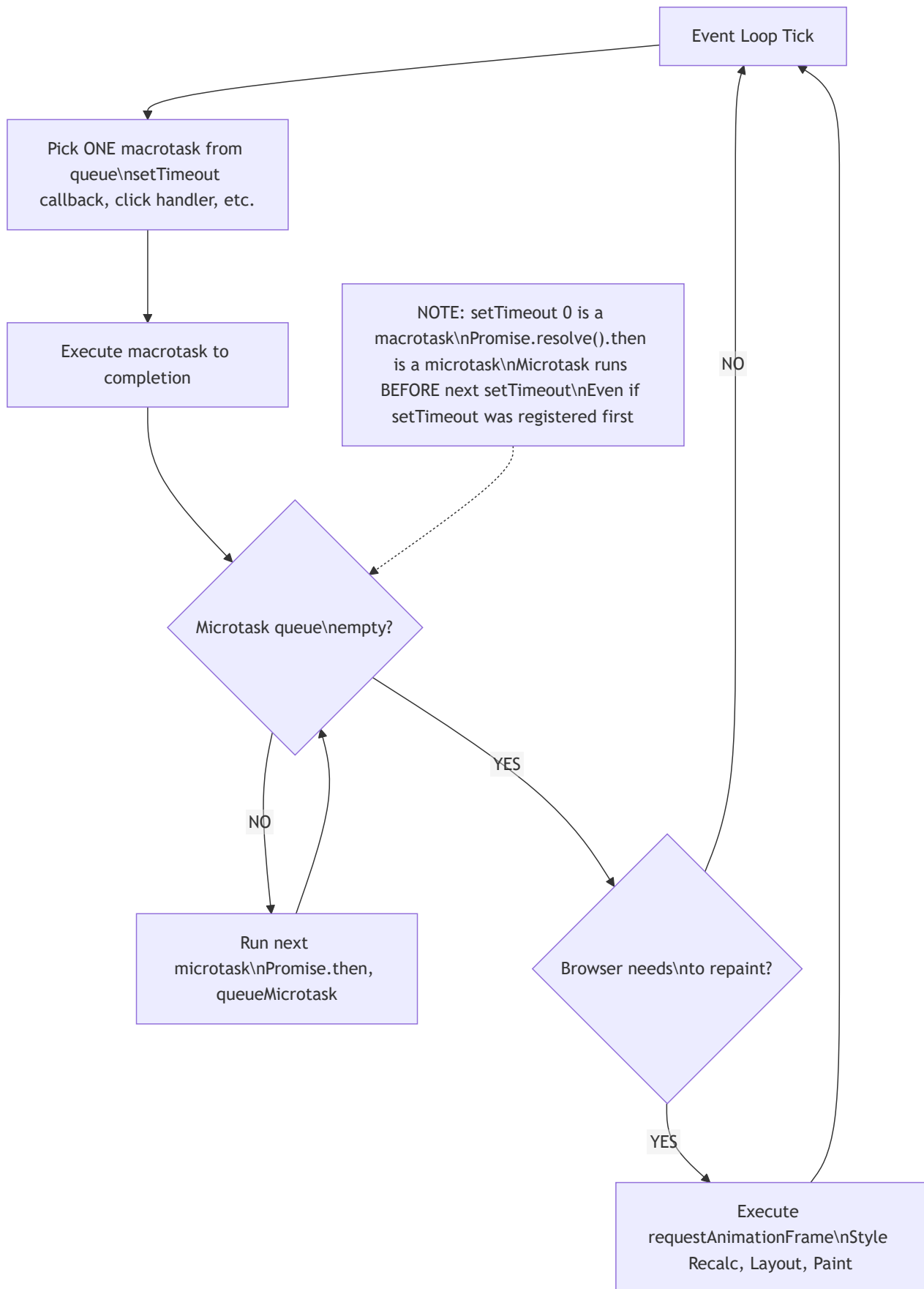
Memoization, Stale Closures, and Referential Equality

`React.memo` and `PureComponent` compare props by referential equality. Passing a new object or function literal on every render bypasses memoization — `{a:1} !== {a:1}`. `useMemo` returns a stable computed value when dependencies are unchanged. `useCallback` returns a stable function reference. The pitfalls: over-memoizing adds overhead without benefit for cheap computations; incorrect dependency arrays cause **stale closures**. A stale closure captures a variable's value at creation time. If a `useCallback` omits a state variable from its dependency array, the callback forever reads the initial value — a subtle bug that appears as a counter that does not count.



Event Loop — Macro vs Microtasks

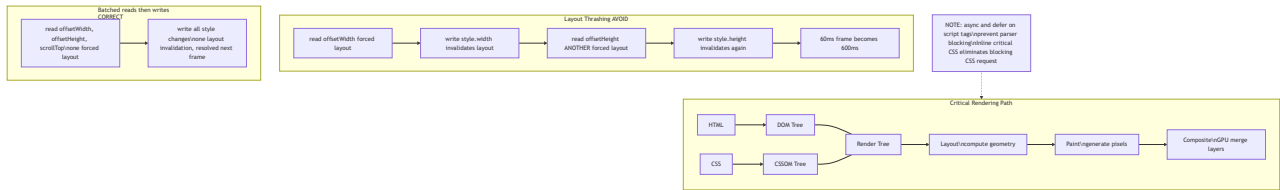
JavaScript is single-threaded. The event loop processes one task at a time. A **macrotask** queue holds: `setTimeout` callbacks, `setInterval` callbacks, event handler callbacks, and I/O completions. A **microtask** queue holds: `Promise .then()` callbacks, `queueMicrotask()`, and `MutationObserver` callbacks. After each macrotask completes, the engine drains the **entire** microtask queue before picking the next macrotask. This means Promise resolutions run before the next `setTimeout`, before the browser repaints, before anything else. A chain of Promises resolving synchronously can delay rendering — starving the UI without appearing as a long task because each individual microtask is short.



Layout Thrashing and Critical Rendering Path

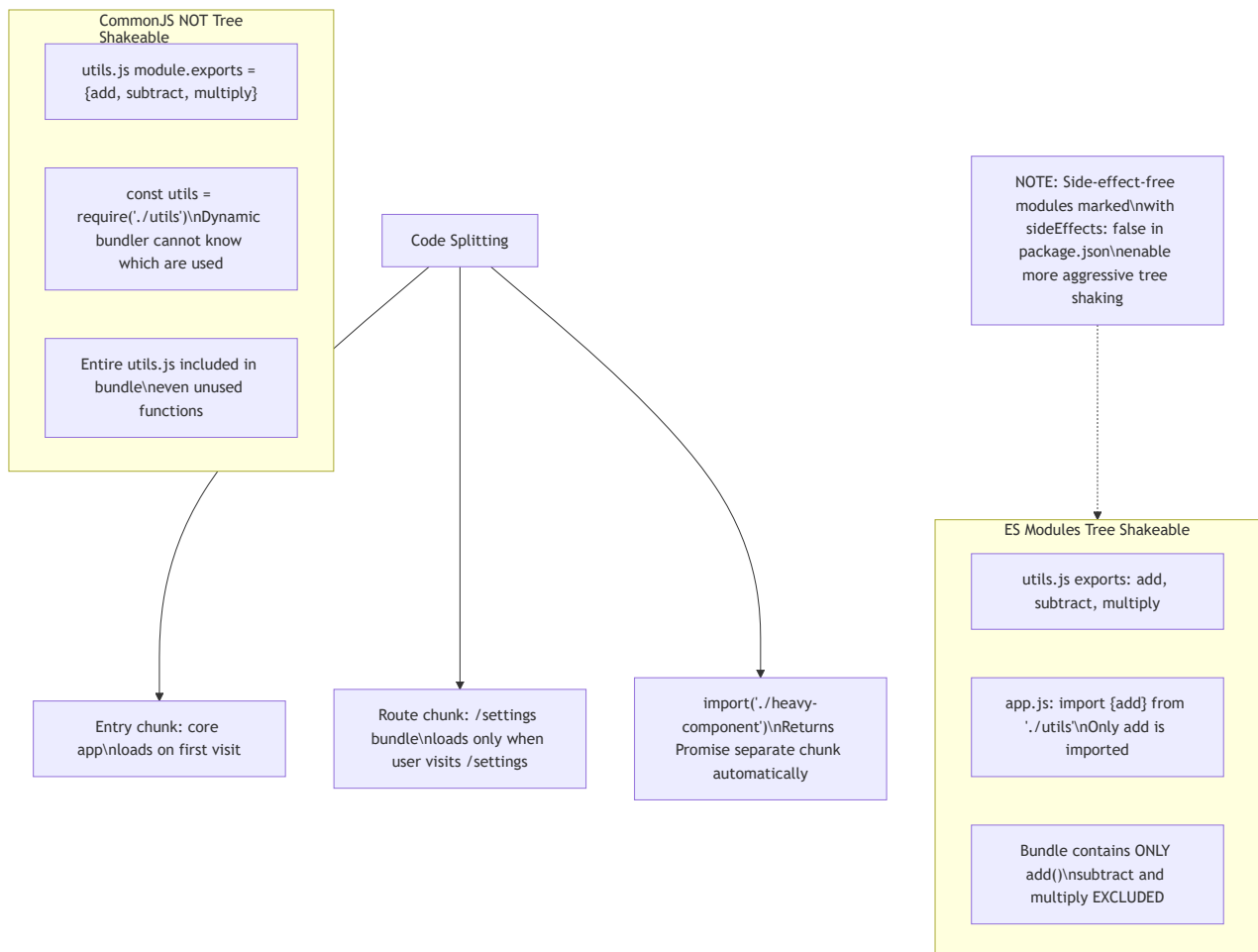
Layout thrashing occurs when JavaScript reads then writes DOM properties alternately and rapidly. Reading a layout property like `offsetWidth` forces the browser to complete

pending style calculations and layout — a forced synchronous layout. If you immediately write to the DOM and then read again, another forced layout occurs. Dozens of these in a loop destroy frame rate. Batch all reads first, then all writes. The **critical rendering path** is the sequence for the first pixel: HTML parsing builds the DOM, CSS parsing builds the CSSOM, combining them forms the render tree, layout computes geometry, paint generates pixels, and composite merges GPU layers. CSS is render-blocking. Synchronous scripts are both render-blocking and parser-blocking.



Tree Shaking and Code Splitting

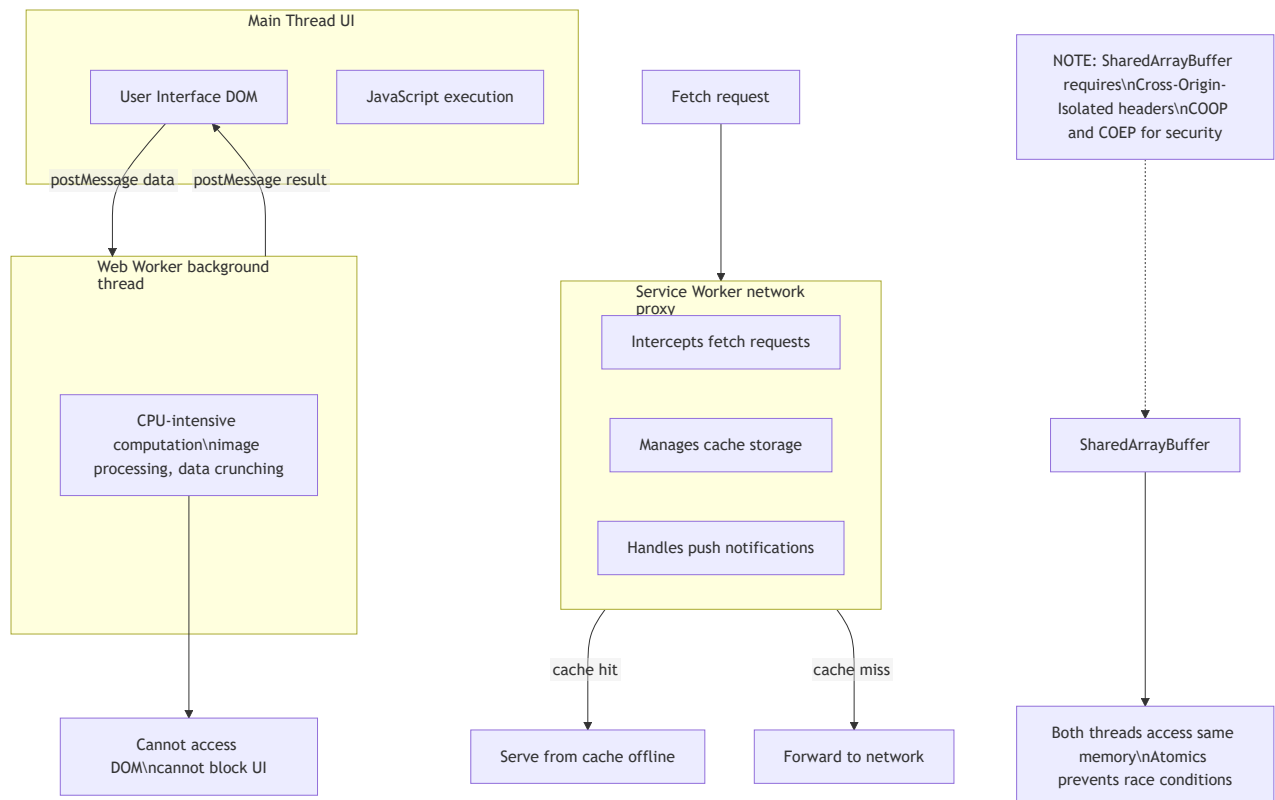
Tree shaking is the elimination of unused code during bundling. Bundlers like Webpack and Rollup analyse the static ES module import-export graph and exclude any code not reachable from an entry point. This requires ES modules; CommonJS `require()` is dynamic and cannot be statically analysed. **Code splitting** divides your bundle into chunks loaded on demand. Route-based splitting loads the settings page bundle only when the user navigates there. Dynamic `import()` is the mechanism — it returns a Promise and the bundler automatically creates a separate chunk.



Web Workers vs Service Workers and SharedArrayBuffer

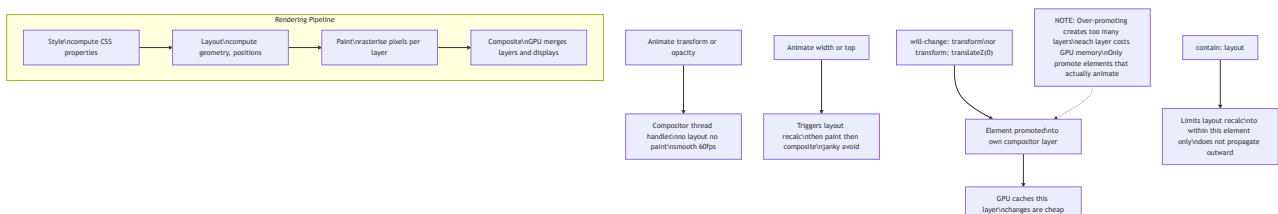
Web Workers are general-purpose background threads for CPU-intensive computation. They cannot touch the DOM but communicate with the main thread via `postMessage`.

Service Workers are event-driven proxy servers between your app and the network, intercepting fetch requests, managing caches, enabling offline support, and handling push notifications. They persist across browser sessions and have a lifecycle of `install`, `activate`, and `fetch` events. **SharedArrayBuffer** enables true shared memory between workers — multiple threads access the same buffer simultaneously. `Atomics` provides synchronisation primitives to coordinate access safely.



Browser Compositing Layers and GPU Acceleration

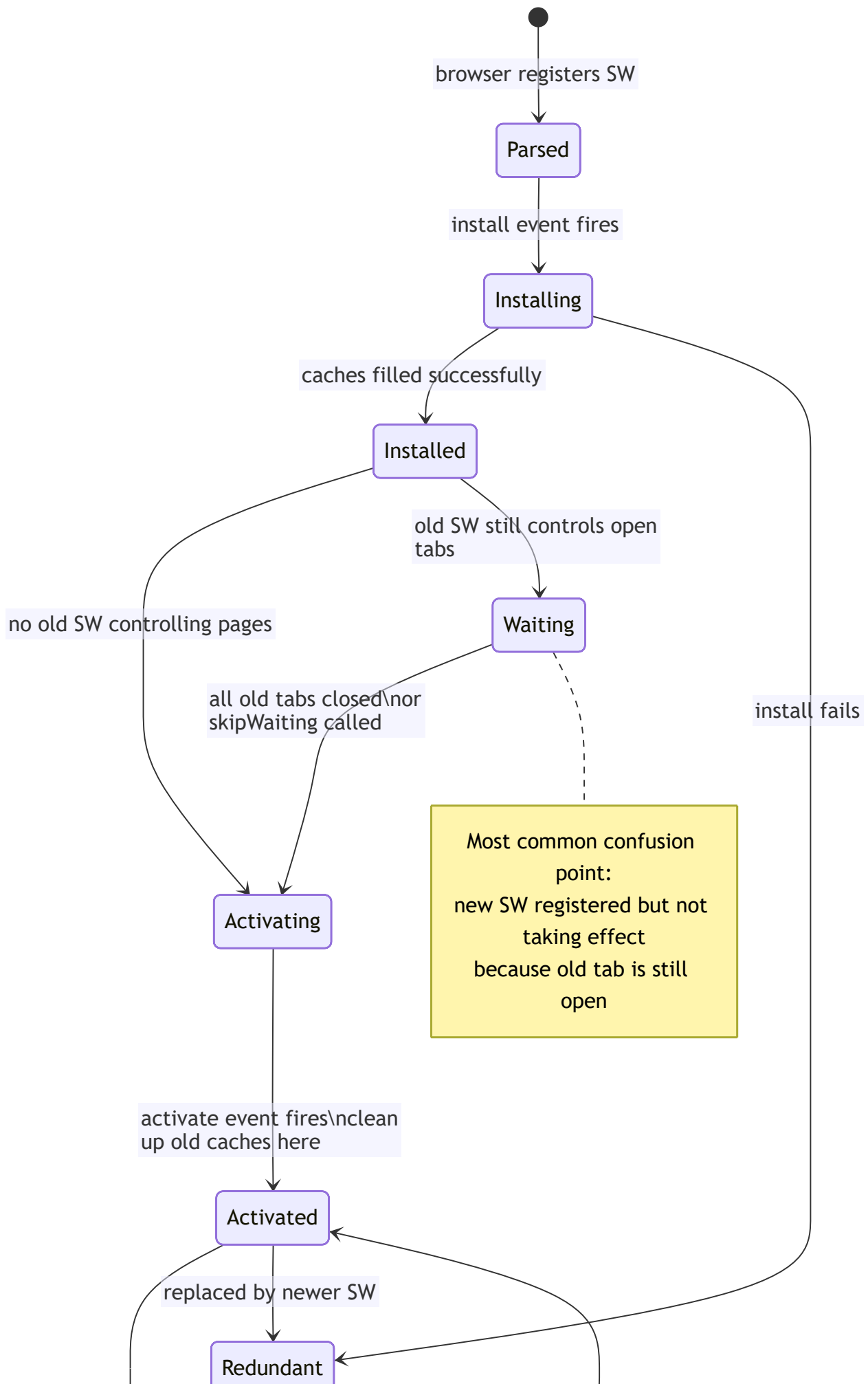
The browser renders through four stages: **Style**, **Layout** (compute geometry and position), **Paint** (rasterise pixels for each layer), and **Composite** (merge layers on the GPU). Layout and paint are expensive. Compositing is cheap — the GPU handles it on the compositor thread, independent of the main thread. Promoting an element to its own compositor layer means changes only trigger compositing, skipping layout and paint entirely. This is why animating `transform` and `opacity` is smooth at 60fps even when the main thread is busy. Animating `width` or `top` triggers layout — avoid it for animations. `CSS containment` (`contain: layout`) limits the scope of layout recalculation.



Service Worker Lifecycle and Cache Strategies

A Service Worker installs when first registered, waits in the **waiting** state while tabs using the old worker remain open, then **activates** when all old tabs are closed. Common lifecycle traps: new service worker not activating immediately because old tabs are open; forgetting to delete old caches in the `activate` event. Cache strategies: **cache-first** serves from cache and only goes to network on miss — ideal for versioned static assets.

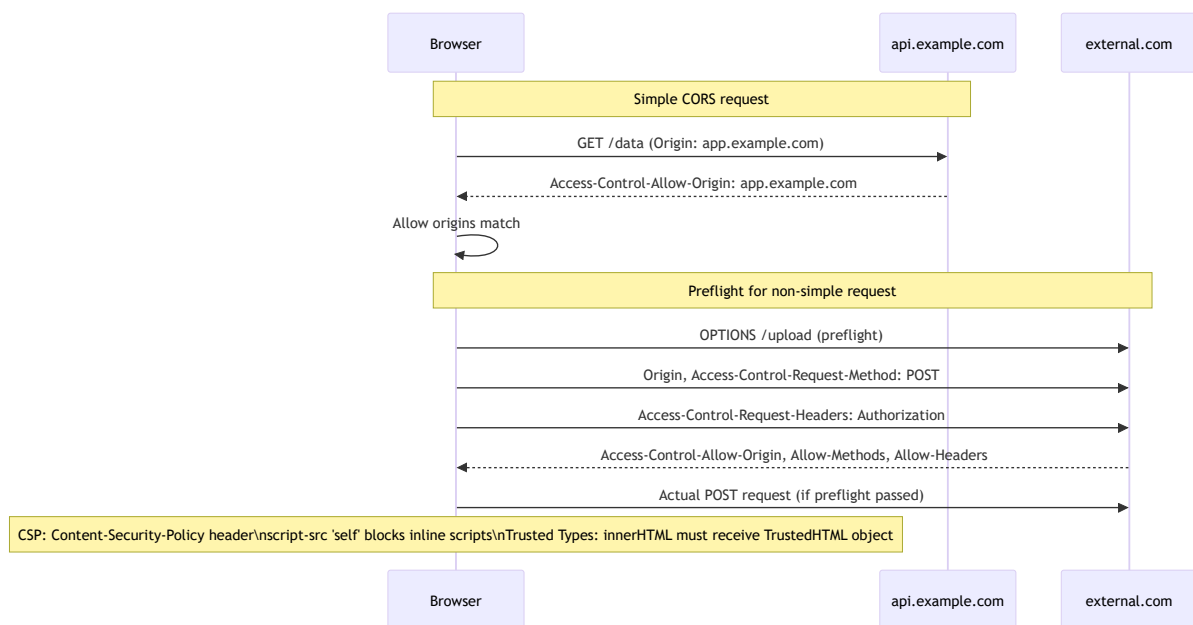
Network-first always tries network and falls back to cache on failure. **Stale-while-revalidate** serves cached content immediately while fetching an update in the background for next time.



handles fetch events
controls all matching pages

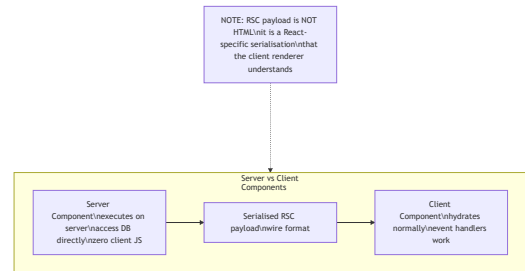
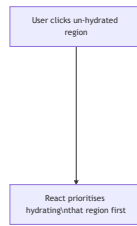
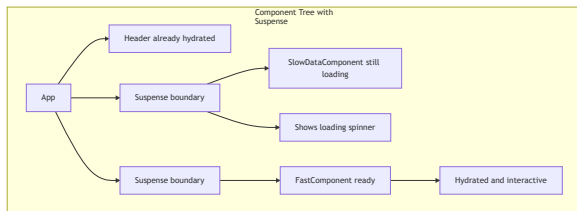
CORS, CSP, and Security Primitives

CORS restricts cross-origin resource requests. Simple requests are sent with an `Origin` header; the server must respond with `Access-Control-Allow-Origin` to permit access. Non-simple requests — those with custom headers or non-simple methods — trigger a **preflight** `OPTIONS` request first. **CSP** (Content Security Policy) is a response header declaring permitted sources for scripts, styles, and images. `script-src 'self'` blocks all injected inline scripts, defeating most XSS attacks. **Trusted Types** go further: requiring all dangerous DOM sinks like `innerHTML` to receive a Trusted value created through a defined policy, preventing DOM-based XSS at the browser level.



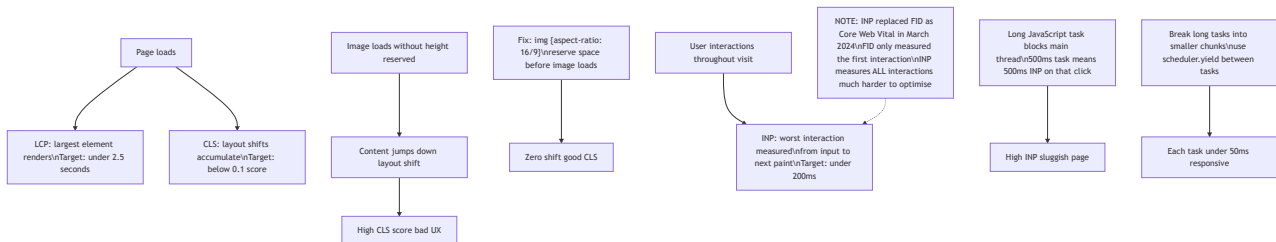
Suspense, Selective Hydration, and Server Components

Suspense boundaries wrap parts of the component tree that may suspend — throw a Promise during rendering to signal they are loading. The boundary shows a fallback while children are suspended. **Selective hydration** (React 18+) allows parts of the page to hydrate as they stream in. If a user clicks an un-hydrated region, React prioritises hydrating that part first — interaction drives hydration priority. **React Server Components** never run in the browser: they execute on the server, output a serialised React tree, and have zero client-side JS cost. Client components are regular React components that hydrate. RSC is the next major architectural shift in React.



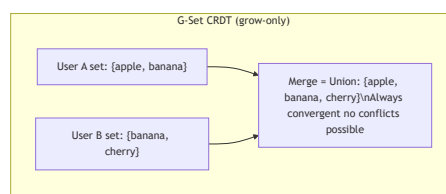
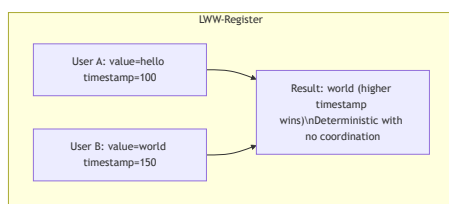
Web Vitals — LCP, CLS, INP, and FID

LCP (Largest Contentful Paint) measures when the largest visible content element — typically the hero image or main text block — finishes rendering. Target: under 2.5 seconds. **CLS** (Cumulative Layout Shift) measures unexpected visual instability — elements moving after apparent page load. An image without reserved dimensions causes content to jump when it loads. Target: below 0.1. **FID** (First Input Delay) measured delay from first interaction to browser starting to handle it — now superseded by **INP** (Interaction to Next Paint), which measures the worst interaction latency across the entire page visit. INP became a Core Web Vital in 2024. Target: under 200ms.

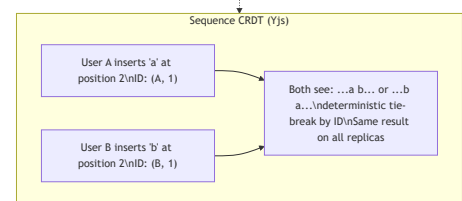


CRDTs and Offline Conflict Resolution

CRDTs (Conflict-free Replicated Data Types) are data structures mathematically designed so that concurrent modifications from multiple sources can always be merged without conflicts and without coordination. The simplest is the **G-Set**: elements can only be added, never removed, so any two replicas merge with set union. A **LWW-Register** (Last-Write-Wins) keeps the value with the highest timestamp. For collaborative text editing, **Sequence CRDTs** (as in Yjs and Automerge) give every character a unique identifier so concurrent insertions at the same position can be deterministically ordered without a server round-trip. Yjs is the standard library for collaborative web applications.

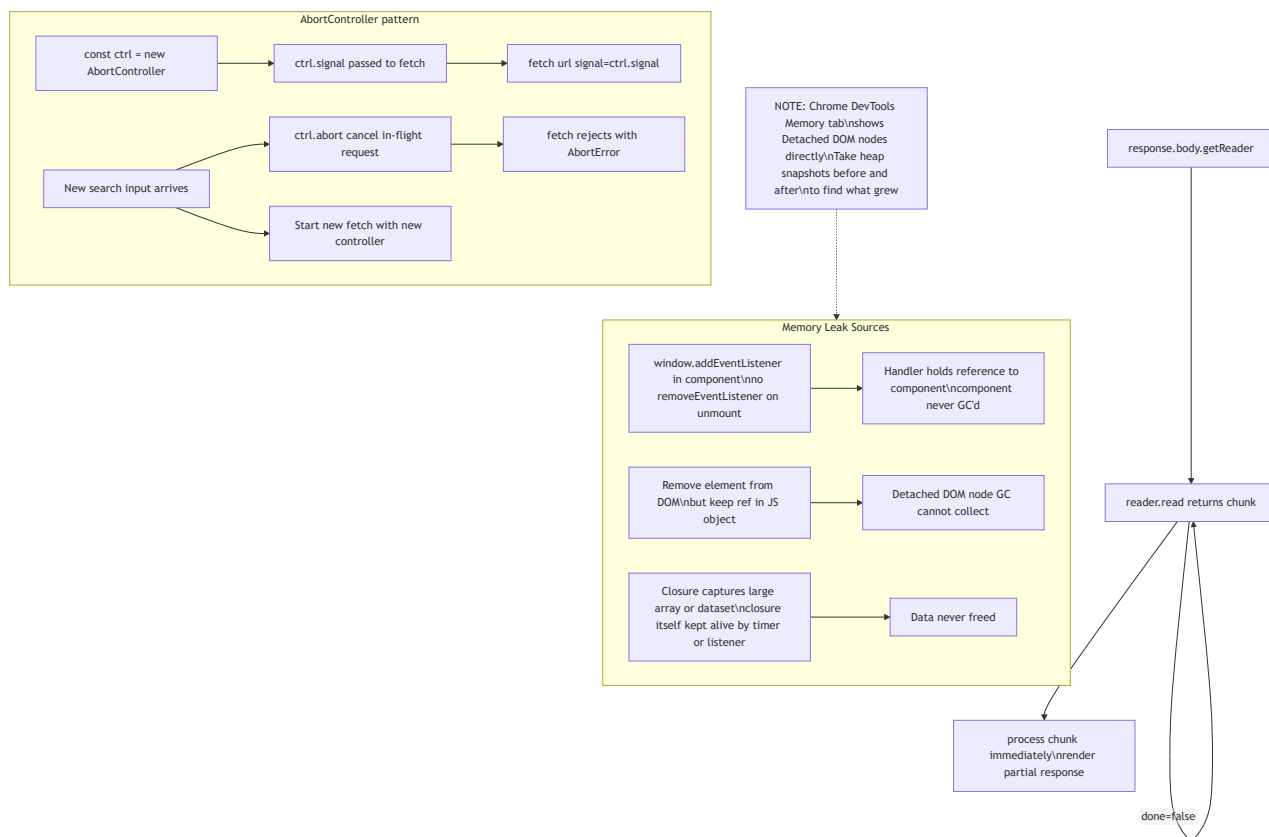


NOTE: CRDTs enable true peer-to-peer collaboration with no server arbitration required. Figma and Notion use sequence CRDTs internally



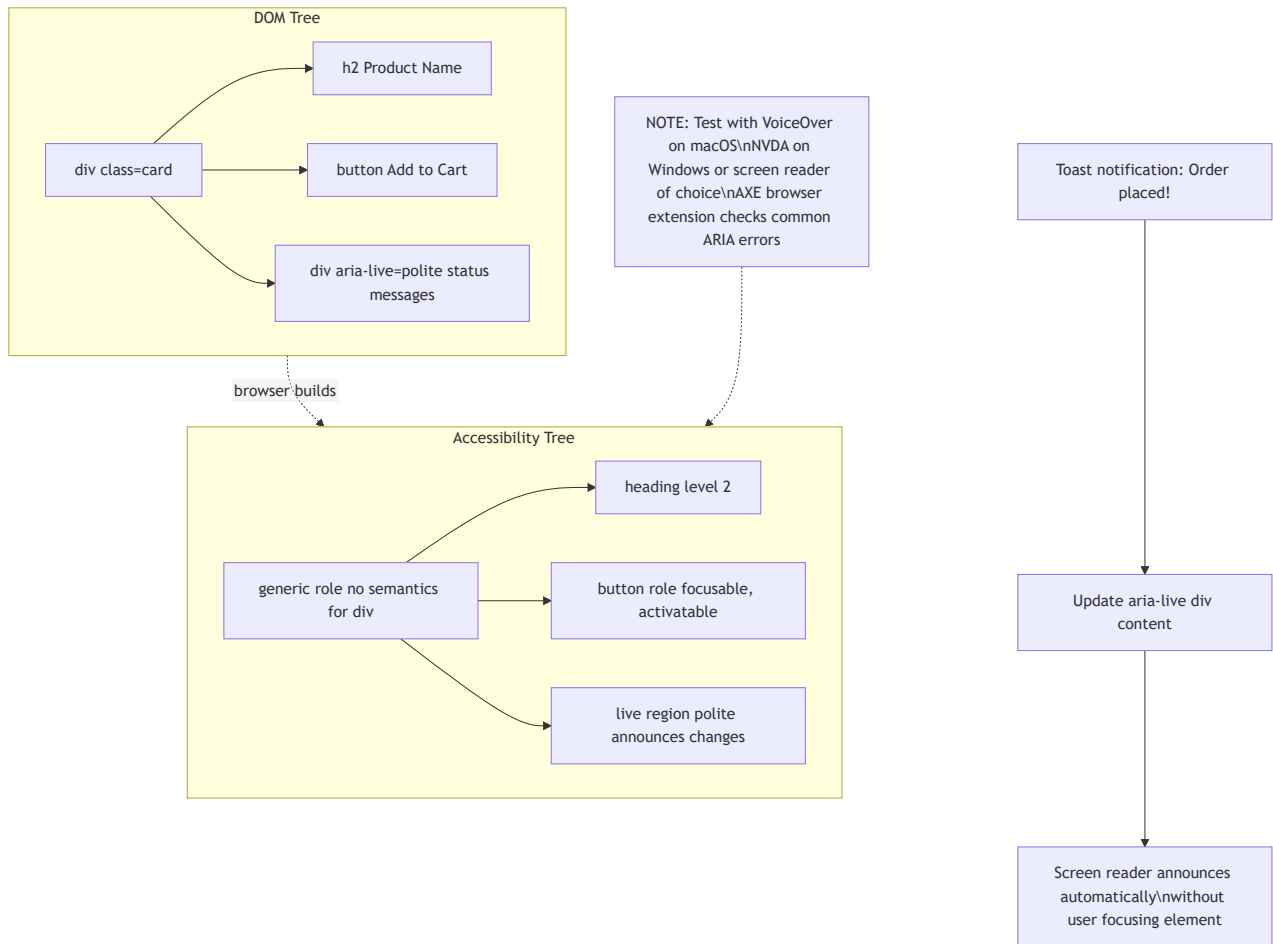
AbortController, Streaming Fetch, and Memory Leaks

AbortController is the standard cancellation mechanism for async operations. You pass its `signal` to `fetch()`; calling `abort()` causes the fetch to reject with `AbortError`. The same signal can cancel multiple operations simultaneously. This is essential for preventing race conditions and for component cleanup on unmount. **Streaming fetch** lets you process responses before they are fully received — `response.body` is a `ReadableStream`. Read chunks with a reader as they arrive, perfect for streaming LLM responses. Browser memory leaks come from: event listeners not removed on unmount, closures capturing large objects, and **detached DOM nodes** — elements removed from the document but still referenced in JavaScript.



Accessibility Tree and ARIA Live Regions

Every DOM element has a corresponding node in the **accessibility tree** — a parallel structure consumed by screen readers and assistive technology. The tree is built from semantic HTML and ARIA attributes. A `div` has no semantic role; a `button` element has `role=button` automatically. Screen readers announce roles, states (`aria-expanded`, `aria-disabled`), and labels (`aria-label`, `aria-labelledby`). **ARIA live regions** tell screen readers to announce content changes automatically without the user focusing the element. `aria-live="polite"` waits until the user finishes their current activity. `aria-live="assertive"` interrupts immediately — reserve for critical urgent messages only. Misusing assertive floods the user with interruptions.



How to Render These Diagrams

Every diagram in this document is written in **Mermaid** syntax. You can render them in several ways.

Paste any diagram block into mermaid.live for an instant interactive, editable render — no install required. GitHub and GitLab render Mermaid inside fenced code blocks marked with ```mermaid` natively, so this file will display diagrams automatically if you commit it to a repository. Notion and Confluence both support Mermaid as a built-in block type. VS Code users can install the **Mermaid Preview** or **Markdown Preview Mermaid Support** extension to see diagrams inline while editing.

For offline or CI use, the Mermaid CLI tool (`mmdc`) converts diagrams to SVG or PNG: install with `npm install -g @mermaid-js/mermaid-cli` and run `mmdc -i diagram.mmd -o diagram.svg`.

The three diagram types used in this document are `flowchart` (top-down TD and left-right LR variants for process flows and architecture), `sequenceDiagram` (for protocol and lifecycle interactions between participants), and `stateDiagram-v2` (for state machines like lock inflation and service worker lifecycle).

Generated by Claude — Anthropic. Every concept explained from first principles.